Universidad Peruana de Ciencias Aplicadas

Carrera de Ingeniería de Software

1ASI0729

Desarrollo de Aplicaciones Open Source

NRC

16692

Informe del Trabajo Final

Docente

Velásquez Núñez, Ángel Augusto

Equipo

CodeNova

Proyecto

Vigilia

Integrantes

Código	Apellidos y Nombres
u202420147	Valladolid Jiménez, Arturo Fernando
u202411843	Diaz Vargas, Fernanda Ysabella
u20241b178	Romero Veliz, Matthias Alonso

Código	Apellidos y Nombres
u202315171	Salazar Miranda, Mateo Paolo

Período 202620

Octubre 2026

Registro de Versiones del Informe

Versión	Fecha	Autor	Descripción de modificación
0.1.0	2026-09-01	Valladolid Jiménez, Arturo Fernando	Se crea la primera versión del informe coregido y actualizado a tb1 con la carátula, el registro de versiones, el Project Report Collaboration Insights y la estructura de capítulos del statement.
0.2.0	2026-09-03	Valladolid Jiménez, Arturo Fernando	Se agregan 1.1 Startup Profile (descripción de la startup y perfiles del equipo) y 1.2.1 Antecedentes y problemática.
0.3.0	2026-09-05	Diaz Vargas, Fernanda Ysabella	Se agrega 1.2.2 Lean UX Process: problem statements, assumptions, hypothesis statements y Lean UX Canvas.
0.4.0	2026-09-07	Salazar Miranda, Mateo Paolo	Se agregan 1.3 Segmentos objetivo y 2.1 Competidores, con el análisis competitivo y las estrategias frente a competidores.
0.5.0	2026-09-09	Romero Veliz, Matthias Alonso	Se agrega 2.2 Entrevistas: diseño, registro y análisis de las entrevistas de los tres segmentos.
0.6.0	2026-09-11	Romero Veliz, Matthias Alonso	Se agrega 2.3 Needfinding: User Personas, User Task Matrix, User Journey Mapping y Empathy Mapping.
0.7.0	2026-09-12	Salazar Miranda, Mateo Paolo	Se agregan 2.4 Big Picture Event Storming y 2.5 Ubiquitous Language.
0.8.0	2026-09-13	Diaz Vargas, Fernanda Ysabella	Se agrega 3.1 User Stories, con epics y acceptance criteria en formato Gherkin.
0.9.0	2026-09-14	Diaz Vargas, Fernanda Ysabella	Se agregan 3.2 Impact Mapping y 3.3 Product Backlog con story points.

Versión	Fecha	Autor	Descripción de modificación
0.10.0	2026-09-15	Valladolid Jiménez, Arturo Fernando	Se agregan 4.1 Style Guidelines, 4.2 Information Architecture y 4.3 Landing Page UI Design.
0.11.0	2026-09-16	Valladolid Jiménez, Arturo Fernando	Se agregan 4.4 Web Applications UX/UI Design y 4.5 Web Applications Prototyping.
0.12.0	2026-09-16	Valladolid Jiménez, Arturo Fernando	Se agregan 4.6 Domain-Driven Software Architecture, 4.7 Software Object-Oriented Design y 4.8 Database Design.
0.13.0	2026-09-17	Valladolid Jiménez, Arturo Fernando	Se agregan 5.1 Software Configuration Management y el planning, los aspectos y el backlog del Sprint 1 (5.2.1.1 a 5.2.1.3).
0.14.0	2026-09-17	Salazar Miranda, Mateo Paolo	Se agregan las evidencias de desarrollo, ejecución, servicios y despliegue del Landing Page en el Sprint 1 (5.2.1.4 a 5.2.1.7).
0.15.0	2026-09-18	Valladolid Jiménez, Arturo Fernando	Se agregan Student Outcome, 5.2.1.8, conclusiones, recomendaciones, referencias y el anexo con el video de exposición. Se cierra la versión de la primera entrega.
0.16.0	2026-10-01	Valladolid Jiménez, Arturo Fernando	Se migra el informe a Markdown en el repositorio project-report (docs-as-code), con un archivo por sección y exportación a PDF con Pandoc.
0.17.0	2026-10-06	Diaz Vargas, Fernanda Ysabella	Se migran y corrigen los perfiles del equipo (1.1.2), el análisis de competidores (2.1) y el diseño de entrevistas (2.2.1).

Versión	Fecha	Autor	Descripción de modificación
0.18.0	2026-10-07	Valladolid Jiménez, Arturo Fernando	Se rehacen 4.6, 4.7 y 4.8 con cinco bounded contexts: Design-Level Event Storming en Miro, diagramas C4 en Structurizr, class diagrams en PlantUML y database diagrams por contexto.
0.19.0	2026-10-07	Diaz Vargas, Fernanda Ysabella	Se migran y corrigen 2.3 Needfinding, 3.1 User Stories y 4.1 Style Guidelines.
0.20.0	2026-10-08	Valladolid Jiménez, Arturo Fernando	Se corrigen 4.6 y 4.7: se agregan los class diagrams CD-08 y CD-09, se corrige el fake API de producción y se unifican las rutas de imágenes. Se retira del informe a un integrante que dejó el curso.
0.21.0	2026-10-08	Salazar Miranda, Mateo Paolo	Se corrige 2.5 Ubiquitous Language: umbral por equipo y métrica, cuatro niveles de severidad y términos nuevos del modelo.
0.22.0	2026-10-08	Diaz Vargas, Fernanda Ysabella	Se agregan las epics en 3.1, se fusionan US15 y US22, se cambian las notificaciones push por avisos dentro de la aplicación y por correo, y se reordena 3.3 por valor de negocio con IAM al final.
0.23.0	2026-10-08	Valladolid Jiménez, Arturo Fernando	Se corrige 5.1 con las herramientas, repositorios y despliegues actuales, y 5.2.1 con el estado real de las tareas del Sprint 1.
0.24.0	2026-10-08	Valladolid Jiménez, Arturo Fernando	Se agrega 5.2.2 Sprint 2 con planning, aspectos, backlog y evidencias de la Frontend Web Application.
0.25.0	2026-10-08	Valladolid Jiménez, Arturo Fernando	Se actualizan el registro de versiones, el Project Report Collaboration Insights, el Student Outcome y el anexo de videos para la segunda entrega.

Project Report Collaboration Insights

URL del repositorio del informe: https://github.com/code-nova-1asi0729/project-report

AV1. Para la primera entrega repartimos las secciones según el rol de cada integrante. Escribimos el informe en un documento compartido y subimos avances al repositorio sin una estructura definida. El líder del equipo consolidó el documento antes de la entrega.

Integrante	Secciones a cargo
Valladolid Jiménez, Arturo Fernando	Carátula, registro de versiones, collaboration insights y Student Outcome. 1.1 Startup Profile y 1.2.1 Antecedentes y problemática. Capítulo IV completo: style guidelines, arquitectura de información, diseño del Landing Page y de la Web Application, prototipo, diagramas C4, class diagrams y database diagrams. 5.1 y la gestión del Sprint 1 (planning, aspectos, backlog y collaboration insights).
Diaz Vargas, Fernanda Ysabella	1.2.2 Lean UX Process. Capítulo III: user stories, impact mapping y product backlog.
Romero Veliz, Matthias Alonso	2.2 Entrevistas y 2.3 Needfinding: user personas, user task matrix, journey maps y empathy maps.
Salazar Miranda, Mateo Paolo	1.3 Segmentos objetivo, 2.1 Competidores, 2.4 Big Picture Event Storming, 2.5 Ubiquitous Language y las evidencias del Landing Page en el Sprint 1.

TB1. Para esta entrega pasamos el informe a Markdown, en un repositorio con un archivo por sección. Aplicamos GitFlow: cada grupo de secciones se trabaja en una rama `feature` desde `develop` y entra con un Pull Request. Los commits siguen Conventional Commits con el capítulo como scope, por ejemplo `docs(chapter-5): add sprint 2 backlog`. El PDF se genera con Pandoc desde el repositorio.

Contenido

Tabla de Contenido

Registro de Versiones del Informe 3

Project Report Collaboration Insights 6

Contenido 7

Student Outcome 11

Capítulo I: Introducción 16
1.1. Startup Profile . . . 16
1.1.1. Descripción de la Startup . . . 16
1.1.2. Perfiles de integrantes del equipo . . . 16
1.2. Solution Profile . . . 19
1.2.1. Antecedentes y problemática . . . 19
1.2.2. Lean UX Process . . . 26
1.2.2.1. Lean UX Problem Statements . . . 26
1.2.2.2. Lean UX Assumptions . . . 27
1.2.2.3. Lean UX Hypothesis Statements . . . 29
1.2.2.4. Lean UX Canvas . . . 30
1.3. Segmentos objetivo . . . 30

Capítulo II: Requirements Elicitation & Analysis 32
2.1. Competidores . . . 32
2.1.1. Análisis competitivo . . . 33
2.1.2. Estrategias y tácticas frente a competidores . . . 40
2.2. Entrevistas . . . 41
2.2.1. Diseño de entrevistas . . . 41
2.2.2. Registro de entrevistas . . . 42
Segmento #1: Administradores de edificios y condominios . . . 43
Segmento #2: Propietarios y residentes . . . 46
Segmento #3: Empresas de mantenimiento . . . 49
2.2.3. Análisis de entrevistas . . . 53
Segmento #1: Administradores de edificios y condominios . . . 53
Segmento #2: Propietarios y residentes . . . 54
Segmento #3: Empresas de mantenimiento . . . 56

Conclusiones del análisis . . . 57
2.3. Needfinding . . . 57
2.3.1. User Personas . . . 58
2.3.2. User Task Matrix . . . 58
2.3.3. User Journey Mapping . . . 59
2.3.4. Empathy Mapping . . . 59
2.4. Big Picture Event Storming . . . 59
2.5. Ubiquitous Language . . . 59

Capítulo III: Requirements Specification **63**
3.1. User Stories . . . 63
3.2. Impact Mapping . . . 113
3.3. Product Backlog . . . 113

Capítulo IV: Product Design **138**
4.1. Style Guidelines . . . 138
4.1.1. General Style Guidelines . . . 138
4.1.2. Web Style Guidelines . . . 140
4.2. Information Architecture . . . 141
4.2.1. Organization Systems . . . 141
4.2.2. Labeling Systems . . . 143
4.2.3. SEO Tags and Meta Tags . . . 144
4.2.4. Searching Systems . . . 145
4.2.5. Navigation Systems . . . 146
4.3. Landing Page UI Design . . . 146
4.3.1. Landing Page Wireframe . . . 146
4.3.2. Landing Page Mock-up . . . 149
4.4. Web Applications UX/UI Design . . . 151
4.4.1. Web Applications Wireframes . . . 151
4.4.2. Web Applications Wireflow Diagrams . . . 154
4.4.3. Web Applications Mock-ups . . . 159
4.4.4. Web Applications User Flow Diagrams . . . 162
4.5. Web Applications Prototyping . . . 168
4.6. Domain-Driven Software Architecture . . . 168
4.6.1. Design-Level Event Storming . . . 169
4.6.2. Software Architecture Context Diagram . . . 177
4.6.3. Software Architecture Container Diagrams . . . 178
4.6.4. Software Architecture Components Diagrams . . . 180

4.7. Software Object-Oriented Design . . . 182
4.7.1. Class Diagrams . . . 182
CD-01. Asset Monitoring . . . 182
CD-02. Incidents . . . 183
CD-03. Maintenance . . . 184
CD-04. Notifications . . . 184
CD-05. IAM . . . 185
CD-06. Web Application: Shared . . . 187
CD-07. Web Application: Asset Monitoring (domain y application) . . . 187
CD-08. Web Application: Asset Monitoring (infrastructure y presentation) . . . 188
CD-09. Web Application: Incidents . . . 189
4.8. Database Design . . . 191
4.8.1. Database Diagrams . . . 191
4.8.1.1. Asset Monitoring . . . 191
4.8.1.2. Incidents . . . 193
4.8.1.3. Maintenance . . . 195
4.8.1.4. Notifications . . . 197
4.8.1.5. IAM . . . 197

Capítulo V: Product Implementation, Validation & Deployment 199
5.1. Software Configuration Management . . . 199
5.1.1. Software Development Environment Configuration . . . 199
5.1.2. Source Code Management . . . 201
5.1.3. Source Code Style Guide & Conventions . . . 202
5.1.4. Software Deployment Configuration . . . 203
5.2. Landing Page, Services & Applications Implementation . . . 204
5.2.1. Sprint 1 . . . 204
5.2.1.1. Sprint Planning 1 . . . 205
5.2.1.2. Aspect Leaders and Collaborators . . . 205
5.2.1.3. Sprint Backlog 1 . . . 206
5.2.1.4. Development Evidence for Sprint Review . . . 210
5.2.1.5. Execution Evidence for Sprint Review . . . 211
5.2.1.6. Services Documentation Evidence for Sprint Review . . . 214
5.2.1.7. Software Deployment Evidence for Sprint Review . . . 215
5.2.1.8. Team Collaboration Insights during Sprint . . . 215
5.2.2. Sprint 2 . . . 216
5.2.2.1. Sprint Planning 2 . . . 216
5.2.2.2. Aspect Leaders and Collaborators . . . 218

5.2.2.3. Sprint Backlog 2 219
5.2.2.4. Development Evidence for Sprint Review 230
5.2.2.5. Execution Evidence for Sprint Review 236
5.2.2.6. Services Documentation Evidence for Sprint Review 247
5.2.2.7. Software Deployment Evidence for Sprint Review 252
5.2.2.8. Team Collaboration Insights during Sprint 254

6. Conclusiones **255**

7. Recomendaciones **255**

8. Referencias bibliográficas **256**

Anexos **257**
Anexo A. Videos de Exposiciones 257
Video de Exposición – AV1 257
Video de Exposición – TB1 257

Student Outcome

El curso contribuye al cumplimiento del Student Outcome ABET: ABET – EAC - Student Outcome 3

Criterio: Capacidad de comunicarse efectivamente con un rango de audiencias.

En el siguiente cuadro se describe las acciones realizadas y enunciados de conclusiones por parte del grupo, que permiten sustentar el haber alcanzado el logro del ABET – EAC - Student Outcome 3.

Criterio específico	Acciones realizadas	Conclusiones
Comunica oralmente con efectividad a diferentes rangos de audiencia.	**Valladolid Jiménez, Arturo Fernando**	**AV1.** Hablamos con administradores, residentes y técnicos de mantenimiento, cada uno con su propio vocabulario. Aprendimos a cambiar la forma de preguntar según la persona y a confirmar lo que entendimos antes de convertirlo en requisitos. Dentro del equipo, explicar en voz alta cada decisión evitó que cada uno interpretara las historias a su manera.

Criterio específico	Acciones realizadas	Conclusiones
	AV1. En las reuniones del equipo expliqué cómo estaba organizado el repositorio y el flujo de trabajo en Git, y qué decisiones técnicas tomamos y por qué.	**TB1.** En el Sprint 2 la comunicación oral fue sobre todo técnica. Explicar el código a un compañero, con un ejemplo funcionando, fue más útil que cualquier documento. En el planning y en la revisión de Pull Requests aprendimos a decir qué cambia y por qué en pocas palabras, y a pedir ayuda antes de bloquearnos.
	TB1. En el Sprint Planning 2 expliqué por qué movimos IAM al final del backlog y cómo se dividía la Web Application por bounded context. Luego hice una demostración en vivo de la vista de edificios, para que cada integrante viera el camino que tenía que seguir en su tarea. En el video de TB1 presento la arquitectura y el despliegue.	
	Diaz Vargas, Fernanda Ysabella	
	AV1. Expliqué al equipo cómo cada necesidad de las entrevistas se conectaba con una user story y con un objetivo de negocio. Usé ejemplos concretos para que diseño y desarrollo entendieran el alcance de cada historia antes de pasarla al backlog.	
	TB1. En el Sprint Planning 2 presenté el nuevo orden del Product Backlog y respondí las dudas del equipo sobre qué historias entraban al sprint. En el video de TB1 explico la vista de sensores y el historial de lecturas.	
	Romero Veliz, Matthias Alonso	

Criterio específico	Acciones realizadas	Conclusiones
	AV1. En el video de exposición expliqué el diseño de las entrevistas para los tres segmentos y lo que encontramos. Al profesor le expliqué el sustento de cada pregunta; al equipo le dije de forma directa qué hallazgos debían usar en el diseño. **TB1.** Expliqué al equipo cómo quedó el modelo de equipos críticos y por qué su Pull Request tenía que entrar primero, ya que las alertas y los sensores dependen de él. En el video de TB1 muestro la gestión de equipos críticos. **Salazar Miranda, Mateo Paolo** **AV1.** Expliqué al equipo el análisis de competidores y qué oportunidades encontramos frente a ellos. También recorrí con el equipo los eventos del Big Picture Event Storming para que todos entendiéramos el dominio de la misma forma. **TB1.** Expliqué al equipo cómo funciona el nuevo bounded context de Incidents y qué cambió en el Landing Page. En el video de TB1 muestro el flujo completo de un incidente: reporte, gestión, resolución y calificación.	
Comunica por escrito con efectividad a diferentes rangos de audiencia.	**Valladolid Jiménez, Arturo Fernando**	**AV1.** Escribimos para públicos distintos: textos simples y persuasivos para los visitantes del Landing Page, artefactos con formato definido para requisitos (Lean UX, Gherkin) y especificaciones técnicas (UML, C4) para quienes implementan.

Criterio específico	Acciones realizadas	Conclusiones
	AV1. Redacté la Student Outcome, el Project Report Collaboration Insights y la documentación del Sprint 1. Tuve que convertir actividades técnicas (configuración del entorno, commits, reporte de desempeño) en un texto que el profesor y el equipo pudieran seguir.	**TB1.** Pasar el informe a Markdown nos obligó a escribir más corto y a dejar cada decisión en el lugar donde se usa. Los mensajes de commit y las descripciones de Pull Request también son comunicación escrita: con Conventional Commits, cualquiera entiende qué cambió sin abrir el código.
	TB1. Reescribí las secciones de arquitectura (4.6 a 4.8) y el capítulo V con textos cortos y diagramas legibles. También escribí la guía de tareas del frontend (`docs/sprint-2-tasks.md`), que dice a cada integrante qué clases crear, en qué orden y cuándo su tarea está terminada.	
	Diaz Vargas, Fernanda Ysabella	
	AV1. Redacté los artefactos de Lean UX y la especificación de requisitos: user stories por epic, acceptance criteria en Gherkin, impact mapping y product backlog. Cuidé que los criterios de aceptación no tuvieran detalles de interfaz.	
	TB1. Migré a Markdown y corregí los capítulos I a III y las style guidelines. En las user stories fusioné las historias de umbrales que se contradecían y reescribí las de notificaciones para que coincidan con el modelo.	
	Romero Veliz, Matthias Alonso	

Criterio específico	Acciones realizadas	Conclusiones
	AV1. Redacté el análisis de las entrevistas, los User Personas, el User Task Matrix, el User Journey Mapping y el Empathy Mapping. En el análisis usé citas de los entrevistados para sustentar cada hallazgo. **TB1.** Escribí los commits y las descripciones de los Pull Requests de equipos críticos con Conventional Commits, para que el equipo supiera qué revisar antes de hacer el merge. **Salazar Miranda, Mateo Paolo** **AV1.** Redacté los segmentos objetivo, el análisis de competidores, el Big Picture Event Storming y el Ubiquitous Language, y documenté las evidencias del Landing Page en el Sprint 1. **TB1.** Corregí el Ubiquitous Language para que use los mismos términos que los diagramas y el código, y agregué los términos nuevos del modelo, como intervention, savings projection y bounded context.	

Capítulo I: Introducción

1.1. Startup Profile

1.1.1. Descripción de la Startup

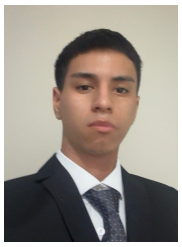
CodeNova es una startup de tecnología dedicada al mantenimiento preventivo de edificios de departamentos y condominios mediante sensores IoT. El equipo crítico de un edificio, como los aires acondicionados, las bombas de agua o los tableros eléctricos, suele realizarse sólo cuando falla. Para entonces la reparación ya es una urgencia: cuesta más que una intervención planificada, interrumpe a los residentes y, en el peor de los casos, compromete la estructura. CodeNova nace para invertir ese orden y detectar el desgaste mientras todavía es corregible.

Nuestro producto, Vigilia, es una plataforma web que concentra el monitoreo y la gestión del mantenimiento de un edificio. Sensores instalados en los equipos miden vibración, temperatura de componentes, humedad y consumo eléctrico; cuando una lectura se sale del rango esperado, la plataforma genera una alerta temprana con su severidad. Sobre esa base se organiza el trabajo del operador: recepción de solicitudes, planificación de visitas, agenda y control de materiales. Los administradores acceden a un dashboard con el estado del edificio y el ahorro acumulado frente al mantenimiento correctivo; los residentes reportan incidentes y siguen el avance de sus solicitudes; las empresas de mantenimiento reciben las alertas priorizadas y ordenan su semana de trabajo. El servicio se oferta bajo una cuota fija mensual por edificio, de modo que los costos de mantenimiento se mantienen en un rango predecible y se reduce el riesgo de fallas graves.

1.1.2. Perfiles de integrantes del equipo

Foto	Integrante	Código	Perfil
	Valladolid Jiménez, Arturo Fernando	u202420147	Soy estudiante de Ingeniería de Software en quinto ciclo. En CodeNova soy el Team Leader: coordino los sprints, organizo el repositorio y el flujo de trabajo en Git, y me encargo de consolidar los informes de cada entrega. Vengo de cursar Algoritmos y Estructura de Datos, Diseño y Patrones de Software, Especificación y Análisis de Requerimientos, Diseño de Base de Datos y Arquitectura de Computadoras, formación que me deja una base para leer y construir diseño de software, no solo código. Manejo HTML, CSS y C++. Cursos como Organización y Dirección de Empresas y Creatividad y Liderazgo me ayudaron con la parte de coordinación, que es la que más pesa en mi rol. En este proyecto quiero sostener el ritmo del equipo y, de paso, dominar Spring Boot y Angular para aportar también en la implementación.

Foto	Integrante	Código	Perfil
	Romero Veliz, Matthias Alonso	u20241b178	Soy estudiante de Ingeniería de Software en quinto ciclo. En este proyecto planeo aportar de forma transversal a lo largo de todo el ciclo de vida del proyecto: desde el diseño y análisis de las entrevistas de Needfinding, la construcción de los User Personas, User Journey Maps y Empathy Maps, hasta la especificación de los User Stories, el Product Backlog y las decisiones de Product Design (Style Guidelines, arquitectura de información y UI del Landing Page y la Web Application). Cuento con conocimientos en diseño de software, algoritmos y estructuras de datos, y desarrollo web, lo que me permite moverme con soltura entre las etapas de investigación de usuarios, especificación de requisitos y diseño de producto. En este proyecto busco seguir fortaleciendo esa visión integral del ciclo de desarrollo, aportando tanto en la documentación como en la implementación técnica de CodeNova.

Foto	Integrante	Código	Perfil
	Diaz Vargas, Fernanda Ysabella	u202411843	Soy estudiante de Ingeniería de Software en el sexto ciclo. En este trabajo CodeNova, me encargué del análisis y levantamiento de requisitos, liderando el Capítulo II. Desarrollé el análisis competitivo, la ejecución de entrevistas y la elaboración de los User Personas, User Journey Maps, Empathy Maps y el Event Storming. Mi experiencia en modelado de procesos y especificación de requisitos permitió traducir las necesidades de los usuarios en las bases del diseño de nuestro producto.
	Salazar Miranda, Mateo Paolo	u202315171	[COMPLETAR: carrera y ciclo.] Soy un estudiante responsable y comprometido, con interés en el desarrollo de soluciones tecnológicas innovadoras. Tengo habilidades de trabajo en equipo, pensamiento analítico y resolución de problemas.

1.2. Solution Profile

1.2.1. Antecedentes y problemática

Antecedentes

El mercado de vivienda de Lima Metropolitana atraviesa su mejor momento en más de una década: las ventas crecieron 23.3 % en 2024, el mejor resultado en once años y el cuarto mejor en los veintinueve años que CAPECO lleva realizando esa investigación (CAPECO 2025). La oferta se concentra en viviendas multifamiliares, al punto de que el propio Estado justificó la actualización de su marco normativo en las "nuevas necesidades de las viviendas multifamiliares" (Decreto Legislativo N.° 1568 2023). Cada uno de esos inmuebles funciona porque una serie de equipos críticos opera todos los días: bombas hidroneumáticas que suben el agua a los pisos altos, ascensores, aires acondicionados y tableros eléctricos.

Esa convivencia se rige por el Decreto Legislativo 1568, publicado en mayo de 2023, que entra en vigencia a los 180 días de publicado su reglamento: los propietarios están obligados a aportar las cuotas necesarias para el mantenimiento y conservación de los bienes comunes, incluidas las instalaciones sanitarias y eléctricas de uso común, y el administrador que designa la junta de propietarios se encarga de cobrar y recaudar esas cuotas de los gastos comunes (Decreto Legislativo N.° 1568 2023).

En la práctica, ese dinero se gasta casi siempre después del daño. Una administradora entrevistada por el equipo estima que cerca del 70 % de su presupuesto de mantenimiento termina siendo gasto correctivo no planificado (Entrevista 2, sección 2.2.2), y una empresa de mantenimiento contrasta una visita programada de una a dos horas (hasta S/ 300) con una reparación de emergencia por cambio de motor que toma un día completo y cuesta entre S/ 2,500 y S/ 4,000, sin contar el sobrecosto del repuesto urgente (sección 2.2.2). Como referencia de planificación, el MEF asigna al mantenimiento preventivo del equipamiento un porcentaje anual de entre 1 % y 3 % de su valor, y al correctivo un 8 % que se aplica de forma periódica según la vida útil del equipo (Ministerio de Economía y Finanzas 2022). Esa estimación fue diseñada para inversiones públicas de salud y educación y no compara costos reales en edificios residenciales, por lo que el equipo la complementa con la evidencia de sus entrevistas. En el sector privado limeño la secuencia es similar: las empresas de mantenimiento de bombas describen que, cuando no hay prevención, el tanque hidroneumático falla, la bomba entra en ciclo corto, el consumo eléctrico se dispara y el motor termina quemado. Antes de eso, la presión ya se había perdido en los pisos altos y las quejas de los residentes ya habían llegado a la administración (Samiria Soluciones 2025). El chequeo mensual que evita esa cadena existe, y consiste justamente en medir vibraciones y amperaje.

El riesgo de incendio y el consumo eléctrico son la otra cara del problema. INDECI registró 11,854 incendios urbanos e industriales en el país entre 2015 y 2024 (Instituto Nacional de Defensa Civil 2025), cifra que da la magnitud del riesgo de incendio en entornos urbanos y que ningún administrador puede anticipar sin información sobre el estado de los tableros y circuitos de su edificio. A eso se suma un consumo que nadie examina: la Guía de Orientación del Uso Eficiente de la Energía del MINEM estima que una vivienda en multifamiliar consume en promedio 7,859 kWh al año, que la refrigeración concentra alrededor del 41 % del consumo final de electricidad del sector residencial y que la iluminación

le sigue con 17 % (Ministerio de Energía y Minas 2022). Para los multifamiliares, la propia guía indica que además del consumo de cada vivienda debe considerarse el consumo eléctrico de las áreas comunes. Hoy, la boleta mensual es la única señal que tiene un administrador sobre el comportamiento de los equipos del edificio.

Problemática

La gestión del mantenimiento en edificios de departamentos de Lima se hace a ciegas. El administrador no conoce el estado de los equipos, solo el monto facturado del último mes. El residente descubre los problemas cuando el agua no sube o el aire acondicionado deja de enfriar. La empresa de mantenimiento se entera por llamada de emergencia, cuando la reparación ya es la opción más cara. Las cuotas que la ley obliga a pagar terminan financiando emergencias en lugar de prevenirlas.

Enunciado del problema

El mantenimiento de los equipos críticos de un edificio multifamiliar (bombas hidroneumáticas, tableros eléctricos, ascensores y aires acondicionados) debería sostener el servicio diario de los residentes con un gasto previsible, financiado por las cuotas que los propietarios están obligados a aportar (Decreto Legislativo N.° 1568 2023). En los edificios de Lima Metropolitana ese objetivo no se cumple: administradores, residentes y empresas de mantenimiento no disponen de información objetiva sobre el estado real de los equipos, las intervenciones suelen ocurrir después de la falla y las cuotas terminan destinándose a emergencias correctivas. Se requiere mejorar la oportunidad con la que las decisiones de mantenimiento se toman y se coordinan entre los tres actores. ¿Cómo lograr que administradores, residentes y empresas de mantenimiento actúen sobre el desgaste de los equipos antes de que se convierta en una emergencia?

Puntos que debe resolver la solución propuesta

- Dar visibilidad del estado de los equipos críticos mediante variables medibles: vibración, temperatura, humedad y consumo eléctrico.
- Detectar el desgaste a tiempo, con alertas tempranas clasificadas por severidad.
- Conectar en un mismo flujo de trabajo a los tres actores: el administrador decide y programa visitas, el residente reporta incidentes y sigue su avance, y la empresa de mantenimiento recibe alertas priorizadas y ordena su agenda.
- Conservar un historial centralizado por equipo que permita al administrador sustentar el gasto de las cuotas ante la junta de propietarios.
- Hacerlo bajo un modelo sostenible de cuota fija mensual por edificio.

  Objetivos

Objetivo general.

Diseñar y construir un MVP web, compuesto por un Landing Page, una Web Application y un RESTful API de elaboración interna, que permita a administradores, residentes y empresas de mantenimiento conocer el estado de los equipos críticos de un edificio multifamiliar de Lima Metropolitana y actuar sobre alertas tempranas, con el fin de validar la propuesta de valor de CodeNova con un edificio piloto.

Objetivos específicos.

1. Ofrecer al administrador un dashboard de alertas por severidad construido sobre lecturas simuladas de vibración, temperatura, humedad y consumo eléctrico.
2. Permitir al residente reportar incidentes en áreas comunes con evidencia fotográfica y consultar el estado de su solicitud.
3. Permitir a la empresa de mantenimiento recibir alertas con contexto técnico previo, confirmar visitas y registrar los resultados de cada intervención.
4. Centralizar el historial de mantenimiento por equipo para el uso de los tres actores.
5. Desplegar el Landing Page, la Web Application y el RESTful API en plataformas cloud con despliegue automatizado.
6. Medir el éxito de la propuesta con los indicadores definidos en las Lean UX Hypothesis Statements.

Restricciones y alcance

- **Productos incluidos:** un sitio web estático como Landing Page, una Web Application adaptable a las dimensiones del dispositivo e integrada con un RESTful API de elaboración interna. La experiencia debe ser consistente entre el Landing Page y la Web Application, y los call-to-action de cada segmento objetivo deben redirigir a la vista correspondiente.
- **Servicios externos:** la solución debe integrar al menos un servicio de terceros además del API propio. El diagrama de contexto identifica dos sistemas externos: la red de sensores IoT y un servicio de almacenamiento de medios en la nube.
- **Datos de sensores:** el MVP trabaja con lecturas simuladas. La instalación de sensores reales queda fuera del alcance académico y corresponde a un piloto posterior.
- **Tecnología:** servidor basado en Java y tecnologías open-source, con despliegue en plataformas server-side o cloud.
- **Ámbito:** edificios de departamentos y condominios de Lima Metropolitana, y los tres perfiles de usuario descritos en la sección 1.3.

Técnica de las 5W y 2H

Para conocer aún más la problemática usaremos la técnica de las 5W y 2H.

What (¿Qué? / ¿Cuál?)

¿Cuál es el problema? El problema es la gestión reactiva del mantenimiento en edificios multifamiliares de Lima: administradores, residentes y empresas de mantenimiento toman decisiones a ciegas, sin información objetiva sobre el estado real de los equipos críticos (bombas, tableros eléctricos, ascensores, aires acondicionados). Como consecuencia, las cuotas de mantenimiento que la ley obliga a pagar terminan financiando emergencias correctivas —más costosas y disruptivas— en lugar de prevenir fallas mientras aún son corregibles.

¿Qué soluciones existen actualmente? Existen soluciones CMMS/EAM con capacidades IoT como Fracttal One y DimoMaint, orientadas a operaciones industriales y portafolios corporativos, no a la gestión residencial de condominios; y plataformas locales de administración de edificios como OORB, que cubren facturación, cobranza y seguridad, pero sin ningún componente de monitoreo técnico de equipos (ver sección 2.1). Ninguna de ellas cubre el cruce específico que necesita un condominio residencial en Lima. Con nuestra propuesta de valor buscamos cerrar esa brecha mediante sensores IoT de bajo costo que generan alertas priorizadas por severidad, conectando en un mismo flujo de trabajo a los tres actores del ecosistema, bajo un modelo sostenible de cuota fija mensual por edificio.

¿Cuál es la relación con el usuario? El usuario es el eje central de la plataforma: el administrador la usa para decidir cuándo intervenir un equipo y sustentar el gasto ante la junta; el residente la usa para reportar incidentes y confiar en que su cuota previene fallas; y la empresa de mantenimiento la usa para priorizar su semana de trabajo con contexto técnico previo, lo que aporta mayor eficiencia y confianza a todo el ecosistema.

Why (¿Por qué?)

¿Cuál es la causa principal del problema? Consideramos que, si bien existen varios factores (falta de presupuesto, urgencia operativa, desconocimiento técnico), todos confluyen en una misma causa principal: no existe ningún canal que conecte el estado real de los equipos críticos con las decisiones de mantenimiento. Este supuesto lo contrastaremos en las Lean UX Assumptions (sección 1.2.2.2). El Decreto Legislativo 1568 obliga a los propietarios a aportar cuotas y encarga al administrador su cobro y ejecución (Decreto Legislativo N.° 1568 2023), pero esa obligación legal no viene acompañada de ninguna herramienta que indique en qué estado están realmente esos equipos, por lo que las decisiones terminan basándose en la última queja recibida y no en el desgaste real.

Who (¿Quién?)

¿Quiénes están involucrados? Está involucrada toda la cadena de gestión del edificio: administradores de propiedad horizontal, residentes y propietarios, empresas y técnicos de mantenimiento, las juntas de propietarios que aprueban los gastos, y de forma indirecta el Estado como regulador a través del Decreto Legislativo 1568.

¿A quiénes les sucede el problema? A administradores de edificios multifamiliares en Lima Metropolitana que gestionan uno o varios edificios con poco tiempo disponible; a residentes y propietarios que pagan una cuota sin visibilidad de en qué se usa; y a empresas de mantenimiento que hoy operan mayormente bajo demanda de emergencia.

When (¿Cuándo?)

¿Cuándo sucede el problema? Constantemente, y de forma creciente: el mercado de vivienda multifamiliar en Lima atraviesa su mejor momento en once años, con un crecimiento de ventas de 23.3 % en 2024 (CAPECO 2025), lo que multiplica la cantidad de edificios operando bajo esta misma lógica reactiva. Además, el Decreto Legislativo 1568 exige formalmente el cobro de estas cuotas, sin ofrecer ninguna herramienta de gestión preventiva que las acompañe.

¿Cuándo el cliente usa el producto? El administrador revisa la plataforma como parte de su rutina semanal de gestión; el residente la usa puntualmente al detectar un problema o notar una falla en las áreas comunes; y la empresa de mantenimiento la consulta antes de salir a cada visita para priorizar su ruta de la semana.

Where (¿Dónde?)

¿Dónde está el usuario cuando usa la plataforma? El administrador la usa desde su oficina o celular mientras gestiona uno o varios edificios; el residente la usa desde su propio departamento al notar una falla; y el técnico de mantenimiento la consulta en campo, antes o durante la visita al edificio.

¿Dónde surge el problema? En los edificios multifamiliares de Lima Metropolitana, específicamente en las zonas donde operan los equipos críticos: cuartos de bombas hidroneumáticas, tableros eléctricos, cuartos de máquinas de ascensores y azoteas con unidades de aire acondicionado.

How (¿Cómo?)

¿En qué condiciones los clientes usan nuestro producto? Los administradores lo usan cuando reciben una alerta y deben decidir si programar una visita preventiva o cuando necesitan sustentar gastos ante la junta; los residentes cuando notan una falla en un área común; y las empresas de mantenimiento cuando organizan su agenda semanal o llegan a una visita y necesitan contexto técnico previo.

¿Cómo se enteran de la aplicación? A través de alianzas con empresas de mantenimiento locales (que recomiendan la plataforma a los administradores que atienden), un programa de edificios fundadores con beneficios de instalación, y contenido educativo dirigido a juntas de propietarios sobre sus obligaciones bajo el Decreto Legislativo 1568.

How much (¿Cuánto?)

¿Cuánto le cuesta este problema a la economía, sociedad o institución del Perú actualmente? El costo no está medido a nivel nacional para edificios residenciales; el equipo lo aproxima con tres evidencias. Primero, la experiencia directa de los actores: una emergencia con cambio de motor cuesta entre S/ 2,500 y S/ 4,000 frente a una visita programada de hasta S/ 300, y una administradora entrevistada estima que cerca del 70 % de su presupuesto de mantenimiento es gasto correctivo no planificado (sección 2.2.2). Segundo, el riesgo de incendio, con 11,854 incendios urbanos e industriales registrados en el país entre 2015 y 2024 (Instituto Nacional de Defensa Civil 2025). Tercero, un consumo eléctrico residencial —concentrado en refrigeración (41 %) e iluminación (17 %)— que hoy nadie monitorea a nivel de edificio (Ministerio de Energía y Minas 2022). Como referencia de planificación, el MEF presupuesta el mantenimiento correctivo del equipamiento en 8 % de su valor, aplicado periódicamente, frente a 1 %–3 % anual del preventivo (Ministerio de Economía y Finanzas 2022).

¿Cuánto costaría implementar la solución propuesta? La estimación siguiente corresponde a llevar la solución a un piloto real en un edificio. En el marco del curso, el desarrollo lo aporta el propio equipo y el MVP usa lecturas simuladas, por lo que el desembolso efectivo se limita sobre todo al hosting. Los montos están en soles, no incluyen IGV y convierten los precios en dólares a S/ 3.4 por US$.

Rubro	Estimación	Base del cálculo
Desarrollo del MVP (Landing Page, Web Application y RESTful API, incluida la integración con servicios externos)	S/ 18,000 – S/ 35,000	Entre 5 y 10 meses-persona a unos S/ 3,560 mensuales por desarrollador: sueldo promedio de desarrollador de software en Perú (S/ 2,543) más cerca de 40 % en beneficios laborales.
Sensores y gateway para un edificio piloto	S/ 2,300 – S/ 5,700 por edificio	3 a 4 sensores de vibración y temperatura (USD 73–203 c/u), 2 sensores ambientales de temperatura y humedad (USD 39–52 c/u), 3 a 4 pinzas de corriente (US$ 84–105 c/u) y 1 gateway LoRaWAN (USD 130–350). Precios de lista de fabricantes, sin envío ni aranceles.
Diseño UI/UX	S/ 2,500 – S/ 6,000	Estimación del equipo para el alcance del MVP.

Rubro	Estimación	Base del cálculo
Hosting e infraestructura cloud	S/ 1,400 – S/ 2,700 al año	API en Render (US$ 7 a 25 al mes), base de datos PostgreSQL gestionada (USD 7 a 20 al mes), frontend en Vercel Pro (USD 20 al mes) y Landing Page en GitHub Pages sin costo. No incluye el dominio.
Seguridad y soporte	S/ 2,000 – S/ 5,000	Estimación del equipo: equivale a entre 0.6 y 1.4 meses-persona de refuerzo de seguridad y soporte inicial.

Con estos rubros, el costo del primer año para un edificio piloto se ubica entre unos S/ 26,000 y S/ 54,000, sin contar la instalación.

1.2.2. Lean UX Process

1.2.2.1. Lean UX Problem Statements

CodeNova busca construir una plataforma de mantenimiento preventivo basada en sensores IoT para edificios de departamentos y condominios en Lima Metropolitana, con el fin de que administradores, residentes y empresas de mantenimiento dejen de operar a ciegas frente al desgaste de los equipos críticos del edificio. La idea central es reemplazar la lógica “se revisa cuando falla” por una de detección temprana, en la que cada bomba hidroneumática, tablero eléctrico, ascensor o unidad de aire acondicionado transmita su propio estado de salud antes de que el problema se convierta en una emergencia costosa.

El problema que abordamos es que la gestión del mantenimiento en edificios multifamiliares de Lima carece de información objetiva sobre el estado real de sus equipos, una situación agravada por el auge inmobiliario que atraviesa la ciudad: las ventas de vivienda crecieron 23.3 % en 2024, el mejor resultado en once años (CAPECO, 2025), concentradas principalmente en proyectos multifamiliares (Decreto Legislativo 1568, 2023). Desde enero de 2025, dicho decreto obliga a los propietarios a aportar cuotas para el mantenimiento de los bienes comunes, y encarga al administrador su cobro y ejecución (Decreto Legislativo 1568, 2023); sin embargo, esa obligación legal no viene acompañada de ninguna herramienta que le diga al administrador en qué estado están realmente esos equipos.

Hemos identificado que esta falta de visibilidad tiene un costo cuantificable: el propio Estado reconoce que el mantenimiento correctivo cuesta hasta 8 % del valor del equipo por año, frente a un 1 %-3 % del preventivo (Ministerio de Economía y Finanzas, 2022), y las empresas de mantenimiento de

bombas en Lima describen una secuencia recurrente —ciclo corto, sobreconsumo eléctrico, motor quemado— que un chequeo mensual básico podría evitar (Samiria Soluciones, 2025). A esto se suma un riesgo eléctrico invisibilizado: 11,854 incendios urbanos e industriales registrados en el país entre 2015 y 2024 (INDECI, 2025) y un consumo eléctrico residencial concentrado en refrigeración e iluminación que hoy nadie monitorea a nivel de edificio (MINEM, 2022).

Las alternativas actuales —hojas de cálculo, llamadas de emergencia, la boleta mensual como único indicador— resultan insuficientes porque no anticipan nada: informan del gasto después de ocurrido, no de la falla antes de que ocurra. Frente a este escenario, nuestra propuesta busca responder a la siguiente interrogante: ¿Cómo podríamos diseñar una plataforma que conecte el estado real de los equipos críticos de un edificio con las decisiones de mantenimiento de administradores, residentes y empresas de mantenimiento, para prevenir fallas antes de que se conviertan en emergencias?

1.2.2.2. Lean UX Assumptions

Para afrontar el problema del mantenimiento reactivo en edificios multifamiliares, partimos de un conjunto de supuestos sobre nuestros tres tipos de usuario y su contexto, los cuales deben validarse antes de construir la solución completa. El éxito de CodeNova dependerá de qué tan acertadas sean estas hipótesis centradas en el administrador, el residente y la empresa de mantenimiento.

Nuestro análisis del contexto de administración de edificios en Lima muestra que los administradores cuentan con la obligación legal de gestionar el mantenimiento (Decreto Legislativo 1568, 2023), pero no con herramientas para hacerlo de forma anticipada. Suponemos que existe un vacío entre lo que la ley les exige y lo que hoy pueden realmente monitorear, y que valorarán información objetiva que les permita justificar gastos ante la junta de propietarios. Consideramos que los residentes, por su parte, no necesitan ni quieren visibilidad técnica detallada de los equipos, pero sí un canal simple para reportar incidentes y ver que su cuota se traduce en prevención real, no solo en reparaciones tras la queja.

Asimismo, identificamos que la desconfianza hacia el destino de las cuotas de mantenimiento es un factor clave: los residentes no siempre saben en qué se gasta su dinero, y los administradores enfrentan reclamos cuando una falla “se pudo haber evitado”. En cuanto a las empresas de mantenimiento, creemos que no rechazan trabajar con datos de sensores de terceros si eso les permite priorizar su semana y llegar con contexto técnico a cada visita, reduciendo el tiempo de diagnóstico en campo.

Al evaluar las alternativas actuales, observamos que la mayoría de administradores usa hojas de cálculo, cuadernos de bitácora o el criterio de la empresa de mantenimiento de turno, sin ningún sistema que centralice lecturas de vibración, temperatura, humedad o consumo eléctrico. Suponemos que esta falta de datos genera decisiones basadas en la última queja recibida y no en el desgaste real del equipo.

Nuestra propuesta se diferenciará al ofrecer una plataforma que traduce lecturas de sensores IoT en alertas priorizadas por severidad, conectando a los tres actores (administrador, residente, empresa de mantenimiento) en un mismo flujo de trabajo. Creemos que, al mostrar el ahorro acumulado frente al mantenimiento correctivo, los administradores justificarán con más facilidad la cuota mensual ante la junta de propietarios, y que las empresas de mantenimiento reducirán su proporción de visitas de emergencia frente a las programadas.

1.2.2.2.1. Assumptions Worksheet

¿Quién es el usuario? Identificamos tres roles principales: - **El Administrador:** Suponemos que es la persona responsable de coordinar el mantenimiento del edificio, con poco tiempo disponible y bajo presión constante de la junta de propietarios y de los residentes. - **El Residente/Propietario:** Consideramos que es alguien que solo quiere que los servicios comunes funcionen y que su cuota se sienta justificada, sin interés en el detalle técnico. - **La Empresa de Mantenimiento:** Suponemos que es un equipo técnico que hoy trabaja mayormente de forma reactiva y valoraría llegar a cada visita con información previa del equipo a revisar.

¿Dónde encaja nuestro producto en su vida o actividades? Creemos que el administrador revisará la plataforma como parte de su rutina semanal de gestión, el residente la usará puntualmente cuando detecte un problema, y la empresa de mantenimiento la consultará antes de salir a cada visita para priorizar su ruta.

¿Qué problemas busca resolver nuestro producto? - **Problema de visibilidad:** Asumimos que la principal barrera del administrador es no saber en qué estado están los equipos hasta que fallan. - **Problema de justificación:** Consideramos que los administradores se frustran al no poder sustentar ante la junta en qué se invierten las cuotas de mantenimiento. - **Problema de priorización:** Creemos que las empresas de mantenimiento pierden eficiencia al no poder distinguir una emergencia real de una revisión rutinaria antes de llegar al edificio.

¿Cuándo y cómo se utiliza el producto? Suponemos que su uso se itensifica cuando se acerca la fecha de revisión de cuotas o tras un incidente reciente. Además, creemos que los administradores prefieren un dashboard simple que muestre alertas por severidad, sin tener que interpretar datos técnicos crudos.

¿Qué características son clave? - **Confiabilidad de las alertas:** Consideramos esencial que los administradores confíen en que una alerta “crítica” realmente lo es, para no generar fatiga de notificaciones. - **Trazabilidad:** Creemos que un historial de intervenciones y ahorro acumulado es fundamental para justificar decisiones ante la junta. - **Simplicidad para el residente**Pensamos que el reporte de incidentes debe tomar segundos, sin fricciones ni formularios largos.

¿Cómo debe ser el producto? Creemos que la experiencia debe transmitir control y prevención, no alarma constante. La plataforma debe sentirse como una herramienta de gestión seria y confiable, no como una app de quejas.

Business outcomes - Reducir la proporción de mantenimiento correctivo frente al preventivo en los edificios afiliados. - Consolidarse como la plataforma de referencia en mantenimiento preventivo IoT para condominios en Lima. - Generar ingresos recurrentes y predecibles mediante el modelo de cuota fija mensual por edificio. - Construir relaciones sostenidas con empresas de mantenimiento como canal de adopción. - Reducir la siniestralidad eléctrica y de equipos críticos en los edificios afiliados.

User outcomes - Los administradores reducen gastos imprevistos y sustentan mejor las cuotas ante la junta. - Los residentes recuperan confianza en que su cuota previene fallas, no solo las repara. - Las empresas de mantenimiento optimizan su semana de trabajo priorizando por severidad real. - Todos los actores acceden a un historial centralizado del estado del edificio.

Features -Sensores IoT de vibración, temperatura, humedad y consumo eléctrico instalados en equipos críticos. - Motor de alertas tempranas con niveles de severidad. - Dashboard administrativo con estado del edificio y ahorro acumulado. - Módulo de reporte de incidentes para residentes con seguimiento de estado. - Agenda y control de materiales para las visitas de mantenimiento. - Historial de intervenciones y lecturas por equipo.

1.2.2.3. Lean UX Hypothesis Statements

Hipótesis de Negocio

Creemos que, al mostrarle al administrador un dashboard con el ahorro acumulado frente al mantenimiento correctivo, lograremos que apruebe la suscripción mensual de CodeNova para su edificio. Validamos esta hipótesis si el 70 % de los administradores que prueban una demo solicita continuar con la suscripción, y si el 50 % de los edificios activos renueva su suscripción después de los primeros tres meses. Consideramos que, al conectar a las empresas de mantenimiento con alertas priorizadas por severidad, aumentaremos la proporción de visitas programadas frente a las de emergencia. Confirmaremos esto si, tras seis meses de uso, los edificios afiliados registran una reducción de al menos 30 % en las visitas de emergencia respecto a su historial previo. Asimismo, creemos que al centralizar el historial de mantenimiento por equipo, facilitaremos que el administrador sustente el gasto de las cuotas ante la junta de propietarios, incrementando la retención del servicio. Esta hipótesis será válida si más del 60 % de los administradores reporta, en encuestas trimestrales, que la plataforma les facilitó justificar los gastos de mantenimiento.

Hipótesis de Usuario

Creemos que, al ofrecer un canal simple de reporte de incidentes a los residentes, estos reportarán problemas más temprano en lugar de esperar a que el daño sea evidente. Sabremos que esto es cierto si el 60 % de los incidentes reportados en la plataforma corresponde a etapas tempranas de falla (según clasificación de severidad) y no a fallas ya consumadas. Consideramos que, al recibir alertas con severidad y contexto técnico previo, las empresas de mantenimiento reducirán su tiempo de diagnóstico en campo. Validaremos esta hipótesis si el 75 % de los técnicos reporta, en encuestas posteriores a la visita, que la información previa aceleró su diagnóstico. Creemos que, al mostrarle al residente el estado y las acciones tomadas frente a su reporte, aumentará su percepción de que la cuota de mantenimiento se usa de forma efectiva. Confirmaremos esto si el 65 % de los residentes encuestados percibe una mejora en la transparencia del uso de sus cuotas tras tres meses de uso de la plataforma. Finalmente, pensamos que al automatizar la priorización de visitas para el administrador, se reducirá el tiempo que dedica semanalmente a coordinar el mantenimiento. Esta hipótesis será válida si el tiempo reportado de gestión semanal disminuye en al menos 30 % según encuestas comparativas antes/después de la adopción.

1.2.2.4. Lean UX Canvas

1.3. Segmentos objetivo

Segmento objetivo #1: Administradores de edificios y condominios

Está conformado por administradores de propiedad horizontal en Lima Metropolitana, designados por la junta de propietarios para gestionar el edificio, según lo establece el Decreto Legislativo 1568 (2023). Suelen administrar uno o varios edificios a la vez, combinando tareas de cobranza, coordinación de proveedores y atención a reclamos de residentes, con poco tiempo disponible para supervisar en detalle el estado técnico de los equipos comunes. Su principal presión proviene de dos frentes: la obligación legal de recaudar y ejecutar las cuotas de mantenimiento de los bienes comunes (Decreto Legislativo 1568, 2023), y la exigencia de la junta de propietarios de justificar en qué se gasta ese dinero. Este segmento opera hoy con información limitada —normalmente solo la boleta eléctrica del mes o la llamada de un residente— y toma decisiones de mantenimiento de forma reactiva, lo que los expone a que un mismo equipo (bomba, tablero, ascensor) termine en falla total antes de que se autorice una revisión preventiva.

Segmento objetivo #2: Propietarios y residentes

Este segmento incluye a los propietarios y residentes que habitan las unidades del edificio y que, según el Decreto Legislativo 1568 (2023), están obligados a aportar las cuotas destinadas al mantenimiento de las instalaciones y bienes comunes. Se ven directamente beneficiados por el auge de vi-

vienda multifamiliar que atraviesa Lima, con un crecimiento de ventas de 23.3 % en 2024 (CAPECO, 2025), lo que hace que este segmento crezca junto con la oferta de nuevos edificios. En la práctica, su experiencia con el mantenimiento es casi siempre negativa: notan el problema cuando el agua no sube, el ascensor se detiene o el aire acondicionado deja de enfriar, y no cuentan con un canal claro para reportarlo ni con visibilidad de si la cuota que pagan mensualmente realmente previene esas situaciones. Esta desconexión entre lo que pagan y lo que perciben genera desconfianza hacia la administración y reclamos que, muchas veces, llegan después de que el daño ya ocurrió.

Segmento objetivo #3: Empresas de mantenimiento

Está compuesto por empresas y técnicos independientes contratados por los administradores para dar servicio a los equipos críticos del edificio (bombas hidroneumáticas, tableros eléctricos, ascensores, aires acondicionados). Este segmento opera hoy mayormente bajo demanda de emergencia: como describen las propias empresas de mantenimiento de bombas en Lima, la secuencia típica sin prevención es que el tanque hidroneumático falla, la bomba entra en ciclo corto, el consumo eléctrico se dispara y el motor termina quemado, cuando un chequeo mensual de vibración y amperaje habría evitado la falla (Samiria Soluciones, 2025). Para este segmento, cada visita de emergencia representa una intervención más costosa y menos planificable que una visita preventiva programada, y llegan al edificio sin datos previos sobre el estado real del equipo, lo que alarga el tiempo de diagnóstico en campo. Su interés coincide con el de CodeNova: mientras más temprano se detecte el desgaste, más ordenable es su semana de trabajo y menor el riesgo de intervenciones de alto costo.

Capítulo II: Requirements Elicitation & Analysis

2.1. Competidores

Fracttal One (Competidor Directo)

Fracttal One es un software CMMS/EAM de origen chileno con presencia en múltiples países de la región, dirigido a empresas de sectores como manufactura, minería, transporte, hotelería, salud y gestión de instalaciones , entre otras industrias. A diferencia de un CMMS tradicional, soporta explícitamente mantenimiento correctivo, preventivo, predictivo y basado en condición, y su módulo de monitoreo acepta tanto lecturas manuales como lecturas automatizadas provenientes de dispositivos IoT que alimentan alertas y planes de mantenimiento a partir de las variables de los activos, evitando paradas no planificadas. Su línea de hardware, Fracttal Sense, ofrece sensores para monitorizar variables como vibración y temperatura en máquinas rotativas, así como temperatura y humedad ambiental , convirtiendo cada lectura en alarmas y planes de acción automáticos. Es el competidor más cercano a la propuesta de valor de CodeNova, pero está orientado a operaciones industriales y de facilities generales, no a la gestión residencial de condominios ni al modelo de cuota fija por edificio que maneja CodeNova.

DimoMaint (Competidor Directo)

DimoMaint es un software CMMS de origen francés con operación en Latinoamérica, orientado a la planificación y el control del mantenimiento en sitios y edificios. Organiza los activos bajo una jerarquía lógica de sitios, edificios, pisos y locales, y mantiene una ficha técnica de cada equipo con historial de intervenciones , además de permitir la creación de listas de verificación para el mantenimiento reactivo o planificado y el seguimiento de indicadores clave y plazos contractuales. Su enfoque en la planificación preventiva y en la trazabilidad de intervenciones es similar al que busca CodeNova, pero su propuesta está pensada para portafolios de instalaciones corporativas administradas por equipos técnicos propios, y no incorpora de forma nativa una capa de sensores IoT específica para el contexto de condominios residenciales ni una experiencia pensada para residentes y juntas de propietarios.

OORB (Competidor Indirecto)

OORB es una plataforma de administración de edificios y condominios desarrollada en Perú, enfocada en boletas automáticas, cobranza, fondos, seguridad con códigos QR y portales diferenciados por rol para administradores, propietarios y personal de seguridad. Comparte con CodeNova el mismo tipo de cliente objetivo —administradores y juntas de propietarios de edificios en Lima— y ya tiene tracción en el mercado peruano de gestión de condominios. Sin embargo, su propuesta de valor se concentra en la gestión administrativa, financiera y de seguridad del edificio, sin ningún componente de monitoreo del estado físico de los equipos críticos ni de mantenimiento preventivo basado en

sensores. Es un competidor indirecto relevante porque podría convertirse en un canal de distribución o en una amenaza si decide incorporar un módulo de mantenimiento en el futuro. En conjunto, este análisis muestra que CodeNova no compite en un espacio vacío: existen soluciones CMMS con capacidades IoT robustas a nivel regional (Fracttal One, DimoMaint) y soluciones locales de administración de condominios con fuerte adopción en Lima (OORB). La oportunidad de CodeNova está en el cruce de ambos mundos, que hoy nadie cubre de forma específica: sensores IoT de bajo costo instalados en los equipos críticos de un condominio, con una experiencia diseñada a la vez para el administrador, el residente y la empresa de mantenimiento, bajo un modelo de cuota fija mensual por edificio.

2.1.1. Análisis competitivo

¿Por qué llevar a cabo este análisis?

Identificar las fortalezas, debilidades y estrategias de las plataformas existentes de gestión de mantenimiento y administración de edificios, para definir la propuesta de valor única de CodeNova como la plataforma de mantenimiento preventivo basada en IoT para condominios en Lima, y detectar las brechas de mercado que ningún competidor cubre hoy.

Atributos	CodeNova	Fracttal One	DimoManit	OORB
Overview	Plataforma web peruana de monitoreo y gestión de mantenimiento preventivo para edificios y condominios mediante sensores IoT (vibración, temperatura, humedad, consumo eléctrico), que conecta a administradores, residentes y empresas de mantenimiento.	CMMS/EAM de origen chileno con presencia regional, dirigido a empresas industriales, de facilities y de múltiples sectores, con soporte nativo para lecturas manuales y automatizadas de sensores IoT (línea Fracttal Sense).	CMMS de origen francés con operación en Latinoamérica, orientado a la planificación y trazabilidad del mantenimiento de sitios y edificios corporativos, con arborescencia de activos y ficha técnica por equipo.	Plataforma peruana de administración de edificios y condominios enfocada en facturación, cobranza, fondos, seguridad con QR y portales por rol , sin monitoreo técnico de equipos.

Atributos	CodeNova	Fracttal One	DimoManit	OORB
Ventaja competitiva ¿Qué valor ofrece a los clientes?	Único enfoque combinado de sensores IoT de bajo costo + experiencia diseñada a la vez para administrador, residente y empresa de mantenimiento, bajo cuota fija por edificio.	Plataforma robusta y probada, con IA integrada, más de 300 integraciones y su propio hardware de sensores (Fracttal Sense).	Trazabilidad histórica sólida de intervenciones y jerarquía de activos por sitio/edificio, pensada para portafolios corporativos.	Fuerte adopción local, configuración de edificio sin costo y ahorro de tiempo administrativo comprobado (más de 50 horas al mes según sus propios testimonios).
Mercado Competitivo	Administradores, residentes y empresas de mantenimiento de edificios multifamiliares en Lima Metropolitana.	Empresas de manufactura, minería, transporte, hotelería, salud y gestión de instalaciones a nivel regional.	Empresas y organizaciones con portafolios de sitios/edificios corporativos en Latinoamérica.	Administradores, empresas de administración, juntas de propietarios y conserjerías de edificios en Perú.
Estrategias de marketing	Alianzas con juntas de propietarios y empresas de mantenimiento como canal de entrada	Marketing B2B consultivo: un consultor evalúa las necesidades de cada empresa antes de ofrecer un plan y presupuesto.	Marketing B2B dirigido a gerencias de facilities/mantenimiento corporativo.	Marketing digital directo a administradores, con testimonios y casos de ahorro de tiempo como gancho principal.

Atributos	CodeNova	Fracttal One	DimoManit	OORB
Productos & Servicios	Sensores IoT, motor de alertas por severidad, dashboard de ahorro acumulado, módulo de incidentes para residentes, agenda y control de materiales.	CMMS completo (OT, activos, inventario), módulo de IA asistente, gateway y sensores propios (Fracttal Sense) para vibración, temperatura y humedad .	Árbol de sitios/edificios/pisos/locales, ficha técnica de equipos, checklists de inspección, seguimiento de KPIs y SLAs .	Facturación automática por departamento, control de morosidad, fondos, rondas de seguridad con QR, portales diferenciados por rol.
Precios & Costos	Cuota fija mensual por edificio.	Planes escalables según número de usuarios/activos, con prueba gratuita y cotización personalizada; sin tarifa pública fija.	Sin precios públicos; cotización personalizada según alcance del proyecto.	Configuración del edificio sin costo inicial; cotización personalizada según cantidad de unidades.
Canales de distribución (Web y/o Móvil)	Web (y app móvil en etapas futuras).	Web, con app móvil para técnicos en campo.	Web.	Web y móvil (Google Play / App Store).

Atributos	CodeNova	Fracttal One	DimoManit	OORB
Fortalezas	Enfoque específico en condominios residencia-les (nicho que ni Fracttal ni DimoMaint atienden); modelo que conecta a los tres actores del ecosistema en un solo flujo; alineado con una obligación legal vigente (DL 1568) que sostiene la demanda.	Producto maduro (4.6/5 en Capterra), IA integrada, ecosistema de sensores propio, fuerte capacidad de integración.	Trazabilidad histórica robusta y jerarquía de activos clara para portafolios grandes de edificios.	Adopción ya consolidada entre administradores en Lima; configuración gratuita y rápida; ahorro de tiempo administrativo demostrado.

Atributos	CodeNova	Fracttal One	DimoManit	OORB
Debilidades	Marca nueva sin reconocimiento ni base instalada; costo de despliegue de hardware IoT por edificio; sin historial que respalde la confiabilidad de las alertas.	Pensado para operaciones industriales, no para la experiencia de un residente o una junta de propietarios; proceso de cotización consultivo, poco ágil para un edificio individual.	Sin sensores IoT nativos ni experiencia pensada para residentes; orientado a clientes corporativos, no a juntas de propietarios.	Ningún componente de monitoreo técnico ni mantenimiento preventivo basado en datos; su “gestión de mantenimiento” se limita a un registro de incidentes, no a la prevención.

Atributos	CodeNova	Fracttal One	DimoManit	OORB
Oportunidades	Crecimiento sostenido de edificios multifamiliares en Lima (CAPECO, 2025); vacío de mercado entre CMMS industriales y apps de administración sin mantenimiento; posible integración con plataformas de administración existentes (como OORB) en vez de competir directamente.	Podría crear una línea “residencial” aprovechando su hardware Fracttal Sense.	Expandirse a la gestión de condominios residenciales en Latinoamérica.	Podría integrarse con proveedores de sensores IoT para ofrecer mantenimiento preventivo dentro de su misma plataforma.

Atributos	CodeNova	Fracttal One	DimoManit	OORB
Amenazas	Que OORB u otro jugador local incorpore un módulo de mantenimiento; resistencia de juntas de propietarios a aprobar una cuota adicional; que Fracttal o DimoMaint bajen su ticket de entrada para atender edificios pequeños.	Un competidor local y más económico enfocado 100 % en condominios, como CodeNova.	Igual que Fracttal, un jugador local especializado en el segmento residencial.	Que un competidor como CodeNova sea adoptado por administradores como complemento y, con el tiempo, absorba también las funciones administrativas.

2.1.2. Estrategias y tácticas frente a competidores

Estrategias - **Diferenciación por especialización residencial:** A diferencia de Fracttal One y DimoMaint, que atienden operaciones industriales o corporativas de gran escala, CodeNova se posicionará exclusivamente para el contexto de condominios residenciales de Lima, con un lenguaje y una experiencia pensada para administradores no técnicos, residentes y empresas de mantenimiento locales. - **Coexistencia en vez de sustitución con plataformas administrativas:** En lugar de competir directamente con OORB u otras plataformas de administración de edificios, CodeNova buscará posicionarse como el módulo de mantenimiento preventivo que estas plataformas no ofrecen, abriendo la puerta a integraciones o alianzas antes que a un enfrentamiento frontal. - **Demostración de retorno económico medible:** Frente al modelo de cotización consultiva de Fracttal y DimoMaint, CodeNova competirá mostrando de forma simple y cuantificada el ahorro acumulado frente al mantenimiento correctivo,

apoyado en cifras públicas ya validadas (MEF, 2022) para dar credibilidad al argumento. - **Adopción progresiva por edificio piloto:** En lugar de un lanzamiento masivo, se validará el modelo con un número reducido de edificios en distritos de alta concentración de multifamiliares, antes de escalar a otros administradores y empresas de mantenimiento. - **Experiencia simple para tres roles distintos:** El diseño priorizará que cada actor (administrador, residente, técnico) vea solo lo que necesita, evitando la complejidad técnica que caracteriza a los CMMS industriales como Fracttal One.

Tácticas - **Alianzas con empresas de mantenimiento locales:** Ofrecer a las empresas de mantenimiento acceso gratuito a las alertas de sus edificios clientes, como incentivo para que recomienden CodeNova a los administradores. - **Programa de edificios fundadores:** Beneficios como instalación de sensores a costo reducido o meses gratuitos para los primeros edificios que se sumen, generando casos de éxito medibles (ahorro real, incidentes evitados). - **Comparativa directa en la propuesta comercial:** Mostrar a los administradores, en la etapa de venta, una comparación simple entre el costo de una falla correctiva evitada y el costo de la cuota mensual de CodeNova. - **Contenido educativo sobre el DL 1568:** Campañas dirigidas a administradores y juntas de propietarios explicando sus obligaciones bajo el nuevo marco normativo y cómo CodeNova facilita cumplirlas de forma más económica. - **Exploración de integración con OORB u otras plataformas administrativas:** Evaluar una API o integración que permita a los administradores que ya usan una plataforma de administración ver también las alertas de mantenimiento de CodeNova sin cambiar de sistema.

2.2. Entrevistas

2.2.1. Diseño de entrevistas

Propietarios - 1. ¿Cómo describirías la última vez que tuviste un problema de mantenimiento en tu departamento o edificio (agua, ascensor, electricidad)? ¿Qué pasó desde que lo notaste hasta que se resolvió? - 2. ¿Sabes en qué se usa exactamente la cuota de mantenimiento que pagas cada mes? ¿Alguna vez has cuestionado o pedido detalle de ese gasto? - 3. ¿Alguna vez has sufrido un corte de agua, falla del ascensor o del aire acondicionado en áreas comunes? ¿Con qué frecuencia ocurre algo así? - 4. ¿Confías en que la administración de tu edificio detecta los problemas a tiempo, o sientes que reaccionan solo cuando ya algo falló? - 5. Si tu cuota subiera porque hubo una reparación de emergencia, ¿qué tanto te molestaría comparado con que subiera por mantenimiento preventivo programado? - 6. ¿Te gustaría poder ver en algún momento el estado real de los equipos del edificio (bombas, tableros, ascensores) o prefieres no involucrarte en eso? - 7. ¿Cómo reportas hoy un incidente (fuga, ruido, falla) al administrador? ¿Qué tan rápido sueles recibir respuesta o seguimiento? - 8. ¿Qué tan importante es para ti que tu edificio tenga un sistema “inteligente” que prevenga fallas, versus otros beneficios (seguridad, áreas comunes, etc.)?

Administradores - 1. ¿Cómo llevas actualmente el control del estado de los equipos del edificio (bombas, tableros eléctricos, ascensores, A/C)? ¿Usas alguna herramienta, hoja de cálculo o solo memoria/experiencia? - 2. Cuéntame sobre la última falla grave que tuviste que gestionar: ¿cómo te enteraste, cuánto costó, y cuánto tiempo tomó resolverla? - 3. ¿Con qué frecuencia programas mantenimiento preventivo versus cuánto terminas gastando en reparaciones de emergencia no planificadas? - 4. ¿Cómo decides a qué empresa de mantenimiento llamar y cómo coordinas las visitas? ¿Qué parte de ese proceso te resulta más tediosa? - 5. Cuando un residente reporta un incidente, ¿cómo registras y das seguimiento a esa solicitud hasta que se cierra? - 6. ¿Qué información te gustaría tener a la mano para justificarle a la junta de propietarios en qué se gastan las cuotas de mantenimiento? - 7. ¿Has tenido que enfrentar reclamos de residentes por fallas que "se pudieron haber evitado"? ¿Cómo manejas esas situaciones? - 8. Si pudieras anticipar una falla antes de que ocurra (por ejemplo, una alerta de que una bomba está por fallar), ¿qué harías distinto en tu día a día?

Empresa de mantenimiento - 1. ¿Cómo reciben hoy los pedidos de servicio de un edificio: llamada, WhatsApp, correo? ¿Qué información suelen tener antes de llegar al sitio? - 2. Cuéntame de un caso reciente donde llegaron a una "emergencia" que, mirando atrás, se pudo haber prevenido si hubieran sabido antes que algo andaba mal. - 3. ¿Cómo priorizan qué edificio o equipo atender primero cuando tienen varias solicitudes en la misma semana? - 4. ¿Qué datos técnicos (vibración, temperatura, consumo eléctrico, horas de uso) sueles necesitar en campo y cuáles no tienes disponibles al momento de la visita? - 5. ¿Qué diferencia, en tiempo y costo, hay entre una visita de mantenimiento programado y una reparación de emergencia para el mismo tipo de equipo? - 6. ¿Cómo llevan el historial de mantenimiento de cada edificio o equipo? ¿Se pierde información cuando cambia el técnico o el administrador? - 7. ¿Qué tan seguido un cliente (administrador) les pide evidencia o reportes del trabajo realizado? ¿Cómo se los entregan hoy? - 8. Si pudieran recibir alertas priorizadas por severidad antes de que el equipo falle completamente, ¿cambiaría su forma de programar la semana de trabajo? ¿Cómo?

2.2.2. Registro de entrevistas

Hicimos entrevistas semiestructuradas a los tres segmentos: dos administradoras, dos residentes y tres personas de empresas de mantenimiento. Todas se grabaron y están juntas en un solo video. Para cada entrevista anotamos los datos de la persona, el minuto en que empieza en el video y un resumen de lo que contó.

- **Nombre del archivo:** upc-pre-202620-1asi0729-16692-CodeNova-needfinding-sprint-1.mov
- **URL del video:** https://acortar.link/rZDIrw

Segmento #1: Administradores de edificios y condominios

Entrevista 1

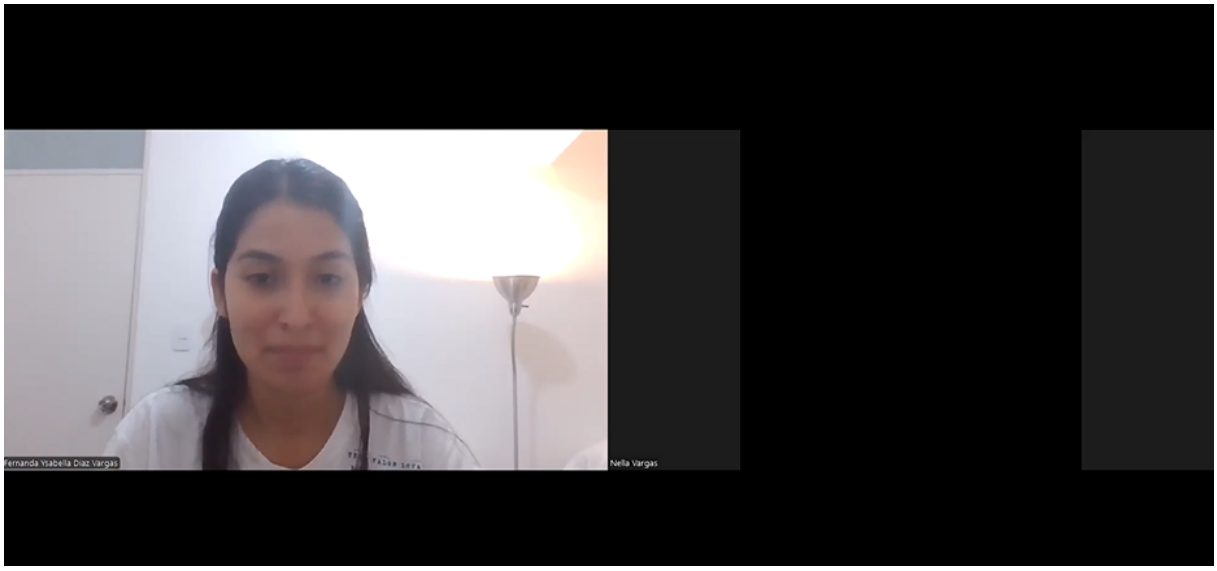


Figura 1: Figura 2.2.2-1 – Captura de la entrevista 1

Dato	Detalle
Nombre y apellidos	Nella Vargas
Edad	64 años
Distrito	San Miguel
Ocupación	Administradora de edificio
Inicio en el video	00:00
Duración	07:24

Resumen. La entrevistada administra un edificio y lleva el control del estado de los equipos (bombas, tableros eléctricos, ascensores, A/C) de forma completamente manual: anota en un cuaderno físico las fechas de próximo mantenimiento y hace seguimiento presencial de que los equipos “trabajen bien”, sin usar ninguna herramienta digital ni hoja de cálculo. Aplica mantenimiento preventivo cada 6 meses a los equipos, y es consciente de que descuidar esa frecuencia le sale más caro: cuando un equipo como el ascensor falla sin haber tenido mantenimiento, debe llamar a una empresa de emergencia a un costo mayor. Como evidencia de una falla grave reciente, relató un corte eléctrico que

afectó solo a su edificio (mientras el resto de la cuadra recuperó luz), lo que derivó en un diagnóstico de la compañía eléctrica y de un electricista, encontrándose que el tablero eléctrico fallaba y requería un componente de mayor potencia; el cambio del equipo tomó aproximadamente 3 a 4 días adicionales una vez identificado el problema. Sobre la coordinación con proveedores, identifica como el punto más tedioso del proceso conseguir que la empresa de mantenimiento le dé tiempo de atención, ya que estas empresas manejan agendas fijas salvo que se trate de una emergencia, caso en el que sí responden de inmediato. Para el registro y priorización de incidentes reportados por residentes, aplica un criterio de urgencia claro: emergencias como fugas de agua, fallas del ascensor o de la luz se atienden de inmediato, mientras que incidentes menores (como una cerradura dañada dentro de un departamento) quedan como segunda prioridad, sin mencionar el uso de un sistema formal de registro más allá de la gestión manual/verbal. En cuanto a la rendición de cuentas ante la junta de propietarios, conserva todos los recibos y boletas de gastos (productos de limpieza, materiales, pagos a proveedores) como evidencia física de en qué se invierte la cuota de mantenimiento, mostrándolos directamente a la junta para demostrar transparencia. Reconoce enfrentar reclamos de residentes que esperan soluciones inmediatas a fallas que “se pudieron evitar”; su manera de manejarlo es priorizar dar una pronta solución, lo cual —según indica— es aprobado por la mayoría de residentes aunque siempre exista algún reclamo persistente. Frente a la posibilidad de anticipar fallas, expresó explícitamente el deseo de contar con una herramienta que le avise cuándo un equipo está por fallar (por ejemplo, una alerta sobre un tablero o fusible próximo a quemarse), de manera que pueda coordinar el mantenimiento antes de que el equipo se malogre, en lugar de reaccionar después del hecho.

Entrevista 2

Figura 2: Figura 2.2.2-2 – Captura de la entrevista 2

Dato	Detalle
Nombre y apellidos	Mireya Perales
Edad	35 años
Distrito	San Borja
Ocupación	Administradora de edificio
Inicio en el video	07:24
Duración	03:49

Resumen. La entrevistada lleva el control del estado de los equipos en un Excel que actualiza tarde —a veces una semana después de cada visita técnica—, por lo que en el día a día confía más en la memoria y en lo que le indica el técnico de turno que en el archivo. Hace dos meses enfrentó una falla grave: se quemó el motor de la bomba de agua de su edificio, de la que se enteró por la llamada furiosa de un residente del piso 8 al no subir el agua. La reparación de emergencia costó S/ 3,900 y tomó

4 días resolverse, porque el técnico habitual no tenía el repuesto en stock. Programa mantenimiento preventivo dos veces al año, pero reconoce que cerca del 70 % de su presupuesto de mantenimiento termina siendo gasto correctivo no planificado. Coordina con 2 o 3 empresas de confianza, llamando a la que responda más rápido, y señala que lo más tedioso es coordinar el acceso con el conserje y hacer seguimiento manual de si la visita realmente se realizó. Los residentes le escriben directamente por WhatsApp a su número personal, y ella anota los reportes en una libreta física de la oficina, tachándolos cuando se resuelven —reconoce que si pierde la libreta, pierde el historial completo—. Le gustaría poder mostrarle a la junta de propietarios un número concreto de ahorro (por ejemplo, “este año evitamos 2 emergencias que hubieran costado S/ 8,000”), ya que hoy solo puede mostrar facturas de lo ya gastado, nunca lo que se ahorró por prevenir. Ha enfrentado reclamos de residentes por fallas que “se pudieron evitar”, sin tener forma de demostrar que no hubo negligencia de su parte, lo que describe como una situación injusta. Si pudiera anticipar una falla antes de que ocurra, llamaría al técnico esa misma semana en vez de esperar una emergencia de fin de semana (más cara y difícil de atender), y avisaría a los residentes con anticipación en vez de que se enteren cuando ya no hay agua.

Segmento #2: Propietarios y residentes

Entrevista 3

Figura 3: Figura 2.2.2-3 – Captura de la entrevista 3

Dato	Detalle
Nombre y apellidos	Luis Becerra
Edad	24 años
Distrito	San Miguel
Ocupación	Propietario
Inicio en el video	38:11
Duración	04:11

Resumen. El entrevistado relata que hace un mes se malogró la bomba de agua de su edificio un sábado por la noche; se dio cuenta porque no subía presión al tercer piso, donde vive. Reportó por el grupo de WhatsApp del edificio (unas 40 personas, usado para todo tipo de temas) a las 9pm, pero el administrador recién respondió el lunes, dejando al edificio casi 36 horas sin presión adecuada de agua. Al llegar el técnico, se supo que el motor llevaba tiempo haciendo un ruido raro que nadie había reportado. Paga S/ 180 mensuales de cuota, de los cuales conoce que cubren personal de limpieza y seguridad, pero desconoce el detalle del gasto en mantenimiento de equipos —una vez pidió esa información y le remitieron un balance anual en PDF que nunca llegó a revisar por no ser comprensible. El ascensor se ha detenido unas 3 veces este año, y reconoce que las bajadas menores de presión ya ni se reportan porque se asume que se normalizarán solas. No confía en que la administración detecte los problemas a tiempo; considera que no es un problema de mala gestión, sino de falta de herramientas para saber que algo anda mal antes de que falle. Le molestaría mucho más un alza de cuota por una emergencia (ej. S/50 de golpe) que por mantenimiento preventivo explicado con anticipación (ej. S/10 programado). No necesita ver datos técnicos, pero sí le gustaría un indicador simple tipo semáforo (verde/amarillo/rojo) del estado de los equipos. Hoy la respuesta a un reporte demora entre unas horas y 2 días. Aunque un sistema “inteligente” de mantenimiento no es su primer criterio al elegir dónde vivir —pesa más la seguridad—, después de vivir la falla de la bomba sí pagaría más para que no se repita.

Entrevista 4

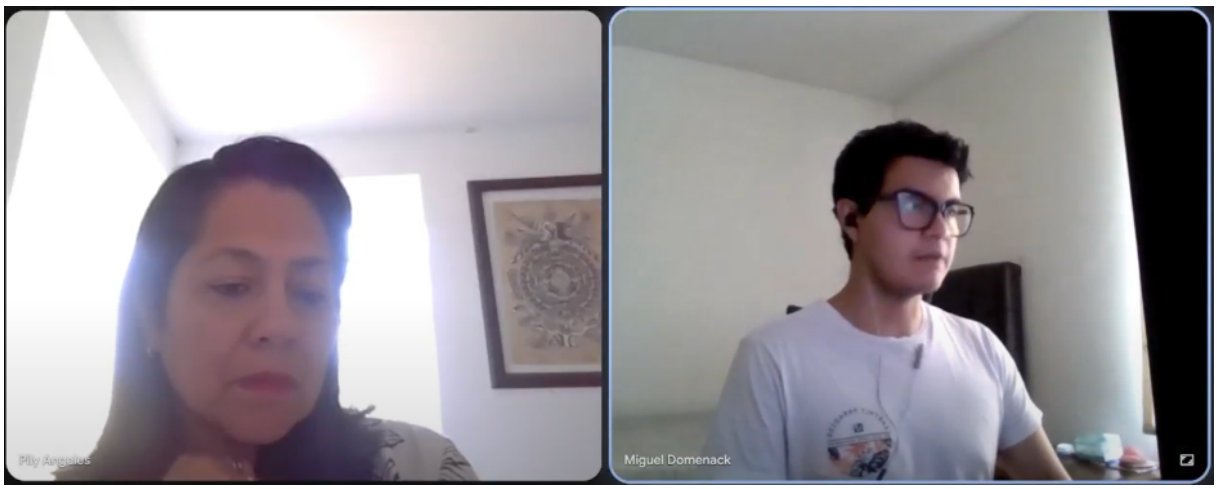


Figura 4: Figura 2.2.2-4 – Captura de la entrevista 4

Dato	Detalle
Nombre y apellidos	Pilar Angeles
Edad	54 años
Distrito	Magdalena
Ocupación	Residente
Inicio en el video	30:11
Duración	08:00

Resumen. La entrevistada es residente de un edificio donde el reporte de incidentes y las coordinaciones con la administración se realizan a través de un chat de propietarios e inquilinos, logrando una comunicación y respuesta rápidas. Hace dos meses, mientras se encontraba fuera del país, ocurrió la ruptura de una tubería en el departamento frente al suyo con filtraciones hacia el pasillo; la emergencia se resolvió en pocas horas gracias a la alerta inmediata por el chat y a que un familiar acudió a abrir el departamento, lo que evitó que el agua ingresara a los ascensores. En el ámbito económico, recibe mensualmente el desglose de los gastos de mantenimiento por correo electrónico y en impresos publicados en el edificio, sobre los cuales nunca ha tenido inconvenientes ni reclamos. Las reparaciones urgentes se financian mediante un fondo de contingencia constituido por el redondeo de la cuota mensual. A pesar de que la administración programa mantenimiento preventivo para bombas y ascensores, los ascensores sufren fallas aproximadamente una vez cada dos meses, las cuales son atendidas por técnicos especializados. Actualmente, la información sobre el estado de los equipos

es reactiva y se limita a fotos o videos enviados por el personal de vigilancia una vez ocurrido el problema; por ello, la entrevistada valora positivamente la implementación de sistemas inteligentes y tecnología de monitoreo en tiempo real que permitan conocer el estado real de los equipos y prevenir fallas.

Segmento #3: Empresas de mantenimiento

Entrevista 5

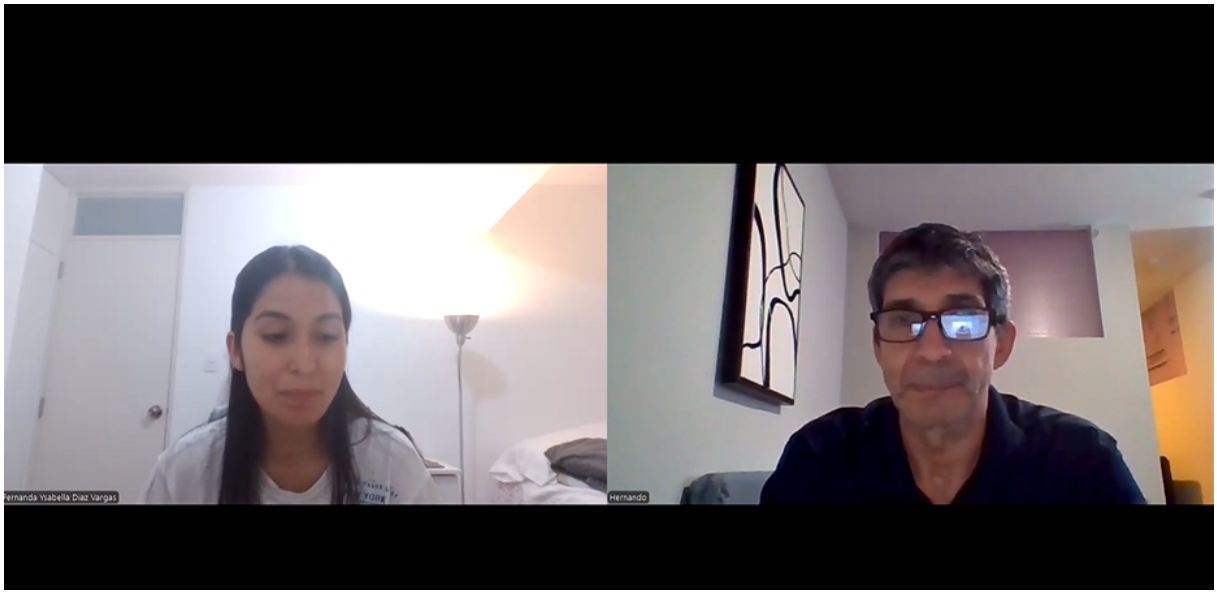


Figura 5: Figura 2.2.2-5 – Captura de la entrevista 5

Dato	Detalle
Nombre y apellidos	Hernando Diaz
Edad	61 años
Distrito	
Ocupación	Trabajador de Empresa de Mantenimiento
Inicio en el video	11:12
Duración	07:58

Resumen. El entrevistado trabaja como técnico en una empresa de mantenimiento y recibe hoy los pedidos de servicio principalmente por correo electrónico, donde el cliente indica el tipo de emergen-

cia o pedido junto con algunas pautas iniciales para el trabajo. Como ejemplo de una falla que se pudo haber prevenido, relató una visita de emergencia por un aire acondicionado que dejó de funcionar: al llegar identificó que el drenaje estaba tapado, un problema que se habría evitado con un mantenimiento programado y un seguimiento regular por apartamento. La priorización de solicitudes cuando hay varias en la misma semana no responde a un criterio de severidad, sino de orden de llegada: se atiende primero a quien llamó más temprano, sin distinguir qué tan crítico es el problema frente a otros. En campo, suele contar con herramientas básicas (termómetro, herramientas de electricidad), pero no dispone de medidores eléctricos ni de instrumentos para medir la presión de gas, por lo que en esos casos debe solicitarlos aparte. Sobre costos, señaló que no existe diferencia entre una visita programada y una reparación de emergencia por el mismo tipo de equipo: el costo es el mismo salvo que se necesite reemplazar una pieza. El historial de mantenimiento se lleva de forma manual: cada técnico reporta a diario sus órdenes de trabajo a su jefe directo, quien las almacena en su computadora; para evitar pérdida de información al cambiar de técnico, todos dejan notas del trabajo realizado en cada orden. Indicó que los residentes rara vez piden reportes o evidencia del trabajo: su única exigencia es que la emergencia quede resuelta. Finalmente, frente a la posibilidad de recibir alertas priorizadas por severidad antes de que un equipo falle por completo, se mostró favorable: considera que le permitiría prepararse mejor, ofrecer un mejor servicio y mantenimiento preventivo en lugar de trabajar solo bajo presión de emergencia.

Entrevista 6

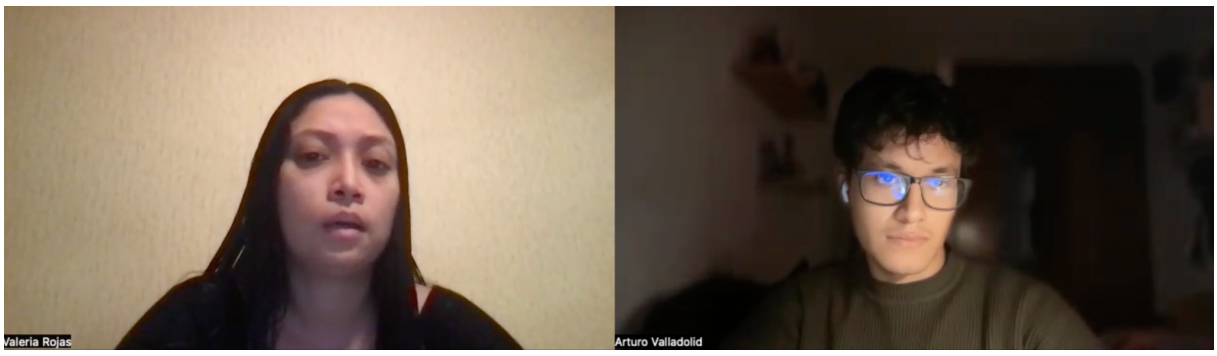


Figura 6: Figura 2.2.2-6 – Captura de la entrevista 6

Dato	Detalle
Nombre y apellidos	Valeria Rojas
Edad	30 años
Distrito	

Dato	Detalle
Ocupación	Coordinadora de Operaciones y Servicio Técnico
Inicio en el video	19:10
Duración	06:52

Resumen. La entrevistada, Valeria, trabaja como Coordinadora de Operaciones y Servicio Técnico en una empresa de mantenimiento de bombas de presión para edificios. Actualmente, los pedidos de servicio llegan principalmente por whatsapp, con información básica como dirección, nombre del condominio y fotos del tablero y la bomba; los detalles técnicos (amperaje, presión, vibración, temperatura) solo se obtienen cuando el técnico está en sitio, lo que afecta el diagnóstico y los materiales llevados. Como caso prevenible, relató una emergencia en Chorrillos donde la bomba dejó de dar agua en pisos altos: al llegar encontraron amperaje elevado y motor sobrecalentado, requiriendo reemplazo completo; lecturas periódicas habrían permitido detectar la falla a tiempo. La priorización se basa en gravedad percibida (sin agua, tableros quemados, bombas contra incendios), pero a veces cede ante clientes insistentes, generando semanas con múltiples urgencias. Alertas clasificadas por severidad aportarían un criterio más objetivo para organizar la agenda. En campo, necesitan datos como presión de entrada/salida, amperaje, temperatura del motor, condición del tanque y ruidos/vibraciones, pero carecen de historial evolutivo de estos parámetros. Una visita programada permite coordinar acceso, revisar varios puntos y llevar materiales; una emergencia implica mover técnicos de otras zonas, cortes sin aviso y gastos adicionales por repuestos y mano de obra. El historial se gestiona con informes PDF, fotos por WhatsApp y Excel; la información se fragmenta cuando intervienen varias personas o cambian administradores, obligando a reexplicar procesos. El historial mínimo debería incluir fecha, técnico, equipo, motivo, diagnóstico, acciones, materiales y evidencias fotográficas. Los clientes solicitan reportes con frecuencia para justificar gastos en juntas de propietarios, entregándose por whatsapp y correo. Alertas priorizadas por severidad permitirían separar casos críticos, preparar técnicos con anticipación, conocer síntomas y evolución de fallas, y organizar la agenda por severidad, tipo de equipo y ubicación, mejorando la eficiencia del servicio.

Entrevista 7

Figura 7: Figura 2.2.2-7 – Captura de la entrevista 7

Dato	Detalle
Nombre y apellidos	Diego Rances
Edad	30 años
Distrito	
Ocupación	Técnico / responsable de campo, empresa de mantenimiento de bombas y sistemas eléctricos
Inicio en el video	26:02
Duración	04:09

Resumen. El entrevistado indica que cerca del 80 % de los pedidos que reciben por WhatsApp o llamada son emergencias, sin ningún dato previo sobre el estado del equipo. Describe un caso reciente en el que un motor se quemó en un edificio de Chorrillos: al llegar, el amperaje estaba disparado y ya no tenía arreglo, algo que se hubiera notado con anticipación si hubiesen medido vibración y amperaje semanas antes. Hoy priorizan sus visitas según quién llama primero o insiste más, no según la gravedad real del problema; un cliente pequeño con una emergencia real puede esperar porque un cliente grande llamó antes por algo menor. Las variables técnicas que más le ayudarían en campo son vibración, amperaje del motor y horas de uso acumuladas, ninguna de las cuales tiene disponible hoy, por lo que hace el diagnóstico completo desde cero en cada visita, incluso en equipos que ya conoce. Señala una diferencia marcada entre una visita programada (1 a 2 horas, hasta S/ 300) y una reparación de emergencia con cambio de motor (todo un día, entre S/ 2,500 y S/ 4,000, más el sobrecosto de conseguir el repuesto con urgencia). El historial de mantenimiento depende de las notas personales de cada técnico; si ese técnico deja la empresa o cambia de zona, la información se pierde casi por completo, quedando solo lo que el cliente recuerda. La evidencia del trabajo se limita a 2 o 3 fotos por

WhatsApp; solo elaboran un reporte formal en PDF cuando el cliente es una empresa grande que lo exige por contrato, nunca para condominios normales. Frente a la posibilidad de recibir alertas priorizadas por severidad, afirma que cambiaría totalmente su forma de trabajar: podría agrupar visitas por zona en una misma ruta en vez de salir de emergencia cada vez que suena el teléfono, ahorrando tiempo de traslado y atendiendo primero lo realmente grave.

2.2.3. Análisis de entrevistas

Para cada segmento separamos las características objetivas (datos de la persona) de las subjetivas (lo que piensa y siente). Entre paréntesis va cuántas personas del segmento coinciden con cada punto.

Segmento #1: Administradores de edificios y condominios

Características objetivas

Edad

- Administradoras entre 35 y 64 años (2 de 2).

Distrito

- San Borja y San Miguel (2 de 2).

Edificios a cargo

- Administran 1 edificio cada una (2 de 2).

Herramientas de control

- Ninguna utiliza un sistema o software especializado de gestión de mantenimiento; el registro es manual (cuaderno físico) o semi-digital (Excel actualizado tarde) (2 de 2).

Características subjetivas

Gestión del mantenimiento

- Aplican mantenimiento preventivo de forma periódica (cada 6 meses o dos veces al año) (2 de 2).
- Reconocen que, pese a la prevención, una parte importante de su gasto y tiempo termina en reparaciones correctivas de emergencia (2 de 2).

- Dependen más de la memoria propia o del criterio del técnico de turno que de un registro sistemático confiable (2 de 2).

Relación con proveedores de mantenimiento

- Identifican como el punto más tedioso conseguir disponibilidad de la empresa de mantenimiento, ya que trabajan con agendas fijas salvo que se trate de una emergencia (2 de 2).
- Ante una emergencia, la respuesta de las empresas sí es inmediata, a diferencia del mantenimiento regular (2 de 2).

Gestión de incidentes de residentes

- Priorizan los reportes por criterio de urgencia manual (emergencias primero), sin un sistema formal de registro (2 de 2).
- El canal de comunicación con los residentes es informal: verbal/presencial en un caso, WhatsApp personal en el otro (2 de 2).

Transparencia y rendición de cuentas ante la junta

- Sustentan el gasto de mantenimiento mostrando evidencia física de lo ya gastado (boletas, facturas, recibos), sin ningún indicador de ahorro por prevención (2 de 2).
- Enfrentan reclamos de residentes por fallas que "se pudieron evitar", sin tener cómo demostrar que no hubo negligencia de su parte (2 de 2).

Interés en la solución

- Ambas expresan explícitamente, sin que se les sugiera, el deseo de contar con una alerta que avise cuándo un equipo está por fallar, para actuar antes de que se malogre (2 de 2).

Segmento #2: Propietarios y residentes

Características objetivas

Edad

- Residentes entre 24 y 54 años (2 de 2).

Distrito

- San Miguel y Magdalena (2 de 2).

Condición

- Ambos son propietarios de su unidad (2 de 2).

Características subjetivas

Percepción de la gestión de incidentes

- Se observa un contraste marcado según el canal de reporte: quien reporta por un chat genérico del edificio (mezclado con otros temas) tuvo una respuesta demorada (36 horas); quien cuenta con un chat dedicado de propietarios tuvo una respuesta y resolución en pocas horas (1 de 2 cada caso) esto sugiere que un canal claro y dedicado mejora directamente el tiempo de respuesta.
- Uno de los dos desconfía de que la administración detecte fallas a tiempo (1 de 2), mientras el otro percibe una gestión más eficiente ante emergencias puntuales (1 de 2); sin embargo, ambos coinciden en que la información sobre el estado de los equipos es hoy reactiva (2 de 2).

Transparencia y uso de la cuota

- También hay contraste en transparencia: uno desconoce el detalle del gasto en mantenimiento y recibió un balance en PDF que no pudo interpretar (1 de 2); el otro recibe un desglose mensual claro por correo e impresos y no ha tenido reclamos al respecto (1 de 2) lo que evidencia que la transparencia sí es alcanzable cuando se comunica de forma simple y constante.

Frecuencia de fallas en equipos comunes

- Ambos reportan fallas recurrentes del ascensor: 3 veces al año en un caso, aproximadamente cada 2 meses en el otro (2 de 2).
- En ambos casos, la evidencia del estado de los equipos llega después de ocurrida la falla (fotos/videos del personal de vigilancia, o simplemente ausencia de reporte hasta que el equipo falla) (2 de 2).

Interés en la solución

- Ambos valoran positivamente contar con visibilidad del estado real de los equipos, aunque con distinto nivel de detalle esperado: uno pide un indicador simple tipo semáforo, el otro pide monitoreo en tiempo real (2 de 2).
- Uno de los dos manifiesta explícitamente disposición a pagar más por mantenimiento preventivo si está bien explicado, especialmente después de haber sufrido una falla (1 de 2).

Segmento #3: Empresas de mantenimiento

Características objetivas

Edad

- Entre 30 y 61 años (3 de 3).

Ocupación

- Técnicos/responsables de campo y una coordinadora de operaciones, en empresas de mantenimiento de bombas, sistemas eléctricos y aire acondicionado (3 de 3).

Canal de recepción de pedidos

- WhatsApp o llamada (2 de 3); correo electrónico (1 de 3).

Características subjetivas

Forma de trabajo actual

- La mayoría de pedidos son de emergencia, con información previa nula o muy limitada sobre el estado del equipo (3 de 3).
- Priorizan por orden de llegada o insistencia del cliente, no por severidad real del problema (2 de 3); uno parte de un criterio de gravedad percibida, aunque reconoce que a veces cede ante clientes insistentes (1 de 3).

Datos técnicos disponibles en campo

- Carecen de datos técnicos previos (vibración, amperaje, presión, temperatura); solo los obtienen una vez en sitio (3 de 3).
- Coinciden en que contar con esas lecturas de forma anticipada habría permitido prevenir la falla que relataron como caso reciente (3 de 3).

Costo y eficiencia

- Diferencia significativa de costo y tiempo entre visita programada y reparación de emergencia (2 de 3); uno reporta que el costo es el mismo salvo que se necesite reemplazar una pieza (1 de 3).
- Pérdida o fragmentación del historial cuando cambia el técnico o interviene más de una persona (2 de 3); uno mitiga esto parcialmente con un reporte diario a su jefe directo (1 de 3).

Evidencia y reportes

- La entrega de evidencia es mayormente informal, por fotos de WhatsApp (2 de 3); los reportes formales en PDF solo se elaboran cuando el cliente lo exige por contrato o para justificar gastos ante la junta (2 de 3).
- El interés del cliente final por la evidencia varía: uno indica que rara vez se la piden, priorizando solo que la emergencia quede resuelta (1 de 3); otro indica que sí se la solicitan con frecuencia para justificar gastos ante la junta (1 de 3).

Interés en la solución

- Alta disposición a recibir alertas priorizadas por severidad para organizar mejor la semana y pasar de un esquema reactivo a uno preventivo (3 de 3).
- Coinciden en que esto permitiría preparar a los técnicos con anticipación y mejorar la eficiencia general del servicio (3 de 3).

Conclusiones del análisis

- Ninguno de los tres segmentos tiene hoy datos del estado de los equipos antes de que fallen. La información llega cuando la falla ya ocurrió.
- Los administradores necesitan mostrar a la junta algo más que facturas. Les sirve un número de ahorro y un historial por equipo.
- Los residentes no quieren datos técnicos. Les basta un canal claro para reportar y ver en qué estado está su reporte.
- Las empresas de mantenimiento priorizan por orden de llegada. Con alertas por severidad podrían ordenar su semana y llegar con contexto a cada visita.

2.3. Needfinding

Para el proceso de Needfinding se consideró realizar entrevistas a los tres tipos de usuario previamente definidos: administradores de edificios y condominios, propietarios y residentes, y empresas de mantenimiento. El propósito de esta investigación es comprender las motivaciones, dificultades y necesidades reales de cada actor frente a la gestión del mantenimiento de los equipos críticos de un edificio multifamiliar en Lima Metropolitana.

Mediante este análisis se busca validar los supuestos iniciales planteados en el Lean UX Canvas (sección 1.2.2.2), como la existencia de una brecha entre la obligación legal de recaudar cuotas de mantenimiento y las herramientas disponibles para gestionarlo de forma anticipada, así como la relevancia de la confianza y la transparencia en el uso de esas cuotas. Los resultados permitirán identificar con

mayor precisión los problemas que CodeNova debe resolver para cada segmento, y sentarán la base para la construcción de los User Personas, el User Journey Mapping y el Empathy Mapping.

2.3.1. User Personas

[COMPLETAR]

2.3.2. User Task Matrix

Segmento Objetivo #1 |Actividades|Frecuencia|Importancia| |:—|:—|:—| |Revisar el estado de los equipos críticos del edificio (bombas, tableros, ascensor) |Con frecuencia| Alta| |Coordinar y confirmar visitas con la empresa de mantenimiento | Con frecuencia| Alta| |Registrar y dar seguimiento a solicitudes de residentes|Con frecuencia|Alta| |Autorizar el pago de una reparación o intervención|A veces|Alta| |Programar mantenimiento preventivo periódico | Frecuente|Alta| |Sustentar los gastos de mantenimiento ante la junta de propietarios |A veces| Alta| |Comunicar a los residentes el estado de una falla o visita |Con frecuencia|Media| |Revisar boletas y consumo eléctrico del edificio |Con frecuencia| Media| |Evaluar o buscar nuevas empresas de mantenimiento|A veces|Media| |Cerrar y documentar una intervención realizada|Con frecuencia|Alta|

Segmento Objetivo #2 |Actividades|Frecuencia|Importancia| |:—|:—|:—| |Reportar un incidente o falla en áreas comunes| AA veces|Alta| |Hacer seguimiento al estado de un reporte enviado|A veces|Alta| |Pagar la cuota de mantenimiento mensual|Con frecuencia|Alta| |Consultar en qué se usa la cuota de mantenimiento|Rara vez|Media| |Comunicarse con vecinos sobre problemas recurrentes (WhatsApp del edificio)|Con frecuencia|Media| |Evaluar si un problema amerita reportarse o esperar a que se resuelva solo|Con frecuencia|Media| |Participar en reuniones de junta de propietarios|A veces|Media| |Recibir y leer comunicados de la administración|Con frecuencia|Baja| |Reportar un incidente o falla en áreas comunes|A veces|Alta| |Hacer seguimiento al estado de un reporte enviado|A veces|Alta|

Segmento Objetivo #3 |Actividades|Frecuencia|Importancia| |:—|:—|:—| |Recibir y atender pedidos de servicio de administradores|Con frecuencia|Alta| |Priorizar qué edificio o equipo atender primero en la semana|Con frecuencua|Alta| |Diagnosticar el estado de un equipo en campo|Con frecuencia|Alta| |Registrar el historial de la intervención realizada|A veces|Alta| |Coordinar horarios de acceso con el administrador o conserje|Con frecuencia|Media| |Cotizar y gestionar la compra de repuestos|A veces|Alta| |Entregar evidencia o reporte del trabajo realizado|A veces|Media| |Comunicarse con otros técnicos sobre casos similares|A veces|Baja| |Evaluar la adopción de nuevas herramientas o tecnología de monitoreo|A veces|Media| |Recibir y atender pedidos de servicio de administradores|Con frecuencia|Alta|

2.3.3. User Journey Mapping

2.3.4. Empathy Mapping

2.4. Big Picture Event Storming

[COMPLETAR]

2.5. Ubiquitous Language

Este glosario reúne los términos que usamos al hablar del dominio con los usuarios y entre nosotros. Los términos están en inglés porque son los mismos nombres que aparecen en los diagramas y en el código. La definición está en español.

En TB1 corregimos algunas definiciones para que coincidan con el modelo del capítulo IV. El umbral ahora es por equipo y por métrica, la severidad tiene cuatro niveles y la orden de trabajo y los materiales dejaron de ser entidades.

Término	Definición
Building	Edificio o condominio afiliado a Vigilia. Es la unidad sobre la que se cobra la cuota fija mensual.
Critical equipment	Equipo crítico del edificio: bomba de agua, tablero eléctrico, ascensor o aire acondicionado (HVAC). Sus estados son OPERATIONAL, REQUIRES_FOLLOW_UP, OUT_OF_SERVICE y DECOMMISSIONED.
Decommission	Dar de baja un equipo. El equipo no se borra: pasa a DECOMMISSIONED y conserva su historial.
Sensor	Dispositivo instalado en un equipo que mide una o más métricas. Pertenece a un solo equipo activo a la vez.
Sensor reading	Lectura individual de vibración, temperatura, humedad o consumo eléctrico enviada por un sensor.

Término	Definición
Metric	Variable que mide un sensor: VIBRATION, TEMPERATURE, HUMIDITY o POWER_CONSUMPTION.
Monitoring threshold	Rango mínimo y máximo esperado para una métrica de un equipo. Cada equipo tiene un umbral por métrica. Si una lectura sale del rango, se genera una alerta.
Disconnected sensor	Sensor que pasó más de 24 horas sin enviar lecturas.
IoT sensor network	Conjunto de sensores instalados en el edificio que envían lecturas a la plataforma. En el MVP la reemplaza un simulador, que envía lecturas por el mismo endpoint que usarían los sensores reales.
Alert	Aviso que genera la plataforma cuando una lectura sale del umbral. Sus estados son ACTIVE, ACKNOWLEDGED (en gestión) y RESOLVED. No se duplica: si ya hay una alerta activa para el mismo equipo y métrica, se actualiza.
Severity level	Nivel de gravedad de una alerta: LOW, MEDIUM, HIGH o CRITICAL. Define qué equipo se atiende primero.
Incident	Problema en un área común que reporta un residente. Puede o no estar ligado a un equipo. Sus estados son REPORTED, IN_PROGRESS y RESOLVED.
Incident evidence	Foto que acompaña un incidente. Un incidente acepta hasta tres.
Incident rating	Calificación de 1 a 5 que da el residente cuando su incidente está resuelto. Se da una sola vez.
Preventive maintenance	Mantenimiento que se hace antes de que el equipo falle, a partir de una alerta temprana o de una visita programada.

Término	Definición
Corrective maintenance	Reparación que se hace cuando el equipo ya falló.
Maintenance visit	Visita de una empresa de mantenimiento a un edificio para revisar un equipo. Puede reprogramarse con una fecha propuesta.
Intervention	Registro de lo que hizo el técnico en la visita: trabajo realizado, materiales usados, costo preventivo y estado final del equipo.
Corrective reference cost	Costo estimado de reparar una falla de un tipo de equipo. Sirve de referencia para calcular el ahorro.
Savings projection	Ahorro estimado de una visita: diferencia entre el costo de referencia correctivo y el costo preventivo real. Se crea cuando la visita termina con el equipo OPERATIONAL.
Accumulated savings	Suma de los ahorros estimados de un edificio. Es el dato que el administrador presenta a la junta de propietarios.
Technician	Persona de la empresa de mantenimiento que hace la visita en campo.
Administrator	Persona designada por la junta de propietarios para gestionar el edificio, según el DL 1568.
Resident	Propietario u ocupante de una unidad del edificio. Paga la cuota de mantenimiento y reporta incidentes.
Maintenance company	Empresa o técnico independiente contratado para dar servicio a los equipos críticos.
Notification	Aviso para un destinatario por un canal: dentro de la aplicación (IN_APP) o por correo (EMAIL).
Subscription	Cuota fija mensual por edificio que da acceso a Vigilia.

Término	Definición
Bounded context	Parte del dominio con su propio modelo y lenguaje. Vigilia tiene cinco: Asset Monitoring, Incidents, Maintenance, Notifications e IAM.

Capítulo III: Requirements Specification

3.1. User Stories

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US01	Registro de administrador con validación de edificio	Como administrador, quiero registrarme en CodeNova asociando mi edificio, para habilitar el monitoreo de mis equipos desde el primer día.	Escenario (Éxito):Dado que el administrador completa los datos del edificio y su cuenta, Cuando presiona 'Crear cuenta', Entonces el sistema crea el edificio y envía un correo de confirmación. Escenario (Fracaso):Dado que el correo ingresado ya está registrado, Cuando intenta crear la cuenta, Entonces el sistema muestra 'Este correo ya tiene una cuenta asociada'.	EPCN01

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US02	Inicio de sesión por rol	Como usuario registrado (administrador, residente o empresa de mantenimiento), quiero iniciar sesión con mi correo y contraseña, para acceder a las funciones de mi rol.	Escenario (Éxito):Dado que el usuario ingresa credenciales correctas, Cuando presiona ‘Iniciar sesión’, Entonces el sistema lo redirige al dashboard de su rol. Escenario (Fracaso):Dado que el usuario ingresa una contraseña incorrecta, Cuando presiona ‘Iniciar sesión’, Entonces el sistema muestra ‘Correo o contraseña incorrectos’.	EPCN01

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US03	Recuperación de contraseña	Como usuario registrado, quiero recuperar mi contraseña olvidada, para volver a acceder a mi cuenta.	Escenario (Éxito): Dado que el usuario ingresa su correo registrado, Cuando presiona 'Enviar enlace', Entonces el sistema envía un correo de recuperación válido por 24 horas. Escenario (Fracaso): Dado que el usuario usa un enlace de recuperación vencido, Cuando intenta acceder a él, Entonces el sistema muestra 'Este enlace ha expirado. Solicita uno nuevo'.	EPCN01

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US04	Registro de residente vinculado a su unidad	Como residente, quiero registrarme indicando mi edificio y número de departamento, para reportar incidentes y hacer seguimiento a mis solicitudes.	Escenario (Éxito): Dado que el residente ingresa un código de edificio válido y un número de unidad existente, Cuando presiona 'Registrarme', Entonces el sistema crea la cuenta y notifica al administrador. Escenario (Fracaso): Dado que el residente ingresa un código de edificio inexistente, Cuando intenta registrarse, Entonces el sistema muestra 'Código de edificio no válido'.	EPCN01

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US05	Registro de empresa de mantenimiento	Como empresa de mantenimiento, quiero registrarme y asociarme a uno o más edificios, para recibir alertas y gestionar mis visitas desde la plataforma.	Escenario (Éxito): Dado que la empresa completa el formulario de registro con su RUC y edificios asignados, Cuando presiona 'Registrar empresa', Entonces el sistema crea la cuenta y la vincula a los edificios indicados. Escenario (Fracaso): Dado que el RUC ingresado no tiene el formato correcto, Cuando intenta registrarse, Entonces el sistema muestra 'Ingresa un RUC válido de 11 dígitos'.	EPCN01

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US06	Edición de datos de perfil	Como usuario registrado, quiero editar mis datos de perfil (nombre, teléfono, correo), para mantener actualizada mi información de contacto.	Escenario (Éxito): Dado que el usuario modifica un campo válido de su perfil, Cuando presiona 'Guardar cambios', Entonces el sistema actualiza la información y confirma el cambio. Escenario (Fracaso): Dado que el usuario ingresa un número de teléfono con formato inválido, Cuando intenta guardar, Entonces el sistema muestra 'Ingresa un número de teléfono válido'.	EPCN01

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US07	Registro de un edificio y sus datos generales	Como administrador, quiero registrar los datos generales de mi edificio (dirección, número de unidades, distrito), para que la plataforma configure correctamente el monitoreo.	Escenario (Éxito): Dado que el administrador completa los datos del edificio, Cuando presiona 'Guardar edificio', Entonces el sistema lo registra y lo muestra en su panel. Escenario (Fracaso): Dado que el administrador deja el campo de dirección vacío, Cuando intenta guardar, Entonces el sistema muestra 'La dirección es obligatoria'.	EPCN02

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US08	Registro de equipos críticos del edificio	Como administrador, quiero registrar los equipos críticos de mi edificio (bombas, tableros, ascensores, aires acondicionados), para que la plataforma pueda asociarles sensores y generar alertas.	Escenario (Éxito): Dado que el administrador completa el tipo y ubicación del equipo, Cuando presiona 'Guardar equipo', Entonces el sistema lo agrega a la lista de equipos del edificio con un código único. Escenario (Fracaso): Dado que el administrador no selecciona un tipo de equipo, Cuando intenta guardar, Entonces el sistema muestra 'Selecciona el tipo de equipo'.	EPCN02

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US09	Edición o baja de un equipo registrado	Como administrador, quiero editar o dar de baja un equipo registrado, para mantener actualizado el inventario cuando se reemplaza o retira un equipo.	Escenario (Éxito): Dado que el administrador marca un equipo como ‘dado de baja’, Cuando confirma la acción, Entonces el sistema deja de generar alertas para ese equipo, conservando su historial. Escenario (Fracaso): Dado que el equipo tiene una visita de mantenimiento programada activa, Cuando el administrador intenta darlo de baja, Entonces el sistema muestra ‘No puedes dar de baja un equipo con una visita programada activa’.	EPCN02

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US10	Asociación de sensores a un equipo específico	Como administrador, quiero asociar uno o más sensores IoT a un equipo registrado, para que las lecturas se vinculen correctamente y generen alertas sobre ese equipo.	Escenario (Éxito): Dado que el administrador ingresa un identificador de sensor válido y no utilizado, Cuando confirma la asociación, Entonces el sensor queda vinculado al equipo. Escenario (Fracaso): Dado que el identificador de sensor ya está asociado a otro equipo, Cuando el administrador intenta asociarlo, Entonces el sistema muestra 'Este sensor ya está asociado a otro equipo'.	EPCN02

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US11	Consulta del listado de equipos por edificio	Como administrador, quiero ver el listado completo de equipos registrados en mi edificio con su estado actual, para tener una visión general del parque de equipos.	Escenario (Éxito): Dado que el edificio tiene equipos registrados, Cuando el administrador accede a ‘Mis equipos’, Entonces el sistema muestra la lista con su tipo, ubicación y estado. Escenario (Fracaso): Dado que el edificio no tiene equipos registrados, Cuando el administrador accede a ‘Mis equipos’, Entonces el sistema muestra ‘Aún no has registrado equipos en este edificio’.	EPCN02

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US12	Recepción y registro de lecturas de sensores	Como sistema, quiero recibir y almacenar las lecturas de vibración, temperatura, humedad y consumo eléctrico enviadas por los sensores, para contar con el historial necesario para generar alertas.	Escenario (Éxito): Dado que un sensor envía una lectura dentro de rango físico válido, Cuando el sistema la recibe, Entonces la almacena asociada al equipo correspondiente con marca de tiempo. Escenario (Fracaso): Dado que un sensor envía un valor fuera de rango físico posible, Cuando el sistema la recibe, Entonces la descarta y la marca como lectura errónea.	EPCN03

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US13	Detección de sensor desconectado	Como sistema, quiero detectar cuando un sensor deja de enviar lecturas durante un periodo prolongado, para notificar al administrador que el monitoreo de ese equipo está interrumpido.	Escenario (Éxito): Dado que un sensor no envía lecturas durante más de 24 horas, Cuando el sistema evalúa su estado, Entonces genera una notificación de 'sensor desconectado' al administrador. Escenario (Fracaso): Dado que el sensor retoma el envío de lecturas antes de las 24 horas, Cuando el sistema evalúa su estado, Entonces no genera ninguna notificación.	EPCN03

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US14	Consulta del historial de lecturas de un equipo	Como administrador, quiero consultar el historial de lecturas de un equipo en un rango de fechas, para entender su comportamiento a lo largo del tiempo.	Escenario (Éxito): Dado que el equipo tiene lecturas registradas en el rango solicitado, Cuando el administrador aplica el filtro de fechas, Entonces el sistema muestra el gráfico de lecturas correspondiente. Escenario (Fracaso): Dado que no existen lecturas en el rango de fechas solicitado, Cuando el administrador aplica el filtro, Entonces el sistema muestra 'No hay lecturas registradas en este periodo'.	EPCN03

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US15	Configuración de rangos esperados por tipo de equipo	Como sistema, quiero mantener rangos de referencia configurables por tipo de equipo (vibración, temperatura, amperaje), para poder comparar cada lectura contra un valor esperado.	Escenario (Éxito):Dado que un administrador de plataforma actualiza el rango esperado de un tipo de equipo, Cuando guarda el cambio, Entonces el sistema aplica el nuevo rango a las evaluaciones futuras. Escenario (Fracaso):Dado que el rango mínimo ingresado es mayor que el rango máximo, Cuando se intenta guardar, Entonces el sistema muestra 'El valor mínimo no puede ser mayor al máximo'.	EPCN03

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US16	Visualización de gráfico de lecturas en tiempo real	Como administrador, quiero ver un gráfico en tiempo real de la última lectura de un equipo, para monitorear su comportamiento durante el día.	Escenario (Éxito): Dado que el equipo tiene un sensor activo, Cuando el administrador abre el detalle del equipo, Entonces el sistema muestra el gráfico actualizado con la última lectura recibida. Escenario (Fracaso): Dado que el equipo no tiene ningún sensor asociado, Cuando el administrador abre su detalle, Entonces el sistema muestra 'Este equipo no tiene sensores asociados'.	EPCN03

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US17	Generación automática de alertas por severidad	Como sistema, quiero comparar cada lectura contra los rangos esperados del equipo y generar una alerta con nivel de severidad cuando se supere el umbral, para anticipar fallas antes de que ocurran.	Escenario (Éxito): Dado que una lectura de vibración supera el umbral crítico configurado, Cuando el sistema la evalúa, Entonces genera una alerta de severidad Crítica. Escenario (Fracaso): Dado que la lectura se mantiene dentro del rango esperado, Cuando el sistema la evalúa, Entonces no genera ninguna alerta.	EPCN04

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US18	Actualización de una alerta activa	Como sistema, quiero actualizar una alerta ya activa cuando llegan nuevas lecturas relacionadas con la misma anomalía, para evitar duplicar alertas del mismo problema.	Escenario (Éxito): Dado que ya existe una alerta activa para un equipo, Cuando llega una nueva lectura que confirma el mismo problema, Entonces el sistema actualiza la severidad de la alerta existente en vez de crear una nueva. Escenario (Fracaso): Dado que la nueva lectura corresponde a una anomalía distinta, Cuando el sistema la evalúa, Entonces genera una alerta independiente.	EPCN04

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US19	Visualización de alertas activas en el dashboard	Como administrador, quiero ver en mi dashboard las alertas activas de mi edificio ordenadas por severidad, para decidir qué equipo requiere atención primero.	Escenario (Éxito): Dado que el edificio tiene alertas activas, Cuando el administrador ingresa al dashboard, Entonces el sistema muestra la lista ordenada de mayor a menor severidad. Escenario (Fracaso): Dado que no hay alertas activas, Cuando el administrador ingresa al dashboard, Entonces el sistema muestra 'Sin alertas activas' y el estado general del edificio.	EPCN04

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US20	Cambio de estado de una alerta	Como administrador, quiero marcar una alerta como ‘en gestión’ o ‘resuelta’, para reflejar el avance real de la atención del problema.	Escenario (Éxito): Dado que el administrador selecciona una alerta activa, Cuando cambia su estado a ‘en gestión’, Entonces el sistema actualiza el estado y lo refleja en el dashboard. Escenario (Fracaso): Dado que la alerta ya fue marcada como ‘resuelta’, Cuando el administrador intenta cambiar su estado nuevamente, Entonces el sistema muestra ‘Esta alerta ya fue cerrada’.	EPCN04

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US21	Notificación de alerta priorizada a la empresa de mantenimiento	Como empresa de mantenimiento, quiero recibir una notificación cuando se genera una alerta de severidad Alta o Crítica en un edificio que atiendo, para priorizar la visita.	Escenario (Éxito): Dado que se genera una alerta Crítica en un edificio con empresa de mantenimiento asignada, Cuando el sistema la procesa, Entonces envía la notificación correspondiente con el detalle del equipo. Escenario (Fracaso): Dado que el edificio no tiene empresa de mantenimiento asignada, Cuando se genera la alerta, Entonces el sistema solo notifica al administrador.	EPCN04

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US22	Configuración de umbrales de severidad por edificio	Como administrador, quiero ajustar la sensibilidad de las alertas de mi edificio dentro de rangos permitidos, para adaptar el sistema a las particularidades de mis equipos.	Escenario (Éxito): Dado que el administrador ajusta un umbral dentro del rango permitido por el sistema, Cuando guarda el cambio, Entonces el sistema aplica el nuevo umbral a las evaluaciones futuras. Escenario (Fracaso): Dado que el administrador intenta ajustar un umbral fuera del rango permitido, Cuando intenta guardar, Entonces el sistema muestra 'El valor ingresado está fuera del rango permitido'.	EPCN04

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US23	Reporte de incidente por parte del residente	Como residente, quiero reportar un incidente indicando el tipo de problema y una breve descripción, para que la administración tome conocimiento y lo gestione.	Escenario (Éxito): Dado que el residente completa el formulario con categoría y descripción, Cuando presiona 'Enviar', Entonces el sistema confirma la recepción y notifica al administrador. Escenario (Fracaso): Dado que el residente no selecciona una categoría, Cuando intenta enviar el reporte, Entonces el sistema muestra 'Selecciona un tipo de incidente'.	EPCN05

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US24	Adjuntar foto a un reporte de incidente	Como residente, quiero adjuntar una foto a mi reporte de incidente, para que el administrador entienda mejor el problema.	Escenario (Éxito): Dado que el residente adjunta una imagen en formato válido y de tamaño permitido, Cuando envía el reporte, Entonces el sistema la almacena junto con el incidente. Escenario (Fracaso): Dado que el residente intenta adjuntar un archivo que excede el tamaño permitido, Cuando intenta enviarlo, Entonces el sistema muestra 'El archivo supera el tamaño máximo permitido (5MB)'.	EPCN05

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US25	Seguimiento del estado de un incidente reportado	Como residente, quiero ver el estado de mi incidente reportado (recibido, en gestión, resuelto), para saber si ya está siendo atendido sin llamar a la administración.	Escenario (Éxito): Dado que el administrador marca un incidente como ‘en gestión’, Cuando el sistema procesa el cambio, Entonces notifica al residente y actualiza el estado visible. Escenario (Fracaso): Dado que el residente consulta un incidente que no le pertenece, Cuando intenta acceder a su detalle, Entonces el sistema muestra ‘No tienes acceso a esta solicitud’.	EPCN05

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US26	Asignación de un incidente a una visita de mantenimiento	Como administrador, quiero asociar un incidente reportado a una visita de mantenimiento programada o de emergencia, para centralizar la gestión y evitar duplicar solicitudes.	Escenario (Éxito): Dado que el administrador selecciona un incidente y una visita programada, Cuando confirma la asociación, Entonces el estado del incidente cambia a ‘en gestión’ y se notifica al residente. Escenario (Fracaso): Dado que el incidente ya está vinculado a otra visita, Cuando el administrador intenta asignarlo nuevamente, Entonces el sistema muestra ‘Este incidente ya está asignado a una visita’.	EPCN05

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US27	Calificación del incidente resuelto	Como residente, quiero calificar la atención recibida una vez que mi incidente se marca como resuelto, para dar retroalimenta-ción sobre el servicio de mantenimiento.	Escenario (Éxito): Dado que el incidente fue marcado como ‘resuelto’, Cuando el residente selecciona una calificación de 1 a 5 estrellas, Entonces el sistema la registra y la asocia a la intervención. Escenario (Fracaso): Dado que el incidente aún no ha sido marcado como ‘resuelto’, Cuando el residente intenta calificarlo, Entonces el sistema muestra ‘Podrás calificar esta solicitud una vez resuelta’.	EPCN05

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US28	Programación de visita de mantenimiento preventivo	Como administrador, quiero programar una visita de mantenimiento preventivo para un equipo con alerta activa, para intervenir antes de que la falla se agrave.	Escenario (Éxito): Dado que el administrador elige equipo, empresa y fecha propuesta, Cuando confirma la programación, Entonces el sistema crea la visita con estado ‘programada’ y notifica a la empresa. Escenario (Fracaso): Dado que la empresa de mantenimiento no tiene disponibilidad en la fecha propuesta, Cuando el administrador confirma, Entonces el sistema muestra ‘La empresa no tiene disponibilidad en esa fecha, elige otra’.	EPCN06

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US29	Confirmación o reprogramación de una visita	Como empresa de mantenimiento, quiero confirmar o proponer una nueva fecha para una visita programada, para ajustarla a mi disponibilidad real.	Escenario (Éxito): Dado que la empresa recibe una visita propuesta, Cuando confirma la fecha, Entonces el sistema marca la visita como 'confirmada' y notifica al administrador. Escenario (Fracaso): Dado que la empresa propone una nueva fecha fuera del plazo máximo permitido para una alerta Crítica, Cuando intenta guardar, Entonces el sistema muestra 'Esta alerta requiere atención dentro de las próximas 48 horas'.	EPCN06

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US30	Consulta de datos técnicos previos antes de la visita	Como técnico de mantenimiento, quiero consultar las últimas lecturas y el historial del equipo antes de asistir a una visita, para reducir mi tiempo de diagnóstico en campo.	Escenario (Éxito): Dado que el técnico abre una visita asignada, Cuando accede a 'Detalle del equipo', Entonces el sistema muestra las últimas lecturas y el historial de intervenciones. Escenario (Fracaso): Dado que el equipo no tiene intervenciones previas registradas, Cuando el técnico consulta el detalle, Entonces el sistema muestra 'Sin intervenciones previas registradas'.	EPCN06

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US31	Registro de intervención realizada	omo técnico de mantenimiento, quiero registrar la intervención realizada al finalizar una visita, para dejar evidencia del trabajo y actualizar el historial del equipo.	Escenario (Éxito): Dado que el técnico completa la descripción del trabajo y adjunta evidencia, Cuando presiona 'Guardar', Entonces la visita se marca como 'completada' y se notifica al administrador. Escenario (Fracaso): Dado que el técnico no describe el trabajo realizado, Cuando intenta guardar, Entonces el sistema muestra 'Describe brevemente la intervención realizada'.	EPCN06

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US32	Priorización semanal de visitas por severidad y zona	Como técnico de mantenimiento, quiero ver mis visitas de la semana ordenadas por severidad y zona, para organizar mi ruta de trabajo de forma eficiente.	Escenario (Éxito): Dado que el técnico tiene varias visitas asignadas en la semana, Cuando accede a ‘Mi semana’, Entonces el sistema muestra la lista ordenada por severidad y proximidad geográfica. Escenario (Fracaso): Dado que no hay visitas asignadas para la semana, Cuando el técnico accede a ‘Mi semana’, Entonces el sistema muestra ‘Sin visitas programadas esta semana’.	EPCN06

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US33	Cancelación de una visita programada	Como administrador, quiero cancelar una visita programada indicando el motivo, para mantener el registro ordenado cuando el problema se resuelve por otro medio.	Escenario (Éxito): Dado que el administrador selecciona una visita programada y un motivo de cancelación, Cuando confirma la acción, Entonces el sistema marca la visita como 'cancelada' y notifica a la empresa. Escenario (Fracaso): Dado que la visita ya fue marcada como 'completada', Cuando el administrador intenta cancelarla, Entonces el sistema muestra 'No puedes cancelar una visita ya completada'.	EPCN06

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US34	Visualización del ahorro acumulado	Como administrador, quiero ver en mi dashboard el ahorro estimado por hacer mantenimiento preventivo en lugar de correctivo, para tener un indicador objetivo del valor del servicio.	Escenario (Éxito): Dado que se registra una nueva intervención preventiva, Cuando el sistema la procesa, Entonces actualiza el indicador de ahorro acumulado del edificio. Escenario (Fracaso): Dado que el edificio no tiene intervenciones registradas aún, Cuando el administrador consulta el indicador, Entonces el sistema muestra 'Aún no hay suficiente historial para calcular el ahorro'.	EPCN07

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US35	Historial de intervenciones por equipo	Como administrador, quiero consultar el historial completo de intervenciones de un equipo, para tener trazabilidad de su mantenimiento a lo largo del tiempo.	Escenario (Éxito): Dado que el equipo tiene intervenciones previas, Cuando el administrador accede a su ficha, Entonces el sistema muestra el historial completo ordenado por fecha. Escenario (Fracaso): Dado que el administrador aplica un filtro de fechas sin resultados, Cuando lo confirma, Entonces el sistema muestra 'No hay intervenciones en el periodo seleccionado'.	EPCN07

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US36	Generación de reporte para la junta de propietarios	Como administrador, quiero generar un reporte descargable en PDF con el estado del edificio y el ahorro acumulado, para presentarlo en la reunión de junta.	Escenario (Éxito): Dado que el administrador selecciona un rango de fechas y presiona ‘Generar reporte’, Cuando el sistema lo procesa, Entonces genera el PDF y lo pone disponible para descarga. Escenario (Fracaso): Dado que no hay información registrada en el rango seleccionado, Cuando el administrador genera el reporte, Entonces el sistema muestra ‘No hay información para el periodo seleccionado’.	EPCN07

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US37	Generación de reporte para la junta de propietarios	Como administrador de varios edificios, quiero ver un panel comparativo del estado y ahorro de cada uno de mis edificios, para priorizar dónde enfocar mi atención.	Escenario (Éxito): Dado que el administrador gestiona más de un edificio, Cuando accede al panel comparativo, Entonces el sistema muestra una tabla con alertas activas y ahorro acumulado por edificio. Escenario (Fracaso): Dado que el administrador solo gestiona un edificio, Cuando accede al panel comparativo, Entonces el sistema muestra 'Este panel está disponible al gestionar más de un edificio'.	EPCN07

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US38	Exportación de historial de lecturas a CSV	Como administrador, quiero exportar el historial de lecturas de un equipo a un archivo CSV, para analizarlo con otras herramientas si lo necesito.	Escenario (Éxito): Dado que el equipo tiene lecturas registradas, Cuando el administrador presiona 'Exportar a CSV', Entonces el sistema genera el archivo y lo pone disponible para descarga. Escenario (Fracaso): Dado que el equipo no tiene lecturas registradas, Cuando el administrador intenta exportar, Entonces el sistema muestra 'No hay datos para exportar'.	EPCN07

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US39	Vinculación de una empresa de mantenimiento a un edificio	Como administrador, quiero vincular una empresa de mantenimiento a mi edificio, para que reciba las alertas y pueda programar visitas.	Escenario (Éxito): Dado que el administrador busca una empresa registrada por su RUC o nombre, Cuando confirma la vinculación, Entonces el sistema la asocia al edificio y le notifica. Escenario (Fracaso): Dado que la empresa buscada no está registrada en la plataforma, Cuando el administrador intenta vincularla, Entonces el sistema muestra 'No se encontró una empresa con esos datos. Invítala a registrarse'.	EPCN08

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US40	Consulta de historial de intervenciones por cliente	Como empresa de mantenimiento, quiero consultar el historial de intervenciones realizadas en cada edificio cliente, para dar seguimiento a mi trabajo con ellos.	Escenario (Éxito): Dado que la empresa tiene intervenciones registradas en un edificio cliente, Cuando accede a su ficha, Entonces el sistema muestra el historial completo. Escenario (Fracaso): Dado que la empresa no tiene intervenciones registradas en ese edificio, Cuando accede a su ficha, Entonces el sistema muestra 'Aún no registras intervenciones en este edificio'.	EPCN08

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US41	Gestión de técnicos dentro de una empresa de mantenimiento	Como representante de una empresa de mantenimiento, quiero registrar a mis técnicos dentro de la plataforma, para asignarles visitas específicas.	Escenario (Éxito): Dado que el representante completa los datos del técnico, Cuando presiona 'Agregar técnico', Entonces el sistema lo registra y habilita su acceso. Escenario (Fracaso): Dado que el correo del técnico ya está registrado en otra empresa, Cuando el representante intenta agregarlo, Entonces el sistema muestra 'Este correo ya está asociado a otra cuenta'.	EPCN08

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US42	Asignación de una visita a un técnico específico	Como representante de una empresa de mantenimiento, quiero asignar una visita programada a un técnico específico, para distribuir la carga de trabajo del equipo.	Escenario (Éxito): Dado que existe una visita sin técnico asignado, Cuando el representante selecciona un técnico disponible, Entonces el sistema le asigna la visita y lo notifica. Escenario (Fracaso): Dado que el técnico seleccionado ya tiene una visita en el mismo horario, Cuando el representante intenta asignarlo, Entonces el sistema muestra 'Este técnico ya tiene una visita programada en ese horario'.	EPCN08

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US43	Ranking de empresas de mantenimiento por tiempo de respuesta	Como sistema, quiero calcular el tiempo promedio de respuesta de cada empresa de mantenimiento ante una alerta, para que los administradores puedan comparar proveedores.	Escenario (Éxito): Dado que una empresa tiene al menos 5 visitas completadas, Cuando el sistema calcula el indicador, Entonces lo muestra en su perfil visible para los administradores. Escenario (Fracaso): Dado que la empresa tiene menos de 5 visitas completadas, Cuando se consulta su perfil, Entonces el sistema muestra 'Historial insuficiente para calcular este indicador'.	EPCN08

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US44	Notificación push de alerta crítica	Como administrador, quiero recibir una notificación push inmediata cuando se genera una alerta Crítica en mi edificio, para actuar sin demora.	Escenario (Éxito): Dado que se genera una alerta de severidad Crítica, Cuando el sistema la procesa, Entonces envía una notificación push al administrador en los siguientes 2 minutos. Escenario (Fracaso): Dado que el administrador desactivó las notificaciones push, Cuando se genera la alerta, Entonces el sistema envía la notificación únicamente por correo electrónico.	EPCN09

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US45	Configuración de preferencias de notificación	Como usuario registrado, quiero elegir por qué canal recibir mis notificaciones (push, correo, ambos), para adaptarlas a mi forma de trabajo.	Escenario (Éxito): Dado que el usuario selecciona sus canales preferidos, Cuando guarda la configuración, Entonces el sistema aplica la preferencia a partir de ese momento. Escenario (Fracaso): Dado que el usuario desactiva todos los canales de notificación, Cuando intenta guardar, Entonces el sistema muestra 'Debes mantener activo al menos un canal de notificación'.	EPCN09

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US46	Resumen semanal por correo para el administrador	Como administrador, quiero recibir un resumen semanal por correo con las alertas, visitas e incidentes de mi edificio, para mantenerme informado sin revisar la app todos los días.	Escenario (Éxito): Dado que el edificio tuvo actividad durante la semana, Cuando llega el día programado, Entonces el sistema envía el resumen por correo al administrador. Escenario (Fracaso): Dado que el edificio no tuvo ninguna actividad en la semana, Cuando llega el día programado, Entonces el sistema no envía el resumen esa semana.	EPCN09

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US47	Notificación al residente por cambio de estado de su incidente	Como residente, quiero recibir una notificación cada vez que cambia el estado de un incidente que reporté, para estar al tanto sin revisar la app constantemente.	Escenario (Éxito): Dado que el estado de un incidente cambia, Cuando el sistema procesa el cambio, Entonces envía una notificación al residente que lo reportó. Escenario (Fracaso): Dado que el residente desactivó las notificaciones de incidentes, Cuando el estado cambia, Entonces el sistema solo actualiza el estado visible en la app, sin enviar notificación.	EPCN09

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US48	Visualización de la landing page informativa	Como visitante, quiero acceder a información clara sobre CodeNova, sus beneficios y funcionalidades, para comprender la propuesta de valor antes de registrarme.	Escenario (Éxito):Dado que el visitante ingresa a la landing page, Cuando la carga completamente, Entonces el sistema muestra la propuesta de valor, funcionalidades y un botón de registro.Escenario (Fracaso):Dado que el visitante intenta acceder desde un enlace roto, Cuando la página no carga, Entonces el sistema muestra una página de error 404 con un enlace de regreso al inicio.	EPCN10

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US49	Solicitud de demo desde la landing page	Como visitante interesado (administrador o empresa de mantenimiento), quiero solicitar una demo desde la landing page, para conocer la plataforma antes de suscribirme.	Escenario (Éxito): Dado que el visitante completa el formulario de contacto con datos válidos, Cuando presiona 'Solicitar demo', Entonces el sistema registra la solicitud y notifica al equipo comercial. Escenario (Fracaso): Dado que el visitante deja el campo de correo vacío, Cuando intenta enviar el formulario, Entonces el sistema muestra 'Ingresa un correo válido para continuar'.	EPCN10

Story ID	Título	Descripción	Criterios de Aceptación	Epic ID
US50	Onboarding guiado para nuevo administrador	Como administrador que se registra por primera vez, quiero un recorrido guiado dentro de la plataforma, para aprender rápidamente a registrar mi edificio y equipos.	Escenario (Éxito): Dado que el administrador inicia sesión por primera vez, Cuando el sistema detecta que no tiene edificios registrados, Entonces muestra un tutorial guiado paso a paso para crear su primer edificio. Escenario (Fracaso): Dado que el administrador cierra el tutorial antes de finalizarlo, Cuando vuelve a iniciar sesión, Entonces el sistema le ofrece retomarlo desde el panel principal sin forzarlo a completarlo.	EPCN10

3.2. Impact Mapping

[COMPLETAR]

3.3. Product Backlog

El Product Backlog reúne las 50 user stories de la sección 3.1. Las ordenamos por valor para el negocio, no por story points. Los story points solo indican el esfuerzo y usan la serie de Fibonacci (1, 2, 3, 5, 8).

En el AV1 habíamos puesto la autenticación (US01 a US06) entre las primeras posiciones. Para TB1 cambiamos el orden. IAM es un bounded context genérico: cualquier plataforma lo tiene y no es lo que hace valiosa a Vigilia. Lo que sí la diferencia es saber el estado de los equipos y actuar antes de que fallen.

El nuevo orden es este:

1. Landing Page (US48 y US49), que ya se entregó en el Sprint 1.
2. Asset Monitoring: edificios, equipos, sensores, lecturas y alertas.
3. Incidents: reporte, seguimiento y calificación de incidentes.
4. Maintenance: visitas, intervenciones, ahorro estimado y empresas de mantenimiento.
5. Notifications: avisos dentro de la aplicación y por correo.
6. Reportes y consultas (US36, US37, US38, US43 y US46). Van al final porque no agregan clases nuevas: leen datos que ya existen.
7. IAM (US01 a US06) y el onboarding (US50), que depende de tener cuentas.

Dentro de cada contexto, primero van las historias que el administrador usa todos los días y que no dependen del backend. Por eso US07, US08, US11, US09, US19 y US20 aparecen antes que la recepción de lecturas (US12) o la generación automática de alertas (US17), que son reglas del REST API.

Orden	User Story ID	Título	Descripción	Story Points
1	US48	Visualización de la landing page informativa	Como visitante, quiero acceder a información clara sobre CodeNova, sus beneficios y funcionalidades, para comprender la propuesta de valor antes de registrarme.	2
2	US49	Solicitud de demo desde la landing page	Como visitante interesado (administrador o empresa de mantenimiento), quiero solicitar una demo desde la landing page, para conocer la plataforma antes de suscribirme.	2

Orden	User Story ID	Título	Descripción	Story Points
3	US07	Registro de un edificio y sus datos generales	Como administrador, quiero registrar los datos generales de mi edificio (dirección, número de unidades, distrito), para que la plataforma configure correctamente el monitoreo.	2
4	US08	Registro de equipos críticos del edificio	Como administrador, quiero registrar los equipos críticos de mi edificio (bombas, tableros, ascensores, aires acondicionados), para que la plataforma pueda asociarles sensores y generar alertas.	3

Orden	User Story ID	Título	Descripción	Story Points
5	US11	Consulta del listado de equipos por edificio	Como administrador, quiero ver el listado completo de equipos registrados en mi edificio con su estado actual, para tener una visión general del parque de equipos.	2
6	US09	Edición o baja de un equipo registrado	Como administrador, quiero editar o dar de baja un equipo registrado, para mantener actualizado el inventario cuando se reemplaza o retira un equipo.	3

Orden	User Story ID	Título	Descripción	Story Points
7	US19	Visualización de alertas activas en el dashboard	Como administrador, quiero ver en mi dashboard las alertas activas de mi edificio ordenadas por severidad, para decidir qué equipo requiere atención primero.	3
8	US20	Cambio de estado de una alerta	Como administrador, quiero marcar una alerta como ‘en gestión’ o ‘resuelta’, para reflejar el avance real de la atención del problema.	2

Orden	User Story ID	Título	Descripción	Story Points
9	US10	Asociación de sensores a un equipo específico	Como administrador, quiero asociar uno o más sensores IoT a un equipo registrado, para que las lecturas se vinculen correctamente y generen alertas sobre ese equipo.	3
10	US14	Consulta del historial de lecturas de un equipo	Como administrador, quiero consultar el historial de lecturas de un equipo en un rango de fechas, para entender su comportamiento a lo largo del tiempo.	3

Orden	User Story ID	Título	Descripción	Story Points
11	US23	Reporte de incidente por parte del residente	Como residente, quiero reportar un incidente indicando el tipo de problema y una breve descripción, para que la administración tome conocimiento y lo gestione.	3
12	US25	Seguimiento del estado de un incidente reportado	Como residente, quiero ver el estado de mi incidente reportado (recibido, en gestión, resuelto), para saber si ya está siendo atendido sin llamar a la administración.	3

Orden	User Story ID	Título	Descripción	Story Points
13	US27	Calificación del incidente resuelto	Como residente, quiero calificar la atención recibida una vez que mi incidente se marca como resuelto, para dar retroalimentación sobre el servicio de mantenimiento.	2
14	US12	Recepción y registro de lecturas de sensores	Como sistema, quiero recibir y almacenar las lecturas de vibración, temperatura, humedad y consumo eléctrico enviadas por los sensores, para contar con el historial necesario para generar alertas.	8

Orden	User Story ID	Título	Descripción	Story Points
15	US17	Generación automática de alertas por severidad	Como sistema, quiero comparar cada lectura contra los rangos esperados del equipo y generar una alerta con nivel de severidad cuando se supere el umbral, para anticipar fallas antes de que ocurran.	8
16	US18	Actualización de una alerta activa	Como sistema, quiero actualizar una alerta ya activa cuando llegan nuevas lecturas relacionadas con la misma anomalía, para evitar duplicar alertas del mismo problema.	5

Orden	User Story ID	Título	Descripción	Story Points
17	US13	Detección de sensor desconectado	Como sistema, quiero detectar cuando un sensor deja de enviar lecturas durante un periodo prolongado, para notificar al administrador que el monitoreo de ese equipo está interrumpido.	5
18	US15	Configuración de umbrales de cada equipo	Como administrador, quiero configurar los umbrales de cada equipo por métrica (vibración, temperatura, humedad y consumo eléctrico), para que las alertas respondan a la realidad de mis equipos.	5

Orden	User Story ID	Título	Descripción	Story Points
19	US16	Visualización de gráfico de lecturas en tiempo real	Como administrador, quiero ver un gráfico en tiempo real de la última lectura de un equipo, para monitorear su comportamiento durante el día.	5
20	US24	Adjuntar foto a un reporte de incidente	Como residente, quiero adjuntar una foto a mi reporte de incidente, para que el administrador entienda mejor el problema.	2

Orden	User Story ID	Título	Descripción	Story Points
21	US26	Asignación de un incidente a una visita de mantenimiento	Como administrador, quiero asociar un incidente reportado a una visita de mantenimiento programada o de emergencia, para centralizar la gestión y evitar duplicar solicitudes.	3
22	US28	Programación de visita de mantenimiento preventivo	Como administrador, quiero programar una visita de mantenimiento preventivo para un equipo con alerta activa, para intervenir antes de que la falla se agrave.	5

Orden	User Story ID	Título	Descripción	Story Points
23	US29	Confirmación o reprograma-ción de una visita	Como empresa de manteni-miento, quiero confirmar o proponer una nueva fecha para una visita programada, para ajustarla a mi disponibilidad real.	3
24	US31	Registro de intervención realizada	omo técnico de mantenimien-to, quiero registrar la intervención realizada al finalizar una visita, para dejar evidencia del trabajo y actualizar el historial del equipo.	5

Orden	User Story ID	Título	Descripción	Story Points
25	US34	Visualización del ahorro acumulado	Como administrador, quiero ver en mi dashboard el ahorro estimado por hacer mantenimiento preventivo en lugar de correctivo, para tener un indicador objetivo del valor del servicio.	8
26	US30	Consulta de datos técnicos previos antes de la visita	Como técnico de mantenimiento, quiero consultar las últimas lecturas y el historial del equipo antes de asistir a una visita, para reducir mi tiempo de diagnóstico en campo.	3

Orden	User Story ID	Título	Descripción	Story Points
27	US33	Cancelación de una visita programada	Como administrador, quiero cancelar una visita programada indicando el motivo, para mantener el registro ordenado cuando el problema se resuelve por otro medio.	2
28	US39	Vinculación de una empresa de mantenimiento a un edificio	Como administrador, quiero vincular una empresa de mantenimiento a mi edificio, para que reciba las alertas y pueda programar visitas.	3

Orden	User Story ID	Título	Descripción	Story Points
29	US42	Asignación de una visita a un técnico específico	Como representante de una empresa de mantenimiento, quiero asignar una visita programada a un técnico específico, para distribuir la carga de trabajo del equipo.	3
30	US41	Gestión de técnicos dentro de una empresa de mantenimiento	Como representante de una empresa de mantenimiento, quiero registrar a mis técnicos dentro de la plataforma, para asignarles visitas específicas.	3

Orden	User Story ID	Título	Descripción	Story Points
31	US32	Priorización semanal de visitas por severidad y zona	Como técnico de mantenimiento, quiero ver mis visitas de la semana ordenadas por severidad y zona, para organizar mi ruta de trabajo de forma eficiente.	5
32	US35	Historial de intervenciones por equipo	Como administrador, quiero consultar el historial completo de intervenciones de un equipo, para tener trazabilidad de su mantenimiento a lo largo del tiempo.	3

Orden	User Story ID	Título	Descripción	Story Points
33	US40	Consulta de historial de intervenciones por cliente	Como empresa de mantenimiento, quiero consultar el historial de intervenciones realizadas en cada edificio cliente, para dar seguimiento a mi trabajo con ellos.	3
34	US21	Notificación de alerta priorizada a la empresa de mantenimiento	Como empresa de mantenimiento, quiero recibir una notificación cuando se genera una alerta de severidad Alta o Crítica en un edificio que atiendo, para priorizar la visita.	5

Orden	User Story ID	Título	Descripción	Story Points
35	US44	Aviso de alerta crítica dentro de la aplicación y por correo	Como administrador, quiero recibir un aviso inmediato dentro de la aplicación y por correo cuando se genera una alerta Crítica en mi edificio, para actuar sin demora.	3
36	US47	Notificación al residente por cambio de estado de su incidente	Como residente, quiero recibir una notificación cada vez que cambia el estado de un incidente que reporté, para estar al tanto sin revisar la app constante-mente.	2

Orden	User Story ID	Título	Descripción	Story Points
37	US45	Configuración de preferencias de notificación	Como usuario registrado, quiero elegir por qué canal recibir mis notificaciones (dentro de la aplicación, correo o ambos), para adaptarlas a mi forma de trabajo.	2
38	US36	Generación de reporte para la junta de propietarios	Como administrador, quiero generar un reporte descargable en PDF con el estado del edificio y el ahorro acumulado, para presentarlo en la reunión de junta.	5

Orden	User Story ID	Título	Descripción	Story Points
39	US37	Panel comparativo entre edificios	Como administrador de varios edificios, quiero ver un panel comparativo del estado y ahorro de cada uno de mis edificios, para priorizar dónde enfocar mi atención.	5
40	US38	Exportación de historial de lecturas a CSV	Como administrador, quiero exportar el historial de lecturas de un equipo a un archivo CSV, para analizarlo con otras herramientas si lo necesito.	3

Orden	User Story ID	Título	Descripción	Story Points
41	US43	Ranking de empresas de mantenimiento por tiempo de respuesta	Como sistema, quiero calcular el tiempo promedio de respuesta de cada empresa de mantenimiento ante una alerta, para que los administradores puedan comparar proveedores.	5
42	US46	Resumen semanal por correo para el administrador	Como administrador, quiero recibir un resumen semanal por correo con las alertas, visitas e incidentes de mi edificio, para mantenerme informado sin revisar la app todos los días.	3

Orden	User Story ID	Título	Descripción	Story Points
43	US01	Registro de administrador con validación de edificio	Como administrador, quiero registrarme en CodeNova asociando mi edificio, para habilitar el monitoreo de mis equipos desde el primer día.	3
44	US02	Inicio de sesión por rol	Como usuario registrado (administrador, residente o empresa de mantenimien- to), quiero iniciar sesión con mi correo y contraseña, para acceder a las funciones de mi rol.	3
45	US03	Recuperación de contraseña	Como usuario registrado, quiero recuperar mi contraseña olvidada, para volver a acceder a mi cuenta.	2

Orden	User Story ID	Título	Descripción	Story Points
46	US04	Registro de residente vinculado a su unidad	Como residente, quiero registrarme indicando mi edificio y número de departamento, para reportar incidentes y hacer seguimiento a mis solicitudes.	3
47	US05	Registro de empresa de mantenimien-to	Como empresa de manteni-miento, quiero registrarme y asociarme a uno o más edificios, para recibir alertas y gestionar mis visitas desde la plataforma.	3
48	US06	Edición de datos de perfil	Como usuario registrado, quiero editar mis datos de perfil (nombre, teléfono, correo), para mantener actualizada mi información de contacto.	2

Orden	User Story ID	Título	Descripción	Story Points
49	US50	Onboarding guiado para nuevo administrador	Como administrador que se registra por primera vez, quiero un recorrido guiado dentro de la plataforma, para aprender rápidamente a registrar mi edificio y equipos.	3

Capítulo IV: Product Design

4.1. Style Guidelines

El diseño gráfico de la plataforma CodeNova fue definido por el equipo mediante la aplicación de distintas estrategias orientadas a garantizar una estética coherente, una interfaz clara y una experiencia visual que transmita control y prevención, sin caer en la alarma constante. El diseño de nuestro logotipo busca encapsular los conceptos de monitoreo, prevención y confianza, que son los pilares de la plataforma.

Para la paleta de colores, se ha elegido un azul principal que transmite confianza, estabilidad y tecnología, elementos cruciales para una herramienta que administradores y empresas de mantenimiento usarán para tomar decisiones sobre equipos críticos. Este se complementa con un verde de acento que evoca prevención y control (‘todo en orden’), y con una escala semántica de severidad (azul, amarillo, naranja y rojo, más verde para lo normal) que es central en la propuesta de valor de CodeNova, ya que las alertas priorizadas por severidad son el corazón funcional del producto. El blanco y el gris claro aportan limpieza visual y mejoran la legibilidad de paneles con datos técnicos. La combinación busca proyectar una imagen seria y confiable, pero accesible incluso para administradores y residentes sin formación técnica.

4.1.1. General Style Guidelines

Aquí se sientan las bases de la identidad visual y verbal de la plataforma, asegurando consistencia en todas las pantallas. Nos hemos basado en principios de diseño inclusivo, considerando que CodeNova será usado tanto por perfiles técnicos (empresas de mantenimiento) como por usuarios sin formación técnica (residentes).

Branding El logo debe representar el monitoreo preventivo y la conexión entre los actores del edificio. Se propone un diseño que combine dos elementos: un contorno simplificado de edificio (representando el condominio monitoreado) atravesado por una línea de pulso o señal (representando la lectura constante de los sensores IoT). Este concepto visual refuerza la propuesta de valor central: anticipar el desgaste antes de que se convierta en una falla.

Figura 28a. Logo Vigilia — versión horizontal (isotipo + wordmark)

Typography

Se propone utilizar la familia tipográfica IBM Plex Sans, una fuente sans-serif de carácter técnico e industrial, pero con buena legibilidad en pantallas. Su estilo sobrio se alinea con el contexto de infraestructura y mantenimiento de equipos, sin perder calidez ni cercanía para los residentes.

- Escala base: 16px
- Interlineado: 1.5
- Weights (pesos): Regular, Medium, SemiBold, Bold Nomenclatura tipográfica (NOMBRE / TAMAÑO / PESO):
- Heading 1: 32px / Bold (títulos de página principal, ej. nombre del edificio en el dashboard)
- Heading 2: 24px / SemiBold (títulos de sección, ej. 'Alertas activas')
- Heading 3: 20px / SemiBold (subtítulos o títulos de tarjetas de equipo)
- Base: 16px / Regular (párrafos y texto principal)
- Label: 14px / Medium (etiquetas de botones, campos de formulario y chips de severidad)

Colors

La paleta de colores está pensada para transmitir confianza y control, y para que la severidad de una alerta se reconozca de un vistazo. - Azul Primario (#0F5C78): refuerza la sensación de estabilidad y tecnología (botones principales, navegación). - Verde Prevención (#2E9E6D): indica que un equipo está en estado normal o que una acción preventiva fue exitosa. - Gris Claro (#F4F6F8): proporciona limpieza visual (fondos). - Gris Oscuro (#22303C): asegura legibilidad óptima (textos). Semántica de severidad (elemento diferenciador de CodeNova, usado en alertas y en el estado tipo semáforo del residente): - Verde (#2E9E6D): Normal / sin alerta. - Azul (#1F4E9C): Severidad Baja (LOW). - Amarillo (#F4B400): Severidad Media (MEDIUM). - Naranja (#F2994A): Severidad Alta (HIGH). - Rojo (#E53935): Severidad Crítica (CRITICAL).

Son cuatro niveles, los mismos del modelo (sección 4.7). En la aplicación cada chip usa un fondo claro del mismo tono para que el texto se lea bien.

Spacing

La unidad base de 8px establece un ritmo visual que facilita la comprensión. -8px: entre íconos y texto. - 16px: entre párrafos y elementos de lista. - 24px: relleno interno en tarjetas de equipo y alertas. - 32px: separación entre secciones principales del dashboard. - 48px: márgenes superiores e inferiores de la página. - **Tono de Comunicación y Lenguaje Aplicado**

El tono debe ser Claro, Tranquilizador y Profesional. Buscamos un equilibrio que transmita control ante una alerta, sin sonar alarmista, y que sea comprensible tanto para un técnico como para un residente sin conocimientos técnicos. - Tono: Sereno y confiable, incluso al comunicar una alerta crítica. Se evita el lenguaje alarmista tipo '¡Peligro!' y se prioriza un lenguaje orientado a la acción ('Requiere atención', 'Programar visita'). - Lenguaje: Claro y sencillo, evitando tecnicismos innecesarios para el

residente (ej. ‘Hay una alerta en la bomba de agua’ en vez de ‘Lectura de vibración fuera de rango en equipo HB-04’).

4.1.2. Web Style Guidelines

Elegimos estos colores porque buscábamos transmitir confianza, prevención y claridad ante la urgencia. El azul simboliza estabilidad y es el color predominante en la navegación. El verde refuerza la sensación de ‘todo bajo control’. La escala de severidad (azul, amarillo, naranja y rojo) permite que administradores y empresas de mantenimiento prioricen de un vistazo, sin tener que leer cada alerta en detalle.

Responsive Design Standards (Mobile-first)

- Mobile (hasta 768px): diseño de una sola columna. Menú de navegación inferior (tab bar) para acceso rápido a Alertas, Edificios e Incidentes. Botones grandes para uso táctil, pensado en el técnico usando la app en campo.
- Tablet (769px - 1024px): layout de hasta dos columnas. Menú lateral colapsable.
- Desktop (1025px+): layouts de dos o tres columnas, con el panel de alertas y el mapa/listado de equipos visibles simultáneamente. Navegación principal siempre visible.

Interactivity

- Botones: bordes redondeados (border-radius: 8px). Hover con ligera sombra o cambio de tono.
- Chips de severidad: forma de píldora (pill) con color de fondo semántico e ícono, para reforzar el reconocimiento visual inmediato.
- Transiciones: animaciones sutiles y rápidas (200-300ms), evitando efectos llamativos que distraigan ante una alerta real.

Accessibility

- Etiquetas () claras en todos los formularios, especialmente en el reporte de incidentes del residente.
- Imágenes y evidencias fotográficas con texto alternativo (alt).
- Navegación completa mediante teclado (Tab, Enter) para los paneles de administrador.
- Contraste de color según pautas WCAG 2.1, verificado especialmente en los chips de severidad sobre fondo claro.

4.2. Information Architecture

En esta sección explicamos cómo organizamos el contenido del Landing Page y de la Web Application para que cada usuario encuentre rápido lo que necesita. Esto importa porque muchas veces el usuario entra en un momento de estrés: una alerta, una falla o un reclamo.

Diferenciamos dos cosas. El **diseño objetivo** es la aplicación completa, con las vistas de los tres roles. Lo **implementado en el Sprint 2** es la parte del administrador para Asset Monitoring y la parte del residente para Incidents. Cuando algo todavía no está construido, lo indicamos.

4.2.1. Organization Systems

Combinamos cuatro formas de organizar la información:

- **Jerárquica.** Es la principal: edificio → equipo → sensores, lecturas y alertas. En la aplicación se ve en el filtro de equipos por edificio y en el historial de lecturas por equipo.
- **Secuencial.** Para las tareas con pasos: reportar un incidente, asignar un sensor o, más adelante, programar una visita.
- **Por categoría.** Los equipos se agrupan por tipo (bomba de agua, tablero eléctrico, ascensor y HVAC) y los incidentes por categoría (agua, electricidad, ascensor, aire acondicionado y otro).
- **Cronológica.** Para el historial de lecturas y de incidentes, del más reciente al más antiguo.

Además, el diseño objetivo se organiza **por audiencia**:

- **Administrador.** Entra al panel de alertas, ordenado por severidad. Revisa el equipo afectado y, desde ahí, más adelante podrá programar una visita y ver el ahorro acumulado para la junta.
- **Residente.** Entra cuando nota una falla. Reporta el incidente, sigue su estado y lo califica cuando está resuelto.
- **Empresa de mantenimiento.** Entra por una alerta priorizada. Revisa su semana ordenada por severidad y zona, consulta los datos del equipo antes de la visita y registra la intervención al terminar.

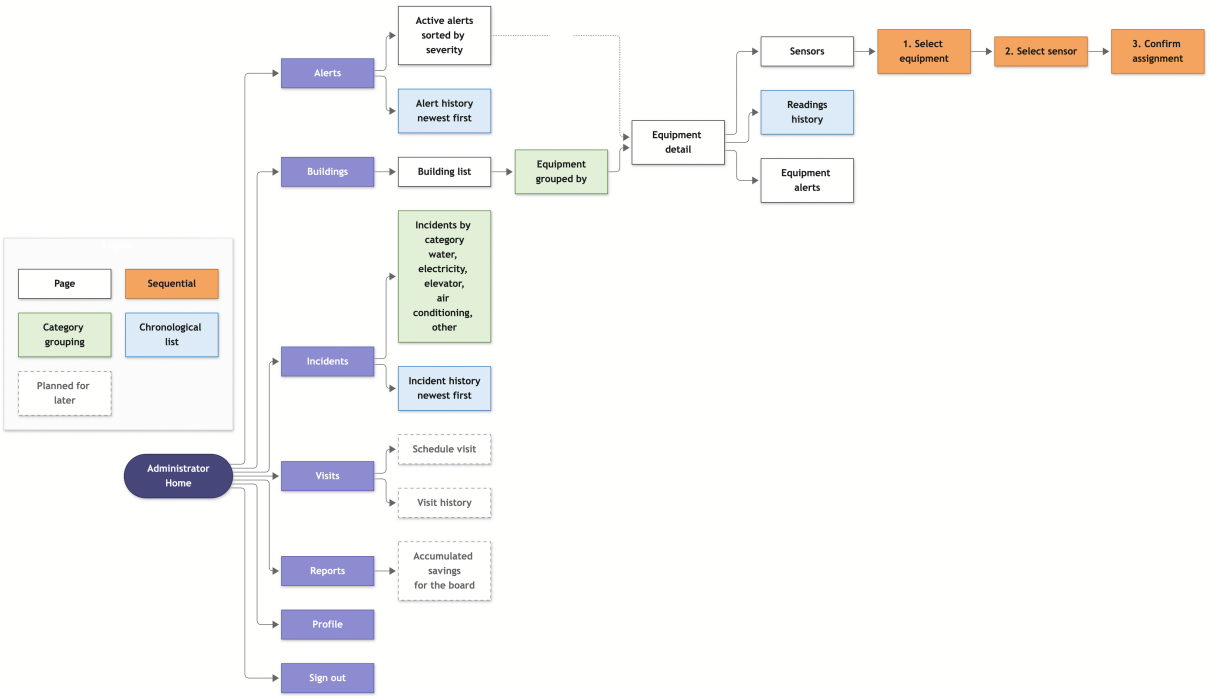


Figura 8: Figura 4.2.1-1 – Arquitectura del diseño objetivo para el administrador

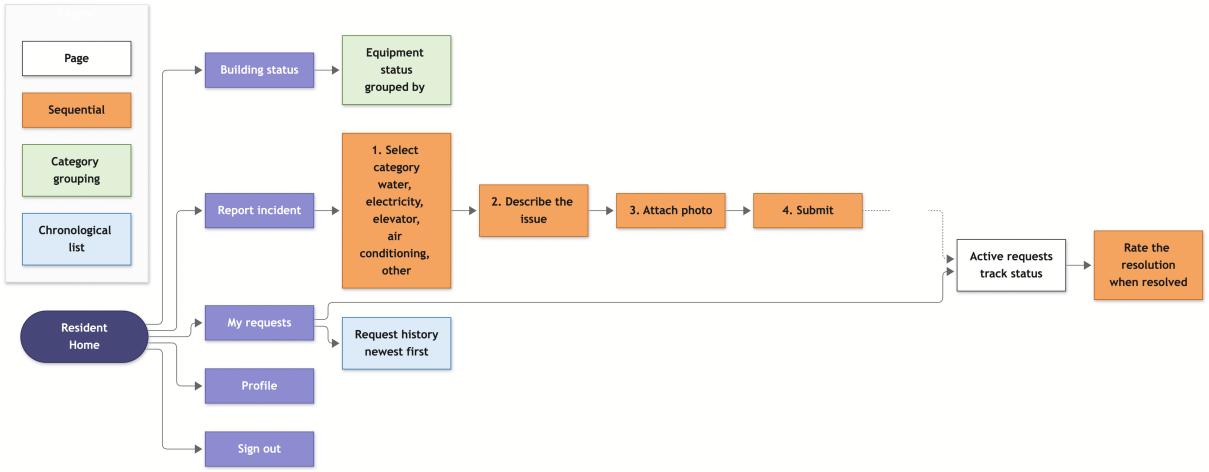


Figura 9: Figura 4.2.1-2 – Arquitectura del diseño objetivo para el residente

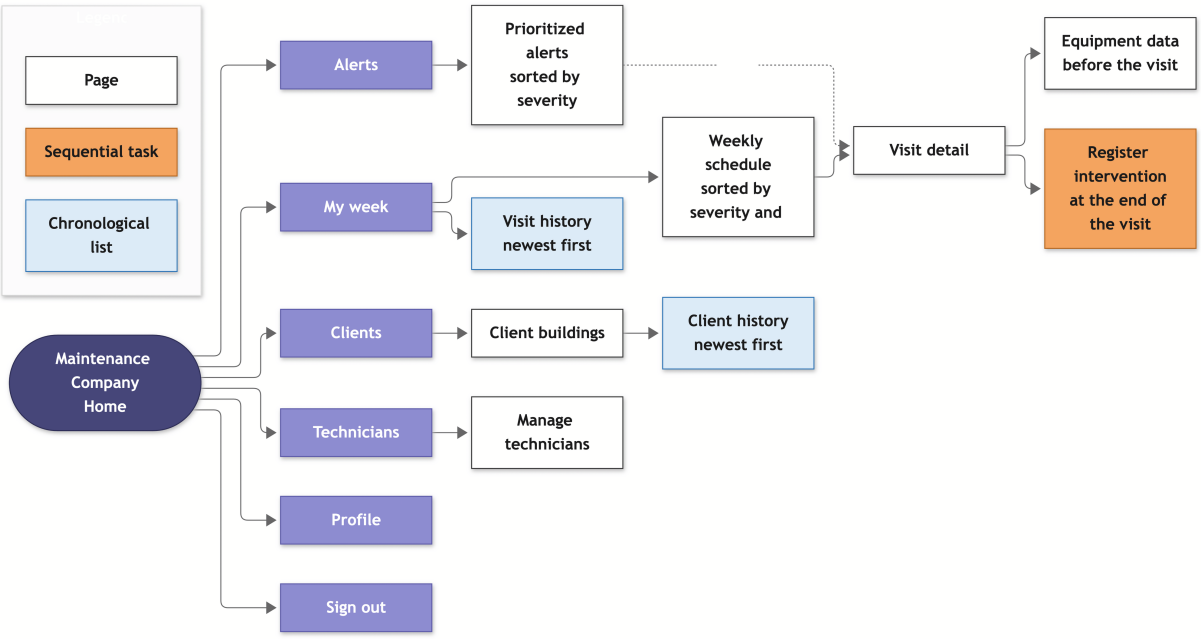


Figura 10: Figura 4.2.1-3 – Arquitectura del diseño objetivo para la empresa de mantenimiento

Los tres diagramas siguen la misma lógica: una pantalla de inicio de la que salen las secciones principales del rol. El del administrador es el más grande, porque concentra edificios, alertas, incidentes, visitas y reportes. El del residente es el más simple.

4.2.2. Labeling Systems

Usamos etiquetas cortas y del lenguaje del usuario, no técnicas. La aplicación está en inglés por defecto y en español con el selector de idioma. Las etiquetas salen de los archivos `public/i18n/en.json` y `es.json`, así que en el informe y en la aplicación son las mismas.

Menú principal (Sprint 2):

Opción	Inglés	Español	Ruta
Inicio	Home	Inicio	`/home`
Edificios	Buildings	Edificios	`/asset-monitoring/buildings`
Equipos	Equipment	Equipos	`/asset-monitoring/equipment`
Alertas	Alerts	Alertas	`/asset-monitoring/alerts`
Sensores	Sensors	Sensores	`/asset-monitoring/sensors`

Opción	Inglés	Español	Ruta
Lecturas	Readings	Lecturas	`/asset-monitoring/readings`
Incidentes	Incidents	Incidentes	`/incidents`

Estados y niveles que ve el usuario:

Concepto	Valores en inglés	Valores en español
Severidad de alerta	Low, Medium, High, Critical	Baja, Media, Alta, Crítica
Estado de alerta	Active, In progress, Resolved	Activa, En gestión, Resuelta
Estado de equipo	Operational, Requires follow-up, Out of service, Decommissioned	Operativo, Requiere seguimiento, Fuera de servicio, Dado de baja
Estado de sensor	Unassigned, Assigned, Monitoring, Disconnected, Inactive	Sin asignar, Asignado, Monitoreando, Desconectado, Inactivo
Estado de incidente	Received, In progress, Resolved	Recibido, En gestión, Resuelto

Los botones dicen la acción de negocio, no la acción técnica: “New equipment” lleva al formulario, “Register” crea el equipo y “Save changes” guarda una edición. Para los cambios de estado usamos verbos claros: “In progress”, “Resolve”, “Decommission”, “Assign”, “Start management” y “Rate”.

En el diseño objetivo el menú cambia según el rol. El residente tendrá “Home”, “My requests” y “Profile”, y la empresa de mantenimiento “My week”, “Alerts” y “Clients”.

4.2.3. SEO Tags and Meta Tags

El Landing Page es la página que tiene que aparecer en los buscadores. Sus etiquetas están en inglés, igual que el contenido del sitio:

```
<title>Vigilia — Smart Preventive Maintenance</title>
<meta name="description" content="Vigilia: an IoT preventive maintenance
 ↪  platform for buildings and condominiums. Catch failures in pumps, panels
 ↪  and elevators before they become emergencies.">
```

```
<meta name="keywords" content="preventive maintenance, building maintenance,
 ↪  condominium, IoT sensors, water pump, elevator, electrical panel, Lima,
 ↪  Vigilia">
<meta name="author" content="CodeNova">
```

La Web Application no necesita posicionarse en buscadores, porque se entra desde el Landing Page. Solo lleva el título “Vigilia” y un título por vista, por ejemplo “Vigilia - Home”.

4.2.4. Searching Systems

La aplicación tiene filtros que ayudan a encontrar rápido lo importante.

Sistema	Descripción	Beneficio para el usuario	Estado
Alertas ordenadas por severidad	La tabla de alertas va de Critical a Low y arriba muestra cuántas hay de cada nivel. Un interruptor muestra u oculta las resueltas.	El administrador ve primero lo más grave sin leer cada alerta.	Sprint 2
Filtro de equipos por edificio	Un selector muestra solo los equipos de un edificio.	Quien administra varios edificios no se confunde de edificio.	Sprint 2
Historial de lecturas por equipo y fechas	Se elige un equipo y un rango de fechas.	El administrador revisa el comportamiento de un equipo en un periodo concreto.	Sprint 2
Orden y paginación en tablas	Las tablas se ordenan por columna y se paginan.	Las listas largas siguen siendo manejables.	Sprint 2
Búsqueda de empresas de mantenimiento	El administrador busca una empresa por RUC o nombre para vincularla a su edificio.	Vincula a su proveedor sin coordinar por fuera de la plataforma.	Sprint 3

4.2.5. Navigation Systems

Implementado en el Sprint 2. La navegación es una barra superior con las siete opciones del menú y el selector de idioma. La opción activa queda subrayada. Home lleva directo al panel de alertas, que es la vista principal del administrador. Desde las listas se pasa a los formularios (“New”, “Edit”, “Assign”, “Rate”) y cada formulario tiene “Cancel” para volver sin guardar. Las rutas son semánticas: `/asset-monitoring/equipment/3/edit` dice qué recurso se edita. Si el usuario escribe una ruta que no existe, ve una página 404 con un enlace a Home.

Diseño objetivo. El panel del administrador sumará el estado general del edificio, el ahorro acumulado y las últimas intervenciones. El residente navegará entre “Home” (semáforo de los equipos comunes), “My requests” y “Profile”. La empresa de mantenimiento navegará entre “My week” (agenda priorizada), “Alerts” y “Clients”. Cuando exista IAM, después de iniciar sesión cada usuario llegará a la pantalla de su rol.

4.3. Landing Page UI Design

El Landing Page tiene que explicar en pocos segundos qué hace Vigilia: anticipar fallas antes de que se conviertan en emergencias. Está pensado para administradores y empresas de mantenimiento que todavía no conocen la plataforma. El diseño aplica las Style Guidelines de la sección 4.1 y la arquitectura de la sección 4.2.

La página sigue un orden de lectura: propuesta de valor, funciones, cómo funciona, planes, el equipo y contacto. En TB1 cambiamos el llamado a la acción principal. En el AV1 era “Solicitar demo”; ahora es “Go to app” y lleva a la Web Application, como pide el statement. El sitio está en inglés por defecto.

4.3.1. Landing Page Wireframe

Desktop Web Browser. El primer bloque tiene el titular, una frase corta, los dos botones de acción y tres datos rápidos (edificios activos, alertas por mes y satisfacción). A la derecha va una vista previa del estado de los equipos. Debajo vienen los tres pasos del servicio: instalar sensores, recibir alertas priorizadas y ahorrar en mantenimiento.

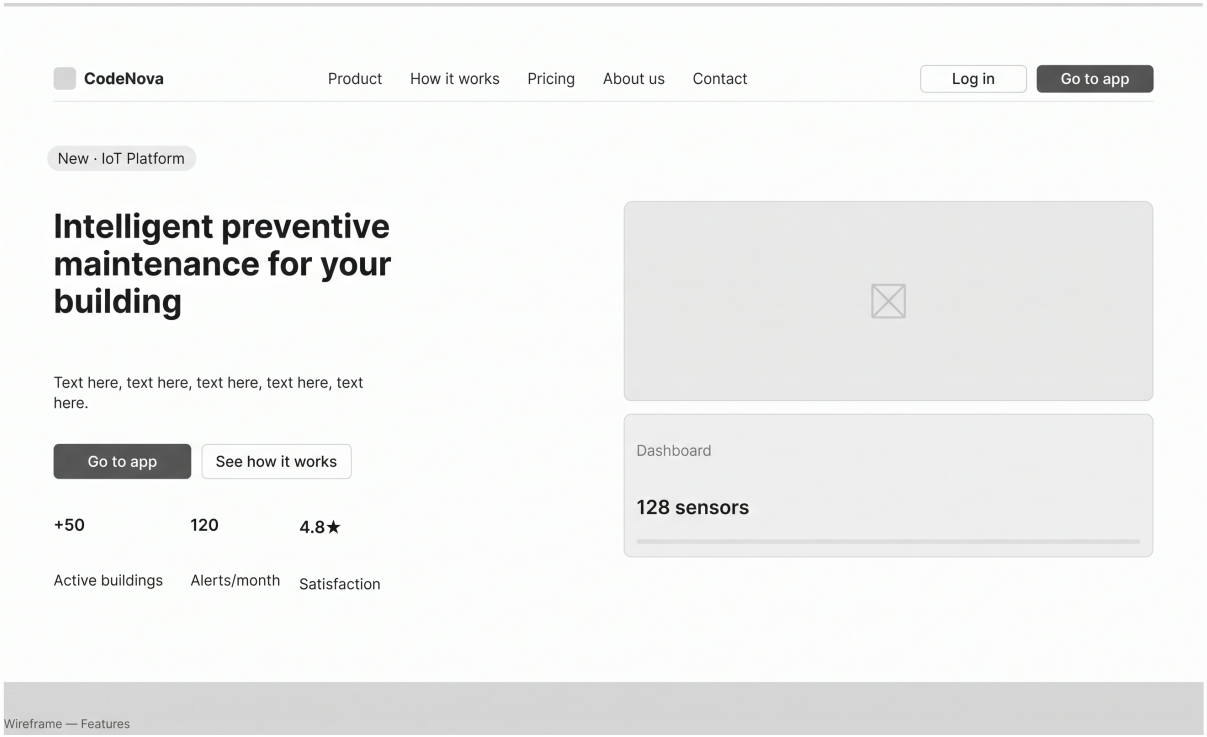


Figura 11: Figura 4.3.1-1 – Wireframe del Landing Page para Desktop Web Browser

Decisiones de diseño:

- **Contraste.** Fondo blanco para que resalten los botones y los textos.
- **Repetición.** Botones con forma de píldora y tarjetas que se repiten en todas las secciones.
- **Proximidad y alineación.** Dos columnas: texto a la izquierda e imagen a la derecha. Lo relacionado va junto.
- **Minimalismo.** Mucho espacio en blanco. Parte del público no tiene formación técnica y una página cargada lo aleja.

Mobile Web Browser. Las secciones se apilan en una sola columna y se recorren con scroll. Los enlaces del menú pasan a un menú desplegable en la esquina superior derecha. Los botones principales van centrados y con un tamaño cómodo para el pulgar.

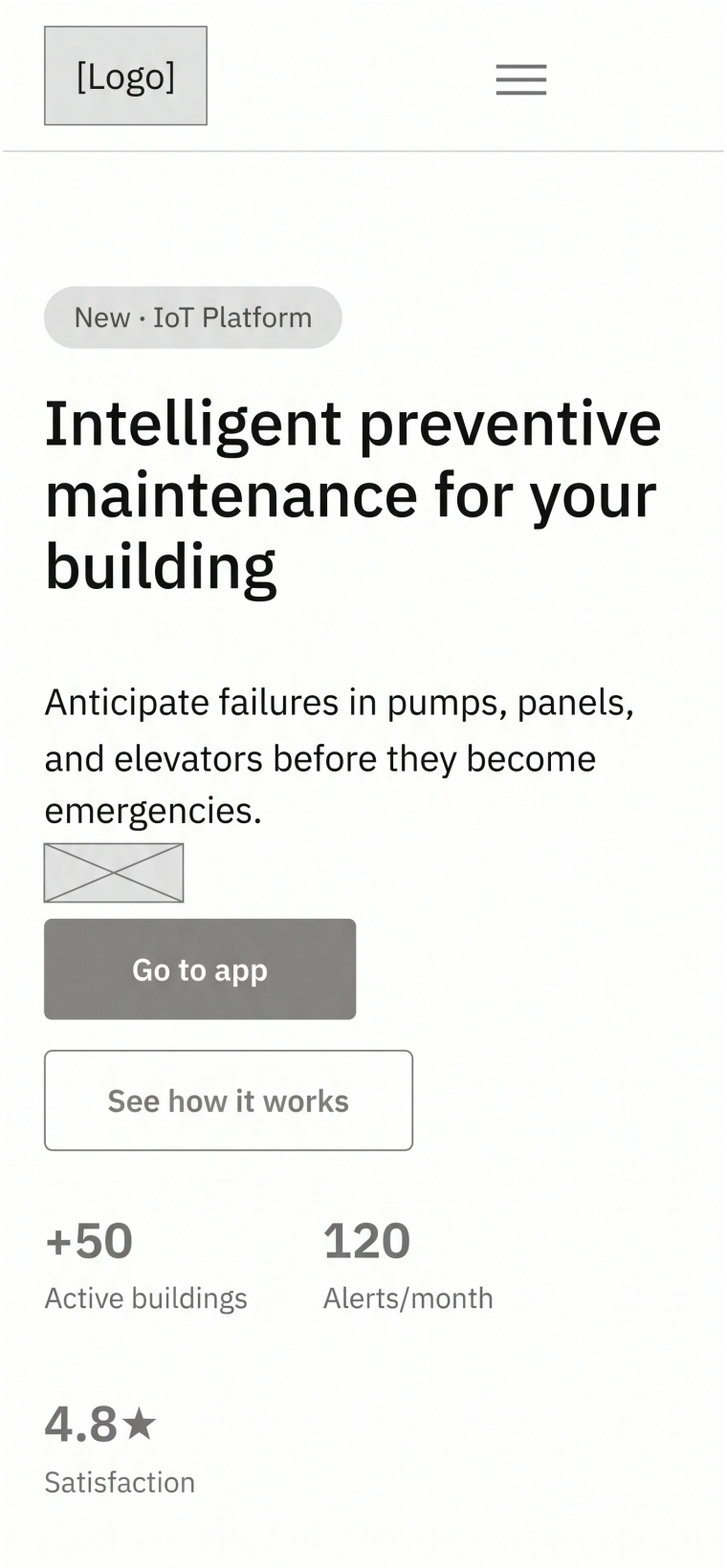


Figura 12: Figura 4.3.1-2 – Wireframe del Landing Page para Mobile Web Browser

4.3.2. Landing Page Mock-up

Desktop Web Browser. El mock-up aplica la identidad de la sección 4.1. El Azul Primario (#0F5C78) va en la navegación y en los botones principales, y el Verde Prevención (#2E9E6D) en los detalles que transmiten control. La tipografía es IBM Plex Sans, con distintos pesos para separar titular, subtítulos y texto. Los colores de severidad solo aparecen en la vista previa del estado de los equipos, para no gastar su significado en adornos.

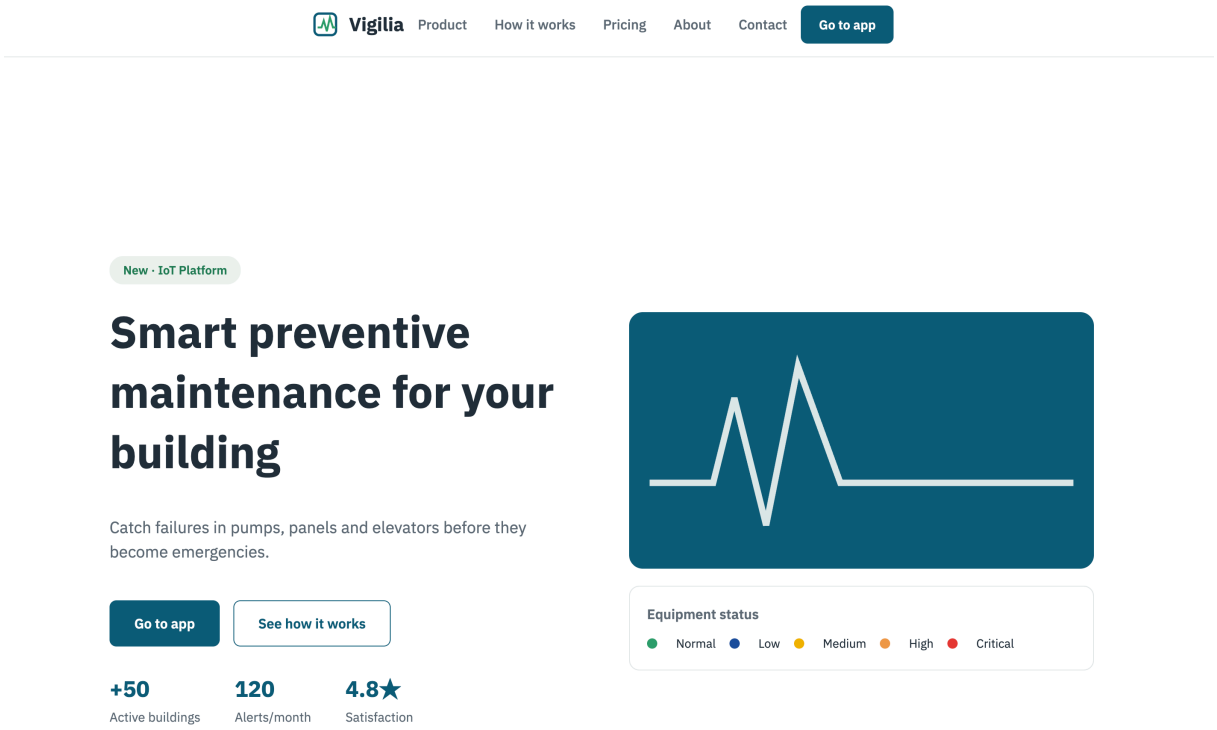


Figura 13: Figura 4.3.2-1 – Mock-up del Landing Page para Desktop Web Browser

Mobile Web Browser. Mantiene los colores y la tipografía, en una sola columna. El menú desplegable libera espacio y el botón principal queda visible apenas carga la página.

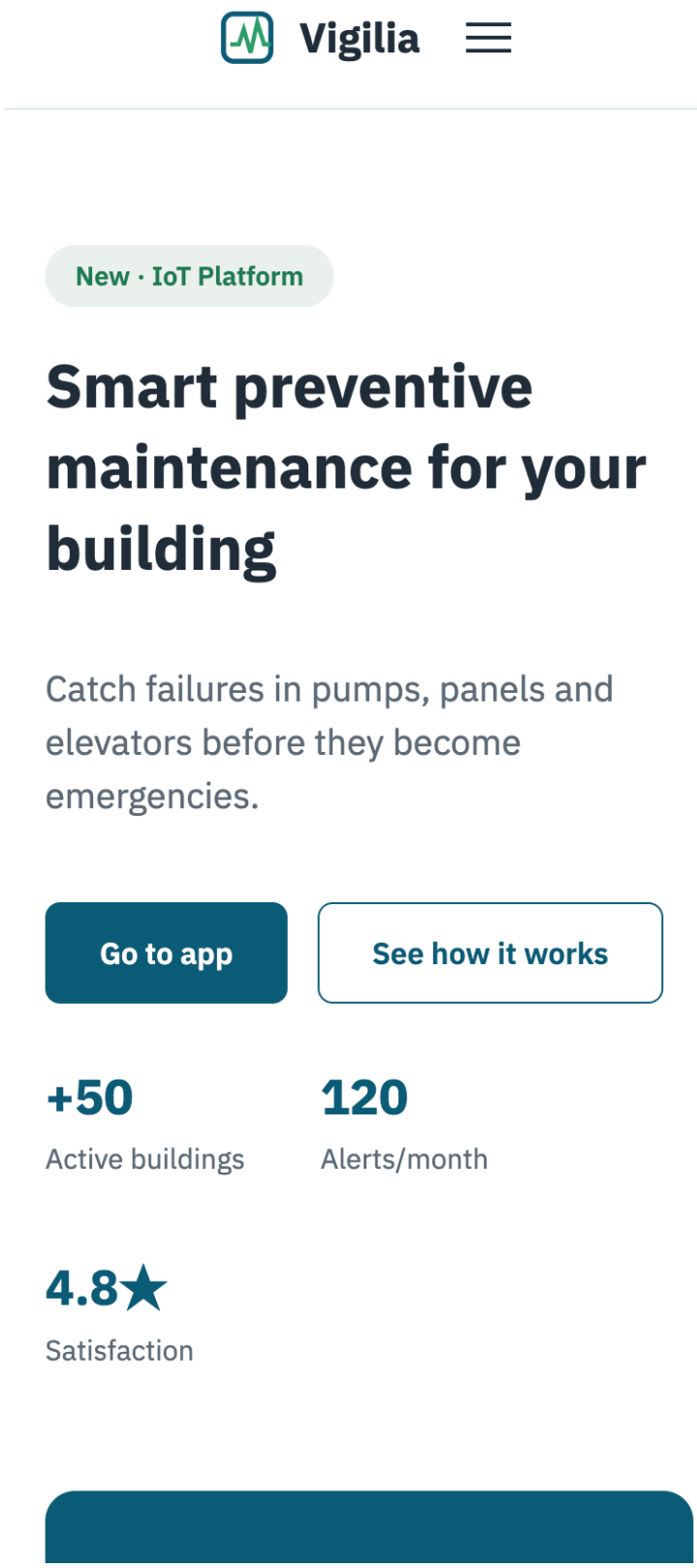


Figura 14: Figura 4.3.2-2 – Mock-up del Landing Page para Mobile Web Browser

En las dos versiones aplicamos varias heurísticas de Nielsen. El menú fijo arriba ayuda a ubicarse (visibilidad del estado). Las etiquetas usan palabras del usuario (relación con el mundo real). Los botones y tarjetas se ven igual en toda la página (consistencia). Solo se muestra lo esencial (diseño minimalista).

4.4. Web Applications UX/UI Design

En esta sección están los wireframes, wireflows, mock-ups y user flows de la Web Application. Los diseños muestran el **diseño objetivo**, con las vistas de los tres roles. En el Sprint 2 implementamos una parte: Asset Monitoring para el administrador e Incidents para el residente (sección 5.2.2). En la implementación usamos una barra de navegación superior en lugar del menú lateral de los diseños, siguiendo el layout del proyecto de referencia del curso.

4.4.1. Web Applications Wireframes

Los wireframes definen la estructura de cada vista sin colores ni imágenes finales. Todas comparten el mismo esqueleto: menú a la izquierda, título de la vista arriba y el contenido en tarjetas o tablas. Los diseños completos están en Figma: https://www.figma.com/design/jNGWsrdmqjlwTasD1MXpMV/CodeNova?node-id=2022-6613

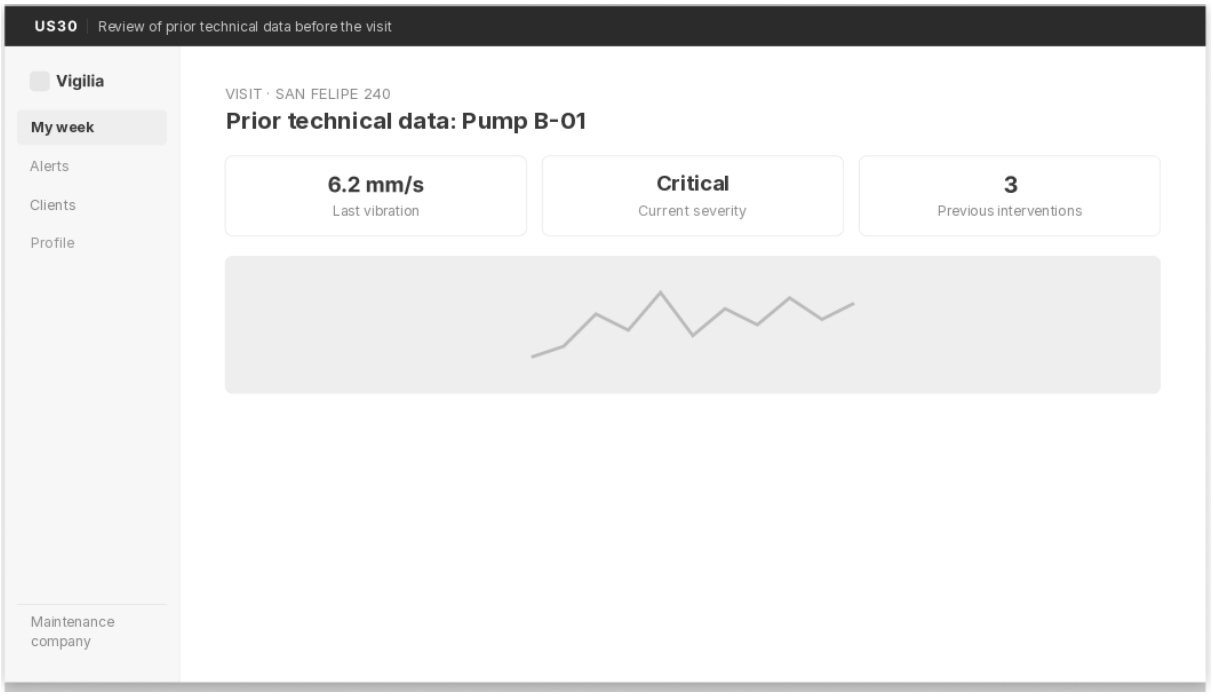


Figura 15: Figura 4.4.1-1 – Wireframe: datos técnicos de un equipo antes de la visita (US30)

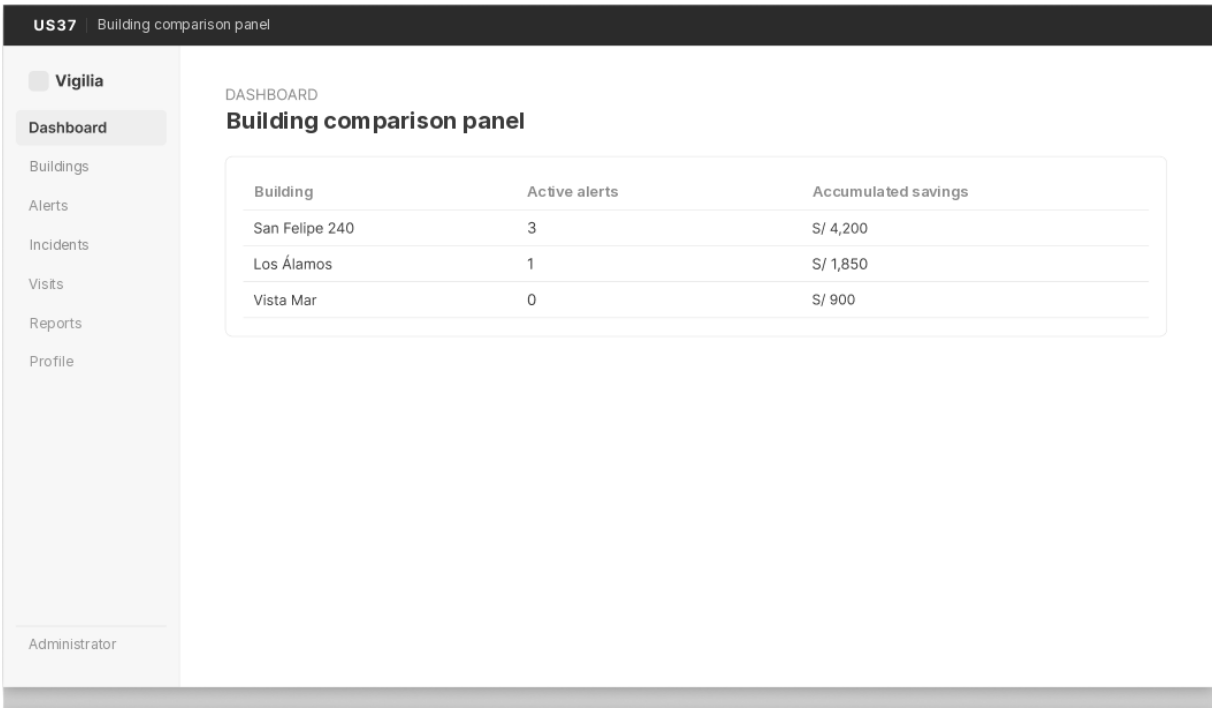


Figura 16: Figura 4.4.1-2 – Wireframe: panel comparativo entre edificios (US37)

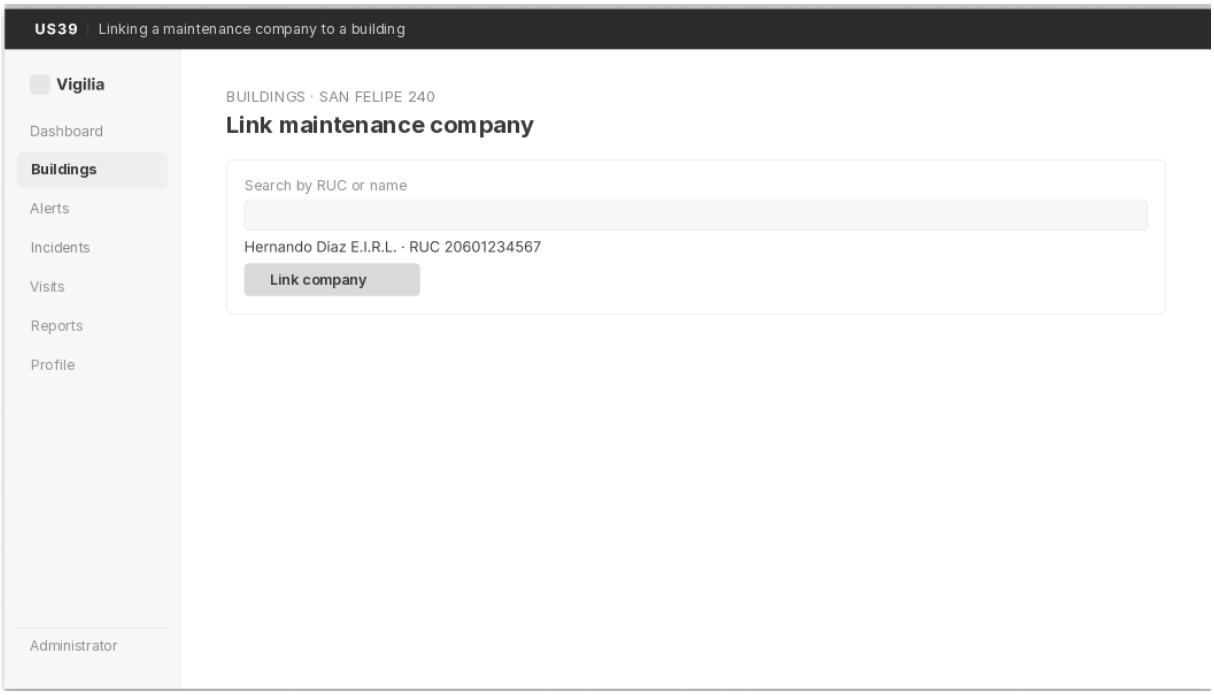


Figura 17: Figura 4.4.1-3 – Wireframe: vincular una empresa de mantenimiento a un edificio (US39)

Principios de diseño:

- **Contraste.** Los botones principales usan el Azul Primario y los chips de severidad sus colores semánticos, sobre fondo blanco o gris claro.
- **Alineación.** Textos y títulos alineados a la izquierda, como en “Active alerts” o en el detalle de un equipo.
- **Repetición.** Las tarjetas se repiten para alertas, incidentes y visitas, y la navegación es la misma en todas las vistas.
- **Proximidad.** En el detalle de un equipo, los datos del equipo van juntos arriba y el historial de lecturas va en su propio bloque.

Heurísticas de Nielsen aplicadas:

- **Visibilidad del estado del sistema.** Cada alerta muestra su severidad y cada incidente su estado (Received, In progress, Resolved).
- **Relación con el mundo real.** El residente ve palabras simples como “Report incident”, no términos como “lectura fuera de rango”.
- **Control y libertad.** Cada formulario tiene “Cancel” para volver sin guardar.
- **Consistencia.** Los botones principales tienen el mismo color y forma en todas las vistas.

- **Prevención de errores.** Dar de baja un equipo pide confirmación. Un sensor que ya tiene equipo no se puede asignar a otro.
- **Reconocer antes que recordar.** El menú siempre está visible, con etiquetas cortas.
- **Diseño minimalista.** Cada vista muestra solo lo necesario para su tarea.
- **Ayudar a recuperarse de errores.** Si no hay lecturas en un rango de fechas, la vista lo dice con un mensaje claro en lugar de quedar vacía.

4.4.2. Web Applications Wireflow Diagrams

Los wireflows combinan los wireframes con la secuencia de pasos que sigue el usuario para cumplir un objetivo.

Wireflow 1: Reporte de incidente y seguimiento

- **User goal:** Como residente, después de notar una falla en un área común, quiero reportarla en segundos y ver que alguien la está atendiendo, sin depender del grupo de WhatsApp del edificio.
- **User persona:** Diego Salinas (residente).
- **User stories:** US23, US24 y US25.

1. El residente entra a “Incidents” y presiona “Report incident”.
2. Elige el edificio, la categoría (agua, electricidad, ascensor, aire acondicionado u otro) y escribe una descripción corta. Si sabe qué equipo falló, lo elige.
3. En el diseño objetivo puede adjuntar una foto (US24, Sprint 3).
4. Al enviar, el incidente aparece en la lista con estado “Received”.
5. El residente vuelve a la lista cuando quiere y ve el estado actualizado: Received → In progress → Resolved.

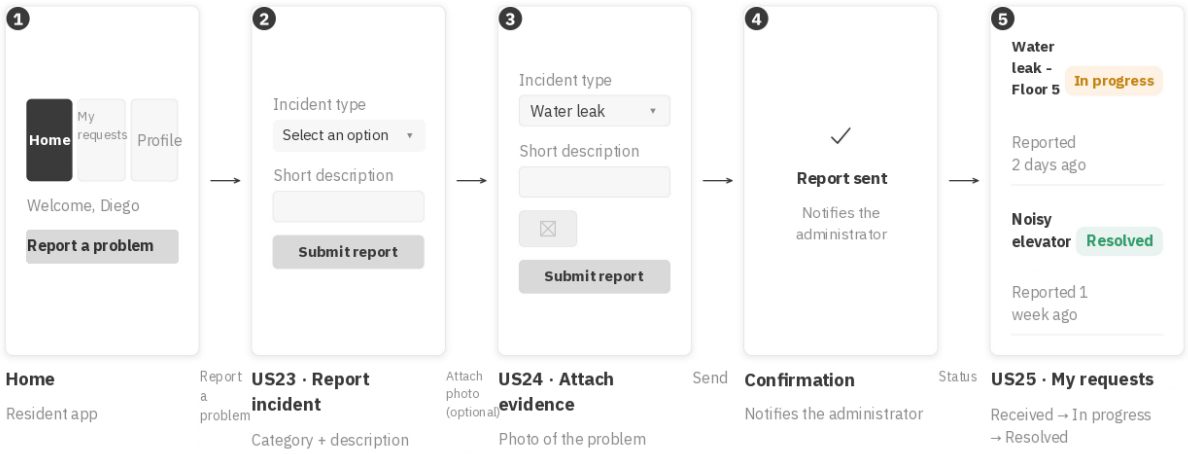


Figura 18: Figura 4.4.2-1 – Wireflow: reporte de incidente y seguimiento

Wireflow 2: Registro de equipo y asignación de sensor

- **User goal:** Como administrador, quiero registrar un equipo crítico de mi edificio y asignarle un sensor, para que la plataforma empiece a monitorearlo.
- **User persona:** Carlos Injante (administrador).
- **User stories:** US08 y US10.

1. El administrador entra a “Equipment” y presiona “New equipment”.
2. Elige el edificio y completa código, nombre, tipo, ubicación, fecha de instalación y estado.
3. Al registrarlo, el equipo aparece en la lista de su edificio.
4. En “Sensors” elige un sensor sin asignar y presiona “Assign”.
5. Elige el equipo y confirma. Desde ese momento las lecturas de ese sensor aparecen en el historial del equipo.

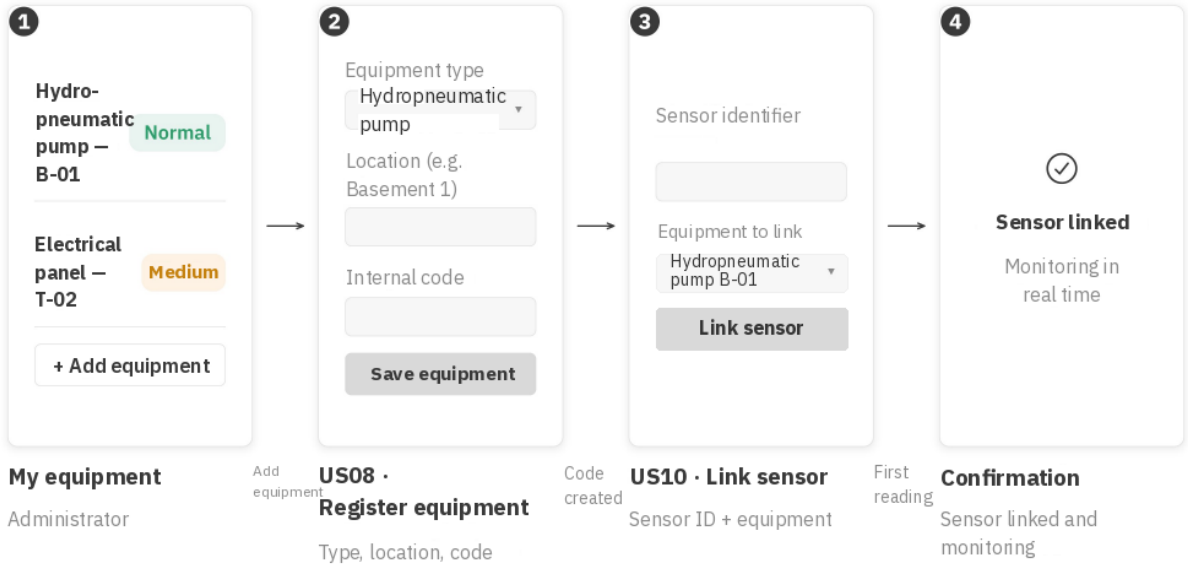


Figura 19: Figura 4.4.2-2 – Wireflow: registro de equipo y asignación de sensor

Wireflow 3: Programación y confirmación de una visita preventiva

- **User goal:** Como administrador, al recibir una alerta, quiero programar una visita con la empresa de mantenimiento y que esta la confirme, para intervenir antes de que la falla se agrave.
- **User personas:** Carlos Injante (administrador) y Renzo Farfán (empresa de mantenimiento).
- **User stories:** US28 y US29.

1. El administrador abre una alerta activa y presiona “Schedule visit”.
2. Elige la empresa de mantenimiento del edificio y una fecha.
3. La visita se crea con estado “Proposed” y la empresa recibe un aviso.
4. La empresa revisa la propuesta y la confirma o propone otra fecha.
5. Cuando se confirma, la visita pasa a “Confirmed” y ambos ven la fecha final.

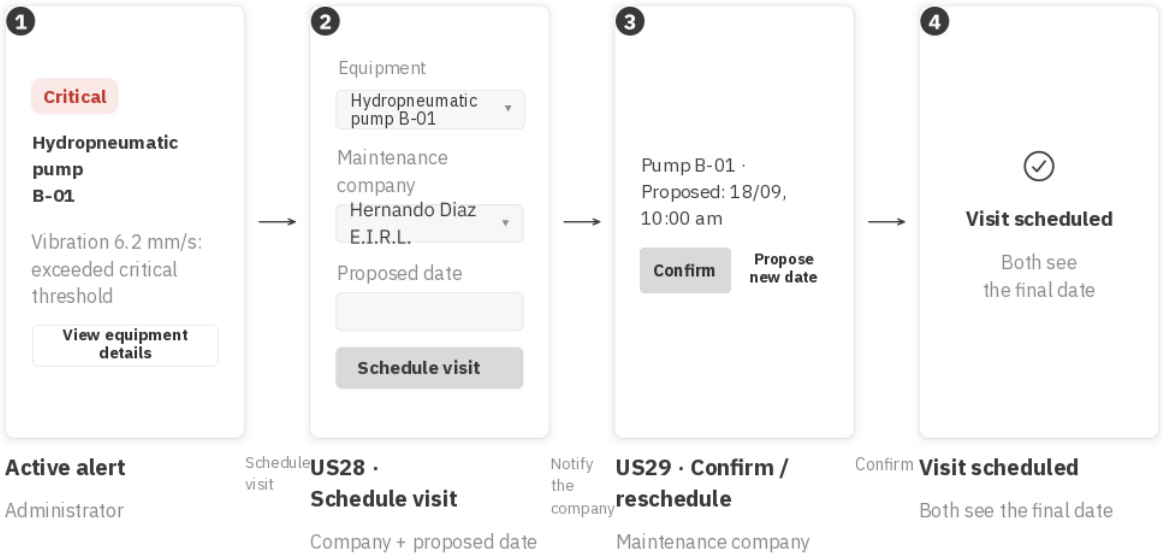


Figura 20: Figura 4.4.2-3 – Wireflow: programación y confirmación de una visita

Wireflow 4: Priorización semanal y registro de intervención

- **User goal:** Como técnico de mantenimiento, quiero ver mi semana priorizada por severidad y, al terminar cada visita, registrar lo que hice.
- **User persona:** Renzo Farfán (empresa de mantenimiento).
- **User stories:** US32, US30 y US31.

1. El técnico entra a “My week” y ve sus visitas ordenadas por severidad y zona.
2. Antes de salir, abre el detalle del equipo y revisa las últimas lecturas y las intervenciones anteriores.
3. Hace la visita con ese contexto.
4. Al terminar, registra la intervención: trabajo realizado, materiales, costo preventivo y estado final del equipo.
5. La visita pasa a “Completed” y el equipo queda con el estado que registró el técnico. Si quedó operativo, la plataforma calcula el ahorro estimado.

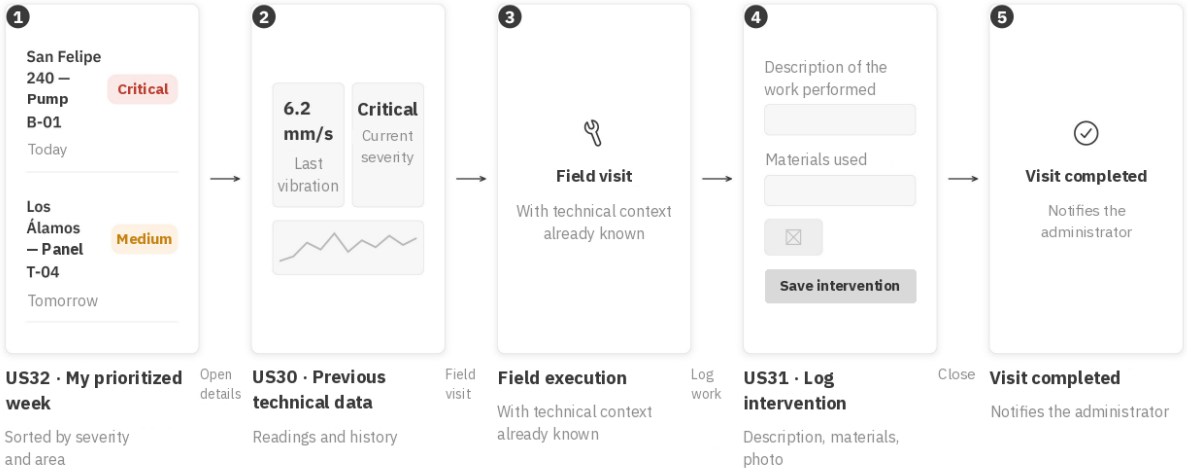


Figura 21: Figura 4.4.2-4 – Wireflow: priorización semanal y registro de intervención

Wireflow 5: Registro e inicio de sesión por rol

Este flujo es de IAM y se implementa en el último sprint (sección 3.3). Lo dejamos diseñado para que el resto de vistas sepa a dónde llega cada rol.

- **User goal:** Como usuario nuevo, quiero registrarme según mi rol y luego iniciar sesión para usar las herramientas que me corresponden.
- **User personas:** los tres.
- **User stories:** US01, US02, US04 y US05.

1. El usuario elige su rol y completa el formulario de registro.
2. La plataforma valida los datos propios del rol, como el código del edificio o el RUC.
3. Al terminar el registro, la plataforma lo lleva a la pantalla de inicio de sesión. Registrarse no inicia sesión.
4. El usuario ingresa su correo y contraseña.
5. La plataforma lo lleva a la vista de su rol: panel de alertas, inicio del residente o “My week”.

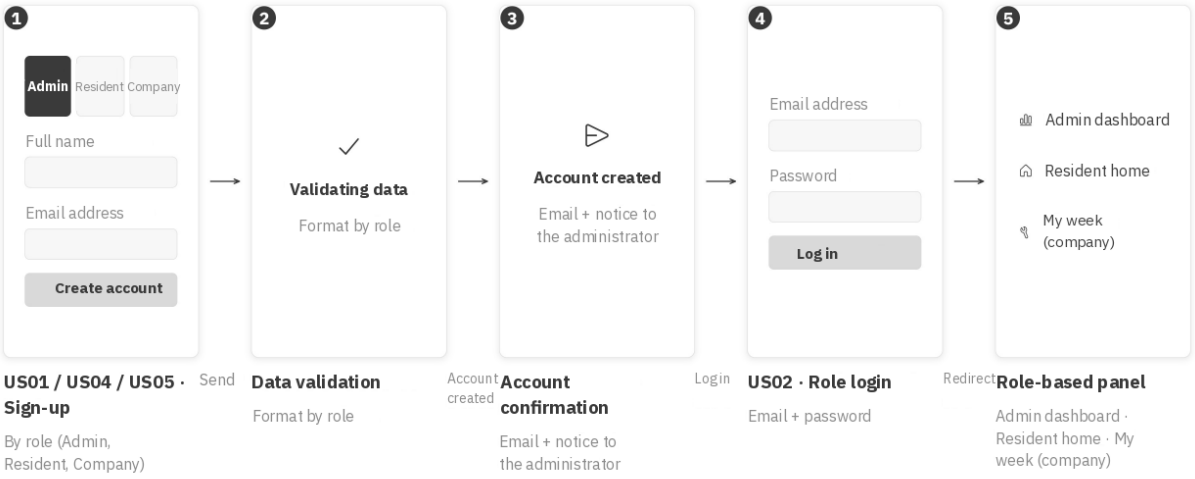


Figura 22: Figura 4.4.2-5 – Wireflow: registro e inicio de sesión por rol

4.4.3. Web Applications Mock-ups

Los mock-ups son la versión de alta fidelidad de los wireframes. Aplican la paleta de la sección 4.1: Azul Primario para navegación y acciones principales, Verde Prevención para estados positivos y la escala de severidad solo en alertas. Los componentes se repiten en todas las vistas: tarjetas, chips de severidad y botones redondeados.

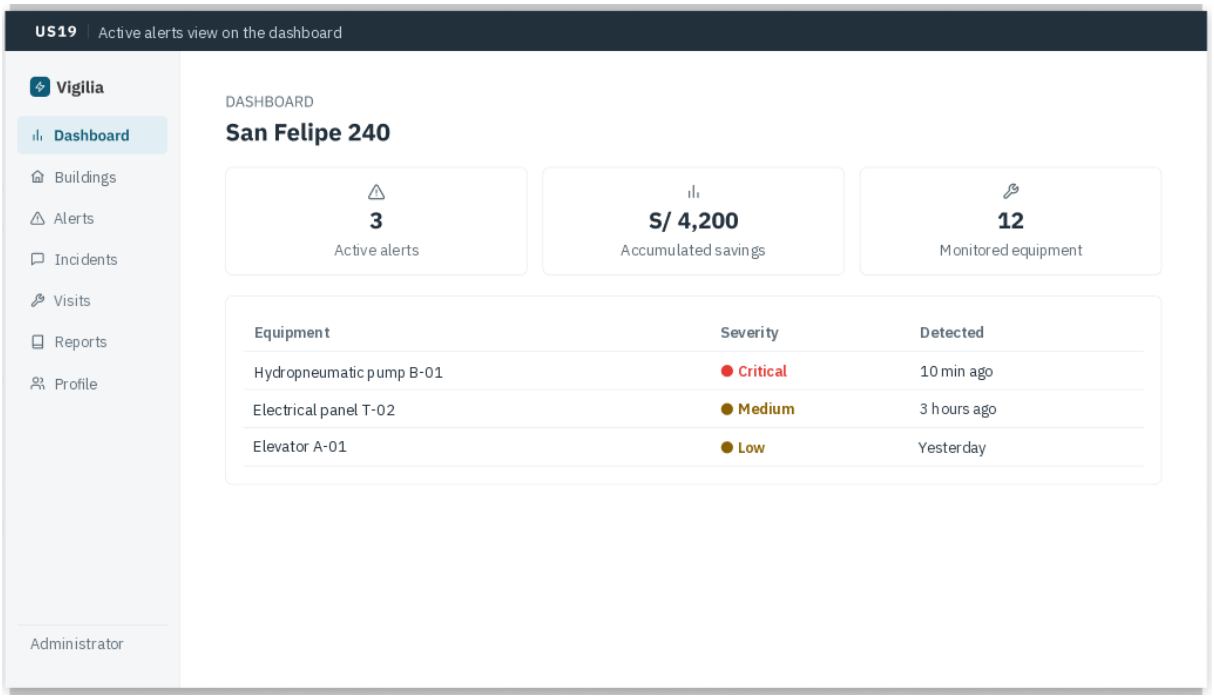


Figura 23: Figura 4.4.3-1 – Mock-up: panel de alertas del administrador (US19)

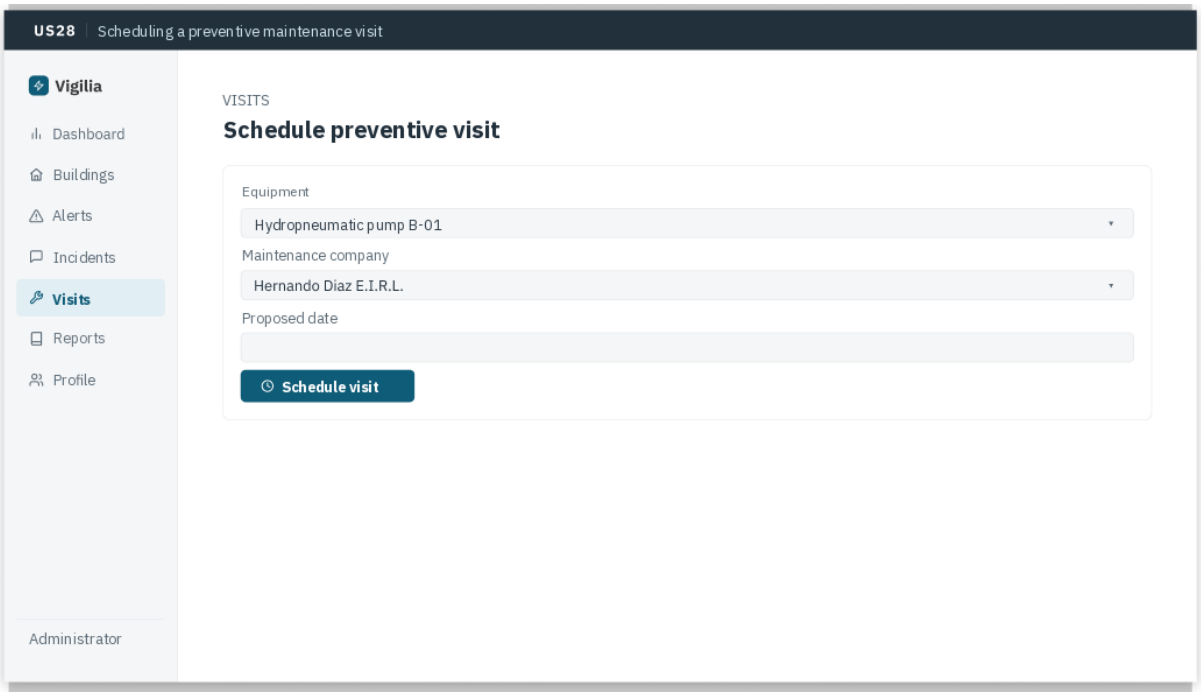


Figura 24: Figura 4.4.3-2 – Mock-up: programar una visita preventiva (US28)

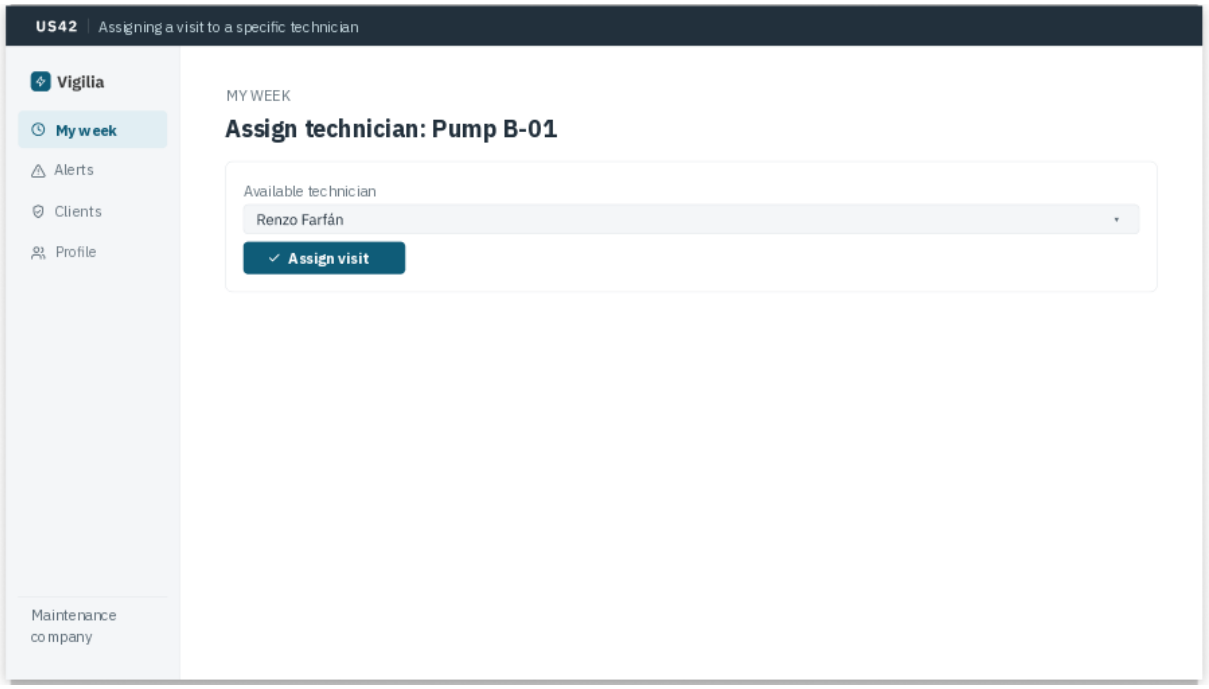


Figura 25: Figura 4.4.3-3 – Mock-up: asignar una visita a un técnico (US42)

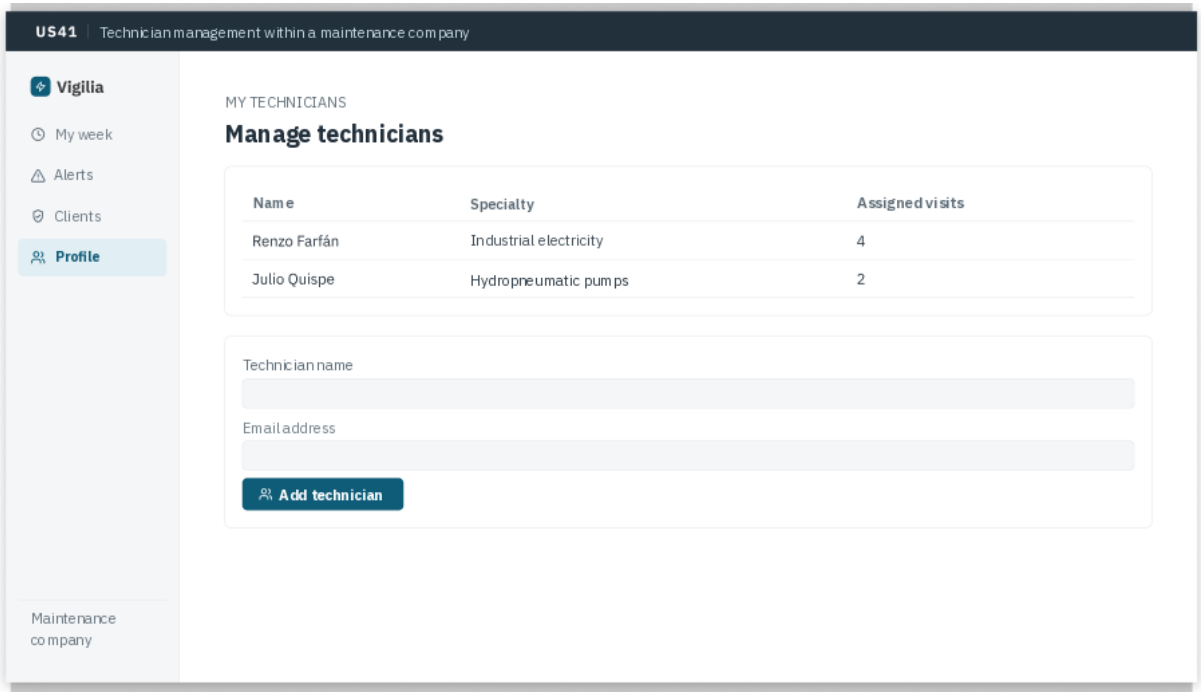


Figura 26: Figura 4.4.3-4 – Mock-up: gestión de técnicos de la empresa (US41)

Los mock-ups cubren las vistas que necesitan los flujos de la sección 4.4.2:

- **Residente:** lista de incidentes → reporte → seguimiento.
- **Administrador:** panel de alertas → detalle de la alerta → programar visita → ahorro acumulado.
- **Empresa de mantenimiento:** “My week” → datos técnicos del equipo → registro de intervención.

4.4.4. Web Applications User Flow Diagrams

Hicimos un user flow por cada user goal. Cada uno tiene su camino feliz (happy path) y un camino alternativo (unhappy path), dibujados sobre los mock-ups.

User Flow 1: Reportar un incidente

- **User goal:** Como residente, quiero reportar una falla que noté en un área común y saber que quedó registrada, sin llamar ni escribir al grupo del edificio.
- **User persona:** Diego Salinas (residente).
- **User stories:** US23, US24 y US25.

Happy path:

1. El residente presiona “Report incident”.
2. Elige la categoría y describe lo que pasó.
3. Si quiere, adjunta una foto.
4. Envía el reporte. La plataforma lo registra al momento.
5. Vuelve a la lista y ve su incidente con estado “Received”.

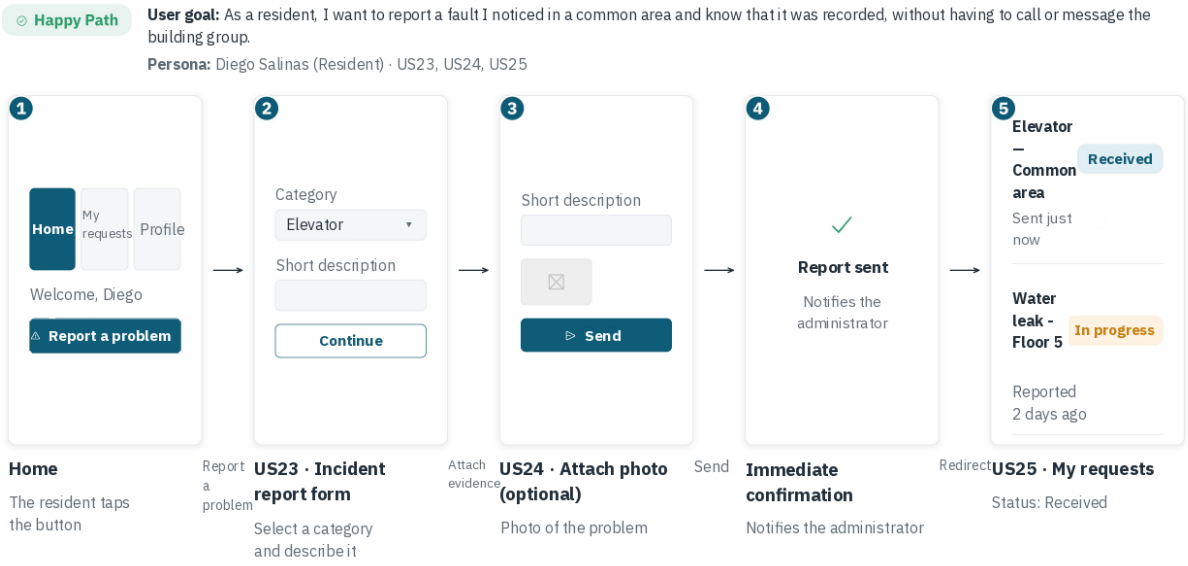


Figura 27: Figura 4.4.4-1 – User flow 1, happy path

Unhappy path: la foto pesa más de 5 MB.

1. El residente elige una foto de su galería.
2. La plataforma detecta que pasa el límite.
3. Muestra “The file exceeds the maximum size (5 MB)” sin borrar el resto del formulario.
4. El residente elige otra foto o sigue sin foto.
5. El reporte se envía igual, con o sin foto.

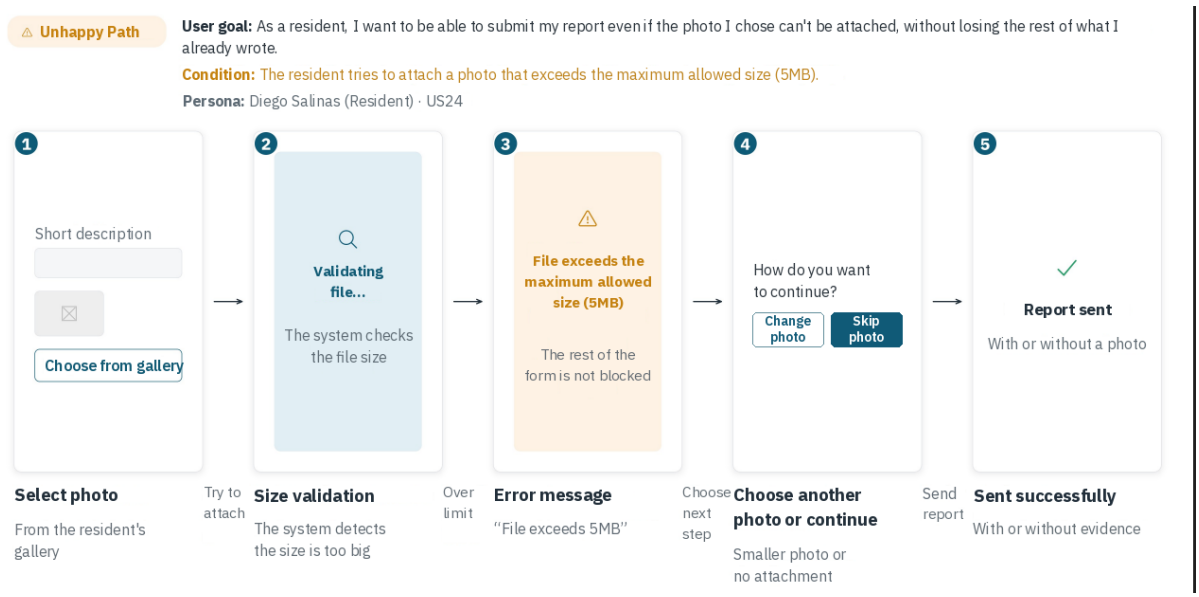


Figura 28: Figura 4.4.4-2 – User flow 1, unhappy path

User Flow 2: Recibir y atender una alerta priorizada

- **User goal:** Como empresa de mantenimiento, quiero recibir una alerta priorizada, revisar el contexto técnico y confirmar la visita, sin llamar al administrador para pedir más datos.
- **User persona:** Renzo Farfán (empresa de mantenimiento).
- **User stories:** US21, US30 y US29.

Happy path:

1. Recibe un aviso dentro de la aplicación por una alerta crítica en un edificio que atiende.
2. Abre el detalle de la alerta: severidad, equipo y últimas lecturas.
3. Revisa el historial del equipo para tener contexto.
4. Confirma la visita en la fecha que propuso el administrador.
5. La visita queda “Confirmed” y aparece en “My week”.

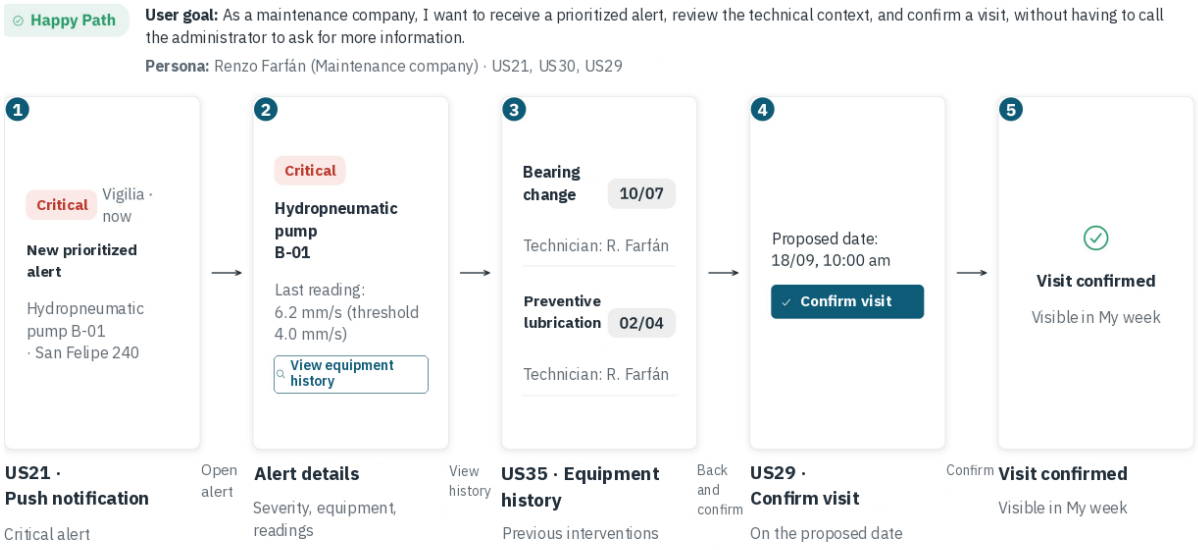


Figura 29: Figura 4.4.4-3 – User flow 2, happy path

Unhappy path: la empresa no puede ir en la fecha propuesta.

1. En el detalle de la alerta revisa la fecha propuesta.
2. Como no tiene disponibilidad, presiona “Propose another date”.
3. Elige una fecha nueva y la envía.
4. El administrador recibe el aviso y aprueba la nueva fecha.
5. La visita queda “Confirmed” con la fecha nueva.

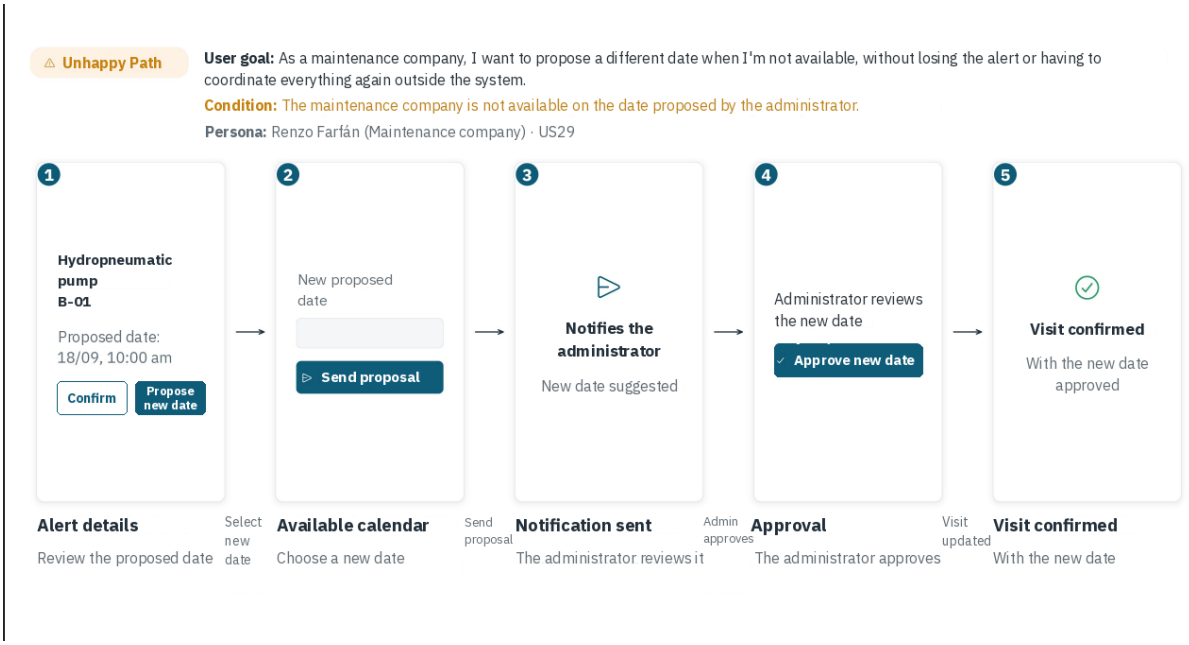


Figura 30: Figura 4.4.4-4 – User flow 2, unhappy path

User Flow 3: Programar una visita preventiva desde una alerta

- **User goal:** Como administrador, al ver una alerta de severidad alta, quiero programar una visita preventiva en pocos pasos, para intervenir antes de que la falla se agrave.
- **User persona:** Carlos Injante (administrador).
- **User stories:** US19 y US28.

Happy path:

1. En el panel de alertas ve las alertas activas ordenadas por severidad.
2. Abre la alerta más grave.
3. Presiona “Schedule visit”.
4. Elige la empresa de mantenimiento y una fecha.
5. Confirma. La visita queda “Proposed” y la empresa recibe un aviso.

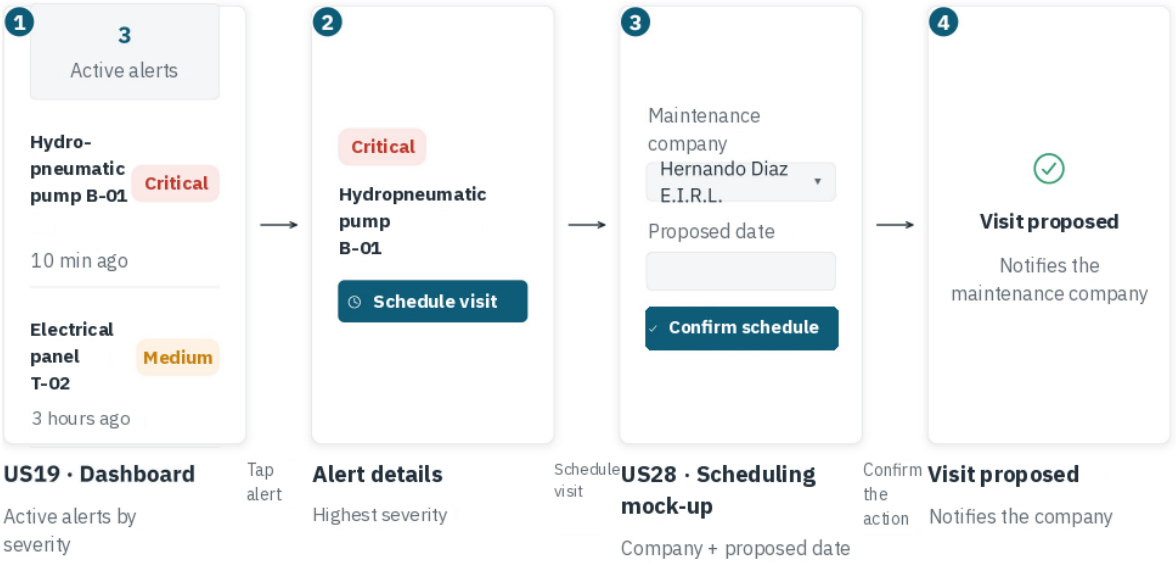


Figura 31: Figura 4.4.4-5 – User flow 3, happy path

Unhappy path: el equipo ya tiene una visita programada.

1. Desde el detalle de la alerta presiona “Schedule visit”.
2. La plataforma detecta que el equipo ya tiene una visita activa.
3. Muestra “This equipment already has a visit scheduled for [date]” y ofrece ver esa visita.
4. El administrador revisa la visita existente.
5. No se crea una visita duplicada.

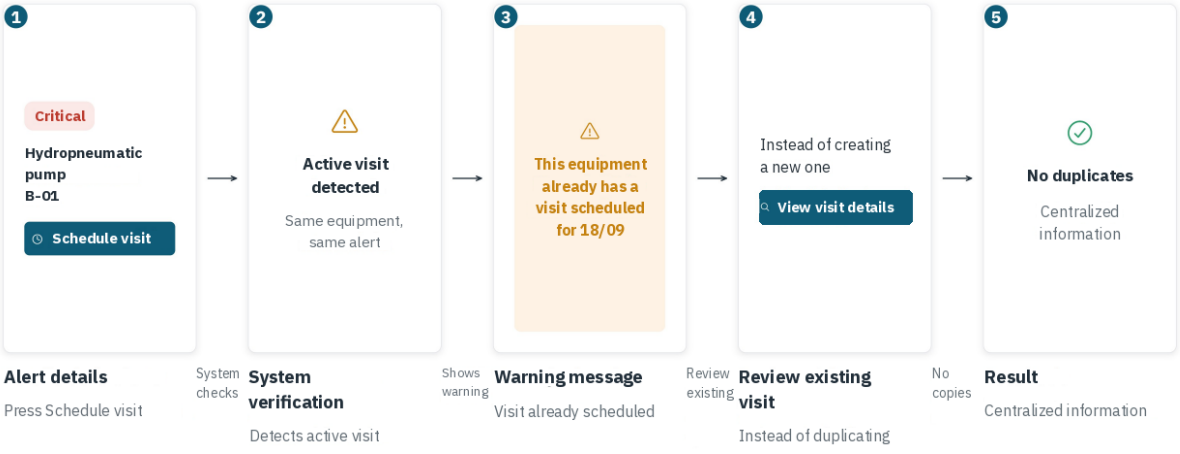


Figura 32: Figura 4.4.4-6 – User flow 3, unhappy path

4.5. Web Applications Prototyping

Armamos un prototipo navegable en Figma a partir de los mock-ups de la sección 4.4.3. Sirve para recorrer los flujos principales antes de programarlos y para mostrarlos en la exposición.

- **Prototipo en Figma:** https://www.figma.com/design/jNGWsrdmqjlwTasD1MXpMV/CodeNova?node-id=2022-6613
- **Video del prototipo:** https://acortar.link/koLkSz
- **Duración:** 02:45

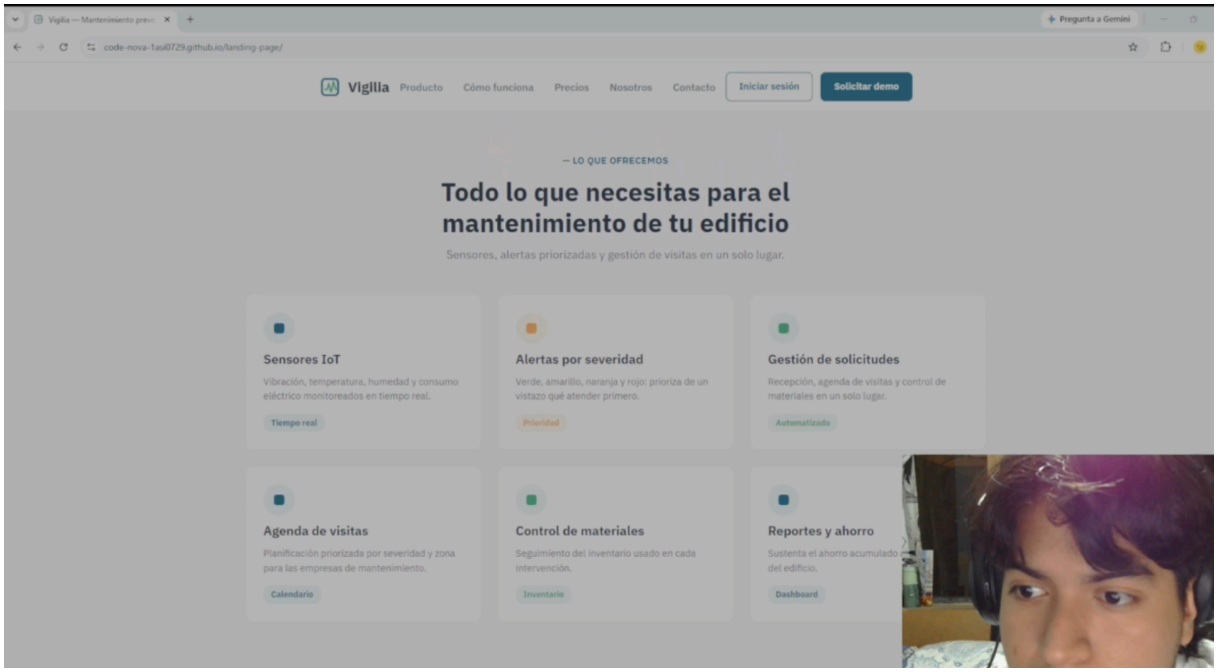


Figura 33: Figura 4.5 - Captura de navegación de prototipo

4.6. Domain-Driven Software Architecture

Esta arquitectura parte del Big Picture Event Storming de la sección 2.4. El dominio de Vigilia queda en cinco bounded contexts: tres core, uno de soporte y uno genérico.

#	Bounded Context	Tipo	Responsabilidad
1	Asset Monitoring	Core	Edificios, equipos, sensores, lecturas y alertas
2	Incidents	Core	Reportes de residentes con evidencia fotográfica
3	Maintenance	Core	Visitas, técnicos, intervenciones y ahorro estimado
4	Notifications	Support	Avisos IN_APP y EMAIL con preferencias por usuario
5	IAM (Identity and Access Management)	Generic	Cuentas, inicio de sesión con token y roles

Los tres contextos core concentran el valor del producto: detectar fallas antes de que ocurran y cerrar el ciclo con una visita. Notifications da soporte a ese ciclo. IAM es commodity: cualquier plataforma lo necesita y no diferencia al producto. Por eso se implementa al final, igual que en el proyecto de referencia del curso.

Vigilia es un monolito modular: un solo REST API con un paquete por contexto. El frontend usa la misma división, con una carpeta por contexto más `shared`. El MVP no necesita desplegar contextos por separado y un solo despliegue reduce el trabajo de operación del equipo.

4.6.1. Design-Level Event Storming

Realizamos la sesión en Miro. Para cada evento identificamos el comando que lo provoca, el actor, el agregado que valida la regla y las consultas que el usuario necesita antes de decidir. Las capturas muestran una vista general con los eventos clave, una vista por cada contexto core y una vista que junta Notifications e IAM.

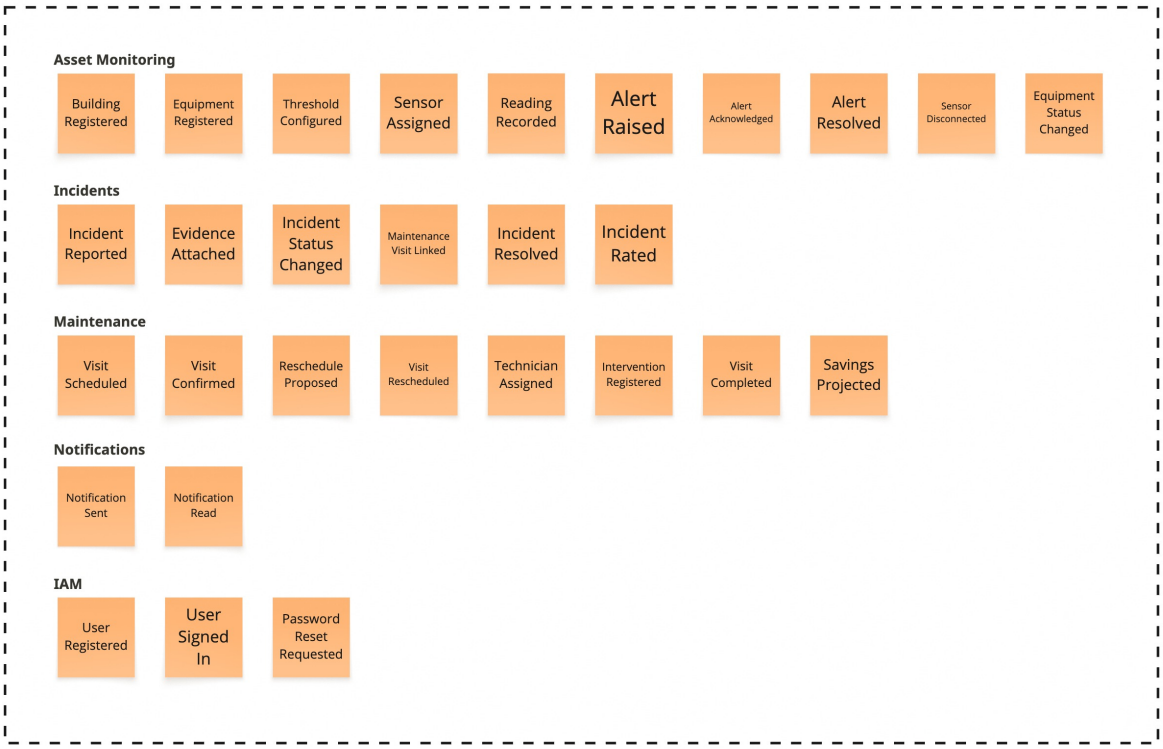


Figura 34: Figura 4.6.1-1 – Design-Level Event Storming: eventos clave de Vigilia

Figura 35: Figura 4.6.1-2 – Design-Level Event Storming: Asset Monitoring

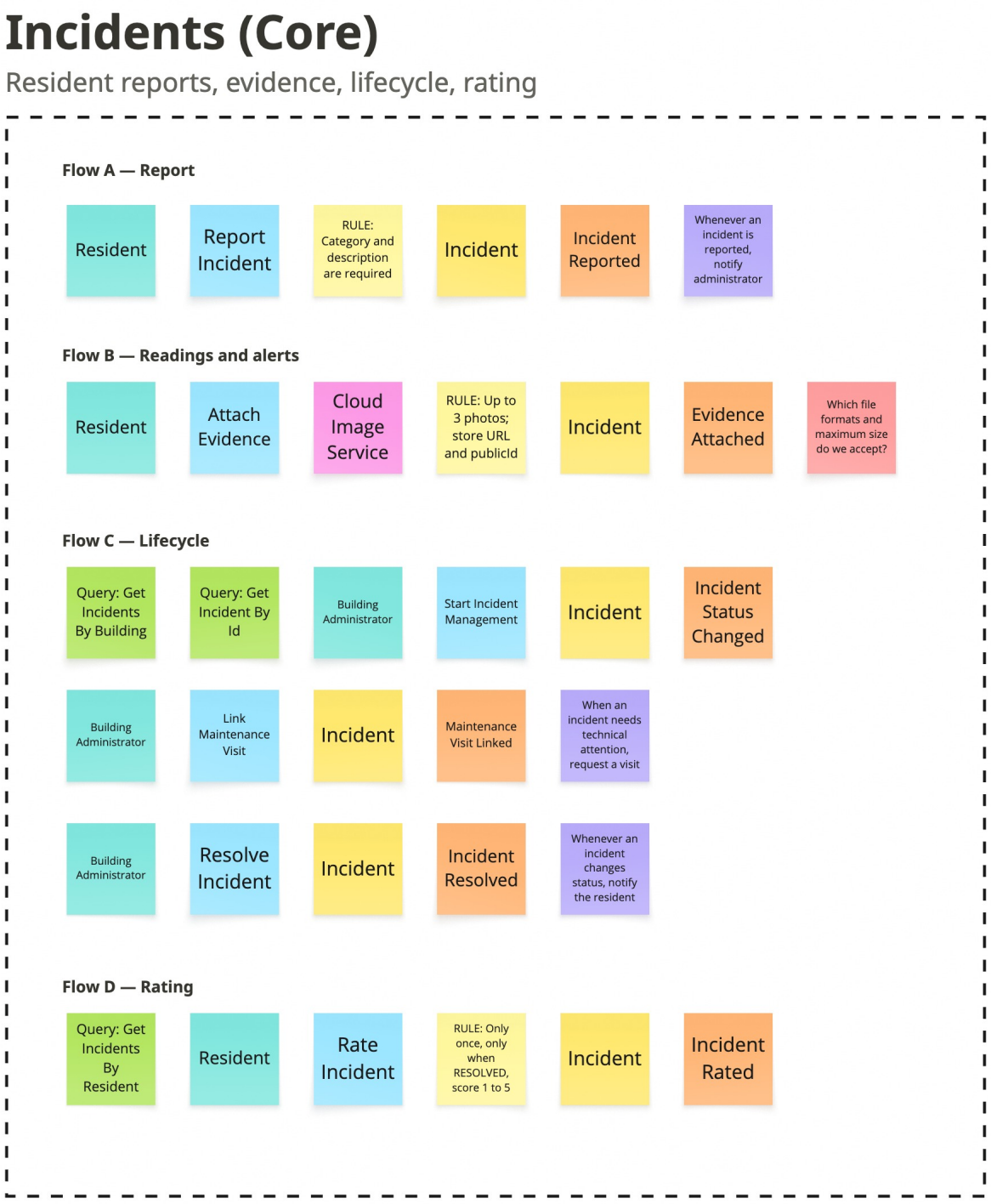


Figura 36: Figura 4.6.1-3 – Design-Level Event Storming: Incidents

Figura 37: Figura 4.6.1-4 – Design-Level Event Storming: Maintenance

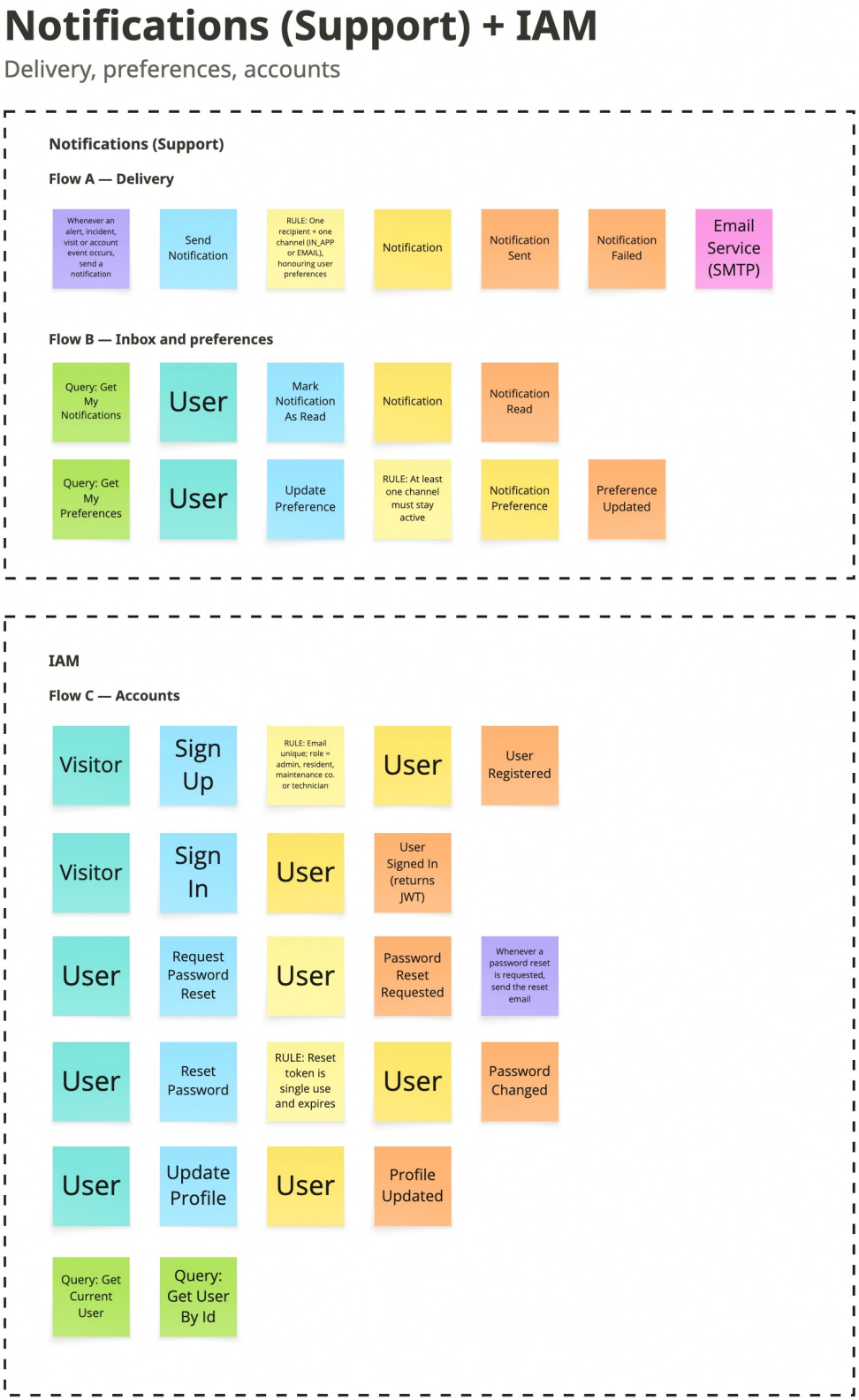


Figura 38: Figura 4.6.1-5 – Design-Level Event Storming: Notifications e IAM

Asset Monitoring (Core)

Elemento	Detalle
Aggregates	Building, CriticalEquipment, Sensor, Alert
Commands	RegisterBuilding, RegisterEquipment, AssignSensor, ConfigureThreshold, RecordReading, AcknowledgeAlert, ResolveAlert, LinkMaintenanceCompany
Queries	GetBuildingEquipment, GetReadingHistory, GetActiveAlerts
Events	EquipmentRegistered, SensorAssigned, ReadingRecorded, ReadingRejected, AlertRaised, AlertSeverityIncreased, AlertResolved, SensorDisconnected, EquipmentStatusChanged

Incidents (Core)

Elemento	Detalle
Aggregates	Incident
Commands	ReportIncident, AttachEvidence, StartIncidentManagement, LinkMaintenanceVisit, ResolveIncident, RateIncident
Queries	GetIncidentsByResident, GetIncidentsByBuilding, GetIncidentById
Events	IncidentReported, EvidenceAttached, IncidentStatusChanged, IncidentResolved, IncidentRated

Maintenance (Core)

Elemento	Detalle
Aggregates	MaintenanceVisit, MaintenanceCompany, CorrectiveReferenceCost
Commands	ScheduleVisit, ConfirmVisit, ProposeReschedule, ApproveReschedule, AssignTechnician, RegisterIntervention, CompleteVisit, RegisterTechnician
Queries	GetVisitsByBuilding, GetInterventionHistory, GetAccumulatedSavings
Events	VisitScheduled, VisitConfirmed, RescheduleProposed, TechnicianAssigned, InterventionRegistered, VisitCompleted, SavingsProjected

Notifications (Support)

Elemento	Detalle
Aggregates	Notification, NotificationPreference
Commands	SendNotification, MarkNotificationAsRead, UpdatePreference
Queries	GetMyNotifications, GetMyPreferences
Events	NotificationSent, NotificationFailed, NotificationRead

IAM (Generic)

Elemento	Detalle
Aggregates	User
Commands	SignUp, SignIn, RequestPasswordReset, ResetPassword, UpdateProfile
Queries	GetCurrentUser, GetUserById

Elemento	Detalle
Events	UserRegistered, PasswordResetRequested, PasswordChanged

Los contextos se comunican en cuatro puntos. AlertRaised, SensorDisconnected, IncidentStatusChanged, PasswordResetRequested y los eventos de visita generan avisos en Notifications. Un incidente que requiere atención técnica pide una visita a Maintenance. VisitCompleted actualiza el estado del equipo en Asset Monitoring.

4.6.2. Software Architecture Context Diagram

Vigilia tiene tres usuarios: el administrador del edificio, el residente y la empresa de mantenimiento con sus representantes. Recibe lecturas de la red de sensores IoT, guarda las fotos de incidentes en el servicio de almacenamiento de imágenes y envía correos por un servicio SMTP.

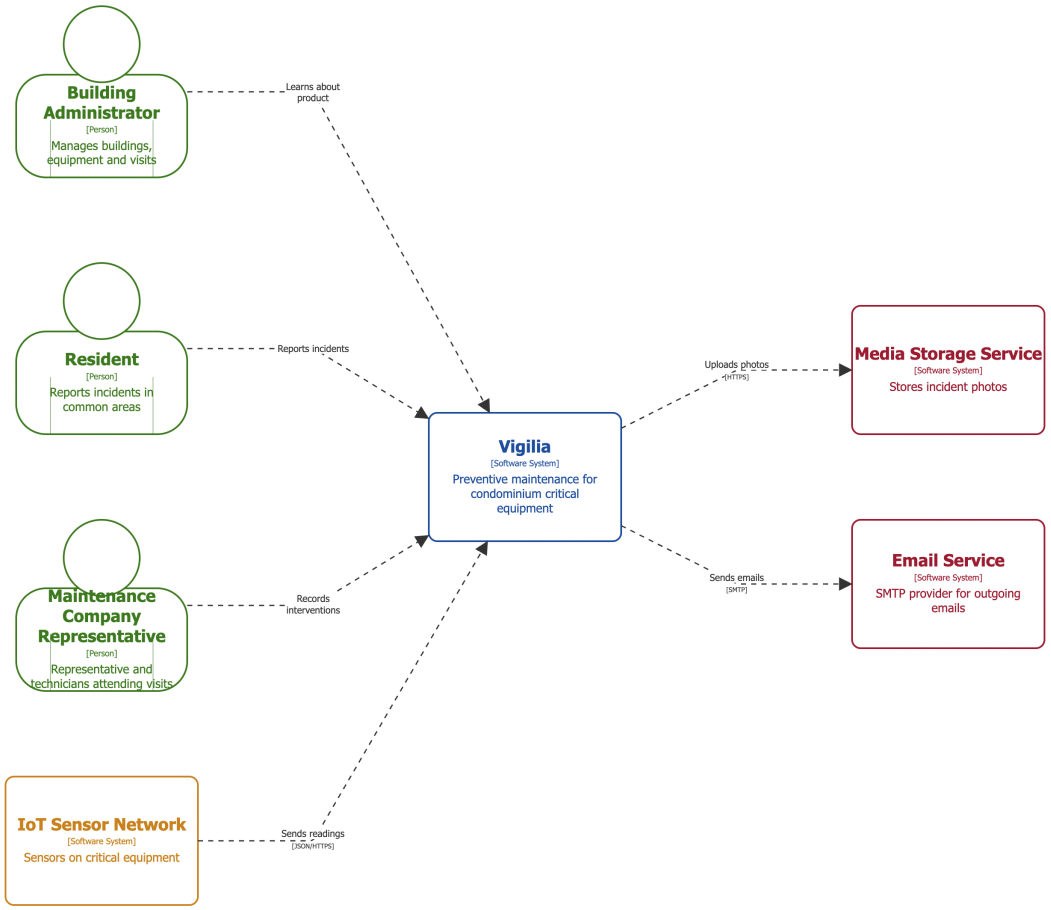


Figura 39: Figura 4.6.2 – Context Diagram de Vigilia Control

La red de sensores, el servicio de almacenamiento de imagen y el servicio de correo son sistemas de terceros. En el MVP, un simulador reemplaza a la red de sensores y envía lecturas por el mismo endpoint que usarían los sensores reales.

4.6.3. Software Architecture Container Diagrams

Vigilia tiene cinco containers. Cada uno es una unidad de despliegue independiente.

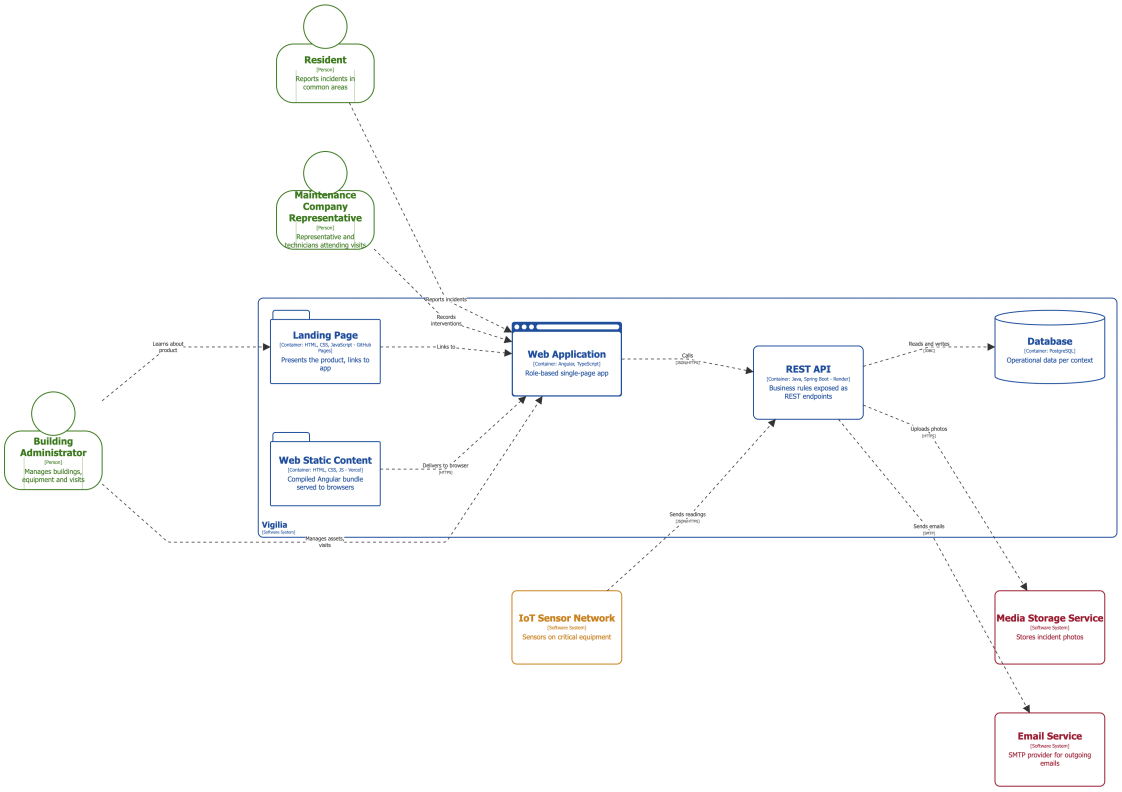


Figura 40: Figura 4.6.3 – Container Diagram de Vigilia Control

Landing Page y Web Static Content son contenido estático (HTML, CSS y JS), por eso tienen forma de carpeta. Landing Page está en GitHub Pages y Web Static Content en Vercel. La Web Application es la aplicación Angular que corre en el navegador: es la única que llama al REST API con JSON sobre HTTPS. El REST API (Spring Boot en Render) recibe las lecturas IoT, sube las fotos al servicio de almacenamiento de imágenes, envía correos por SMTP y guarda los datos en PostgreSQL con Spring Data JPA.

Landing Page es un container aparte porque usa otra tecnología y se actualiza sin tocar la aplicación. Mientras el REST API no está desplegado, la Web Application usa un fake API con los mismos endpoints: json-server en local para desarrollo y json-server desplegado en Render para producción. Cambiar al API real solo cambia la URL base en `environment.ts`.

4.6.4. Software Architecture Components Diagrams

Los dos containers con lógica, la Web Application y el REST API, tienen un componente por bounded context. Así, el frontend y el backend usan los mismos nombres de contexto.

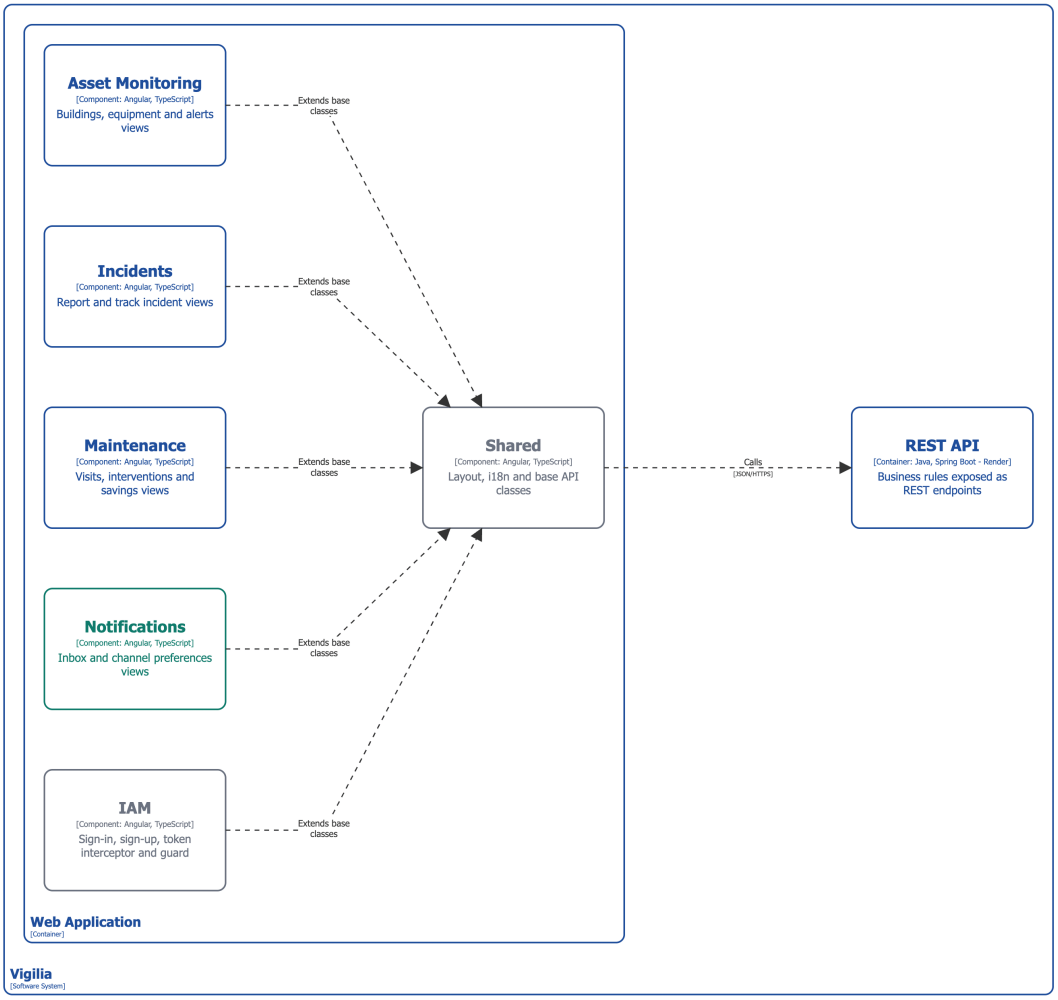


Figura 41: Figura 4.6.4-1 – Component Diagram de la Web Application

Cada componente web es una carpeta con las capas `domain`, `application`, `infrastructure` y `presentation`. Cada contexto tiene un solo store (por ejemplo `AssetMonitoringStore`) y un API facade que agrupa sus endpoints. `shared` contiene el layout, la internacionalización y las

clases base. Los endpoints de cada contexto heredan de `BaseApiEndpoint`, que es quien hace las llamadas HTTP. En IAM, un interceptor agrega el token a cada solicitud y un guard protege las rutas.

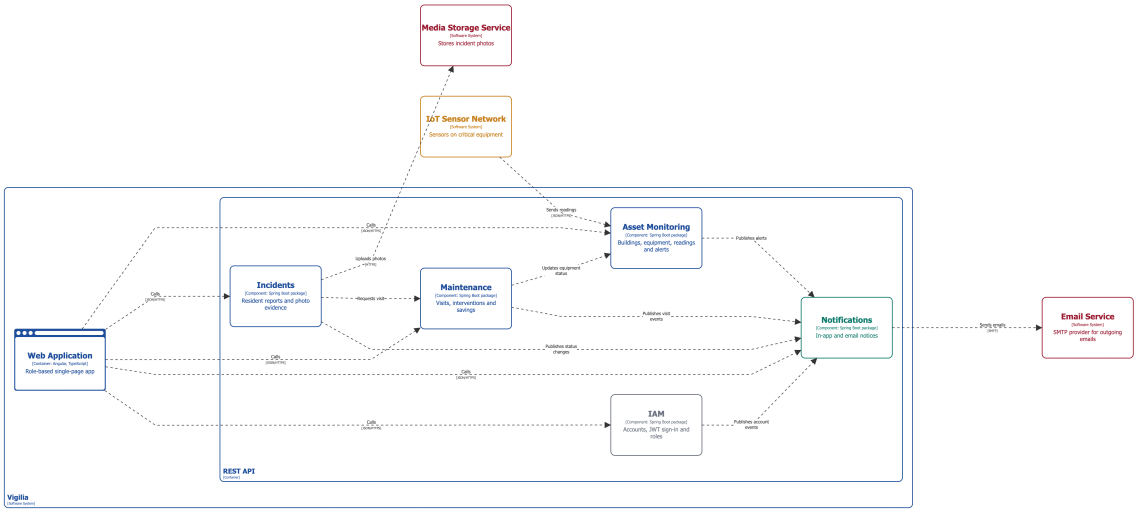


Figura 42: Figura 4.6.4-2 – Component Diagram del REST API

Incidents pide visitas a Maintenance, y Maintenance actualiza el estado del equipo en Asset Monitoring. Los demás contextos publican eventos que Notifications convierte en avisos y correos. IAM valida el token JWT de cada solicitud. Esa relación no se dibuja para no cruzar todo el diagrama.

Bounded Context	Recursos REST
Asset Monitoring	`/buildings`, `/equipment`, `/sensors`, `/sensor-readings`, `/alerts`
Incidents	`/incidents`
Maintenance	`/maintenance-visits`, `/maintenance-companies`, `/technicians`, `/savings-projections`
Notifications	`/notifications`, `/notification-preferences`
IAM	`/authentication/sign-in`, `/authentication/sign-up`, `/users`

4.7. Software Object-Oriented Design

Los class diagrams están hechos en PlantUML y se organizan por producto. Para el REST API hay un diagrama por bounded context (CD-01 a CD-05), con tipos Java. Para la Web Application hay un diagrama de `shared` (CD-06) y diagramas de los contextos que implementamos en el Sprint 2: Asset Monitoring (CD-07 y CD-08) e Incidents (CD-09). Asset Monitoring va en dos diagramas porque en uno solo no se leían los nombres. Los demás contextos web se agregan en el sprint en que se implementan.

Las asociaciones dentro de un contexto llevan nombre, dirección y multiplicidad. Una referencia a otro contexto es un atributo de identificador (por ejemplo `buildingId`), sin asociación.

4.7.1. Class Diagrams

CD-01. Asset Monitoring

Building agrupa los equipos críticos. Cada equipo define un MonitoringThreshold por métrica. Un sensor pertenece a un solo equipo activo a la vez y guarda en `lastSeenAt` su última señal; si pasa más de 24 horas sin lecturas, queda DISCONNECTED. AlertDeduplicationPolicy arma la clave equipo + métrica: si ya hay una alerta activa con esa clave, la actualiza en lugar de crear otra.

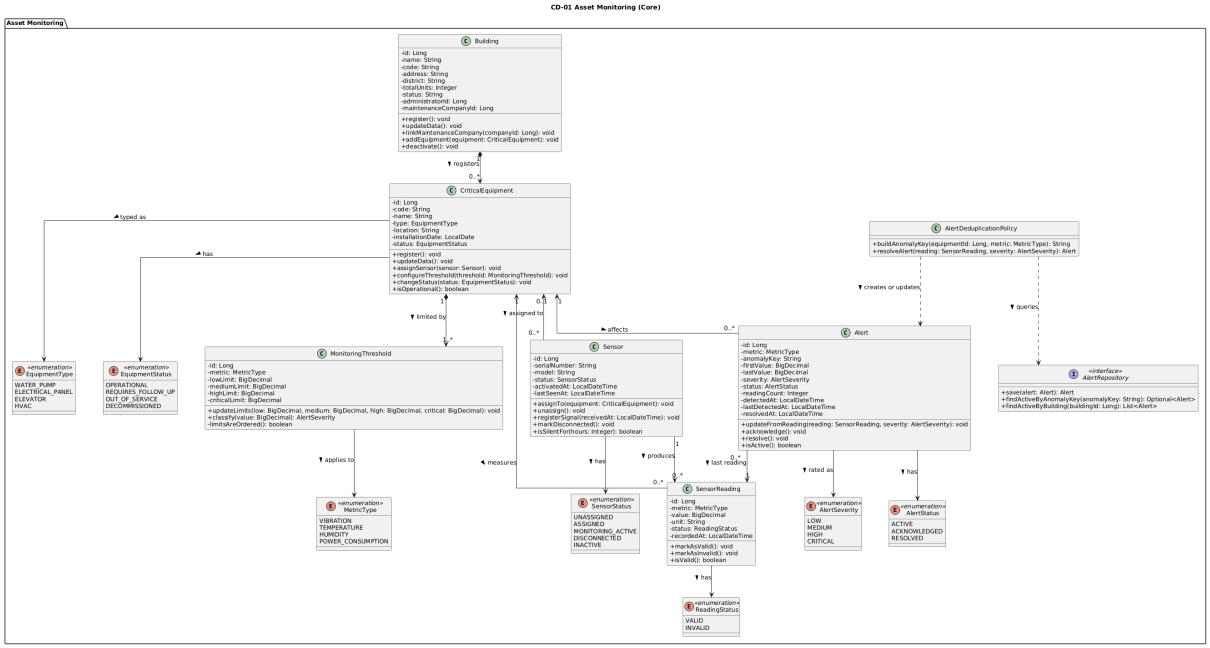


Figura 43: Figura 4.7.1-1 – Class Diagram de Asset Monitoring

CD-02. Incidents

Incident es el agregado. Las referencias a edificio, residente, equipo y visita son identificadores de otros contextos. Un incidente acepta hasta tres IncidentEvidence; la foto se guarda en Cloudinary y Vigilia conserva la URL y el `publicId`. El residente califica una sola vez, de 1 a 5, y solo cuando el incidente está RESOLVED.

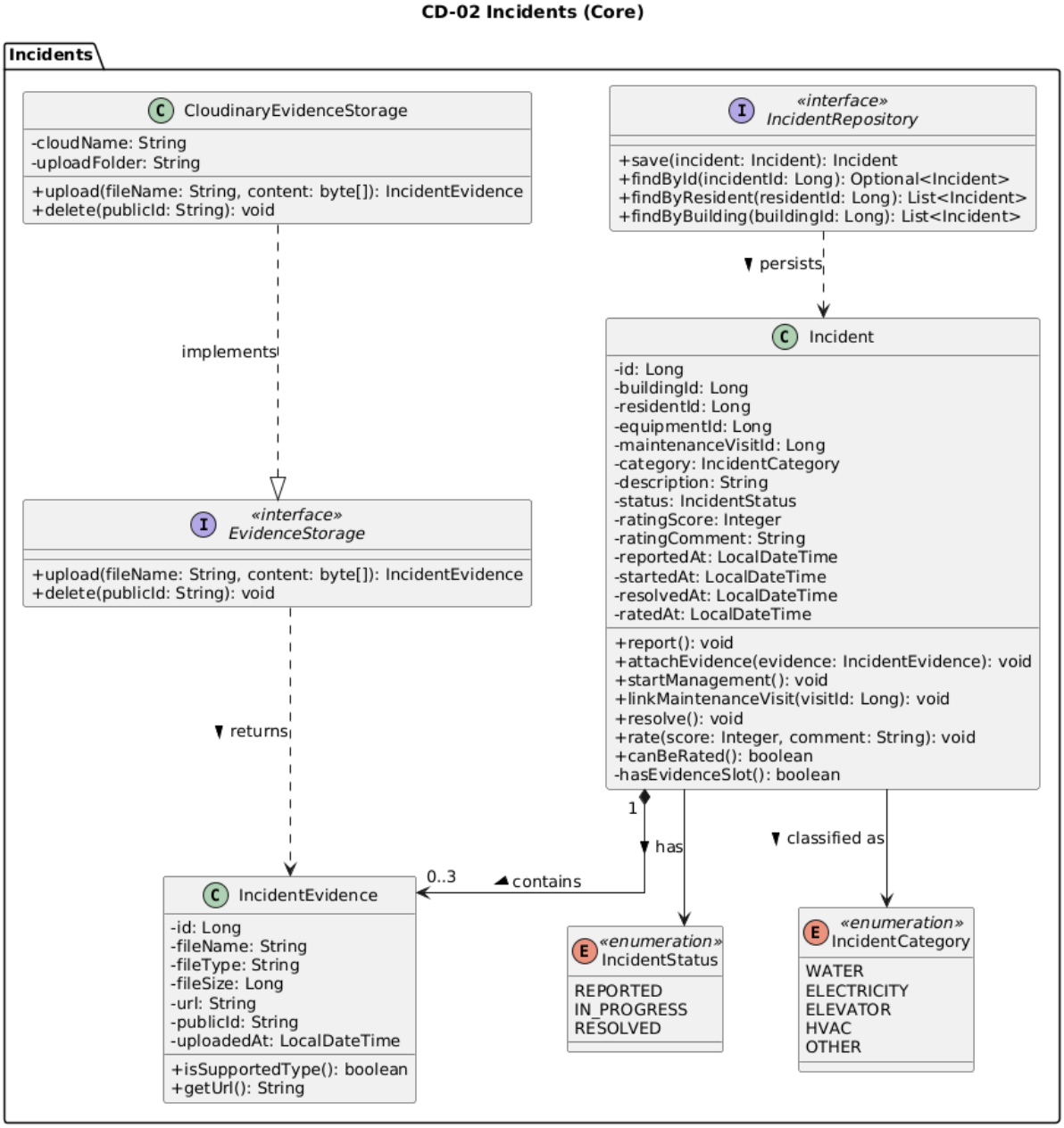


Figura 44: Figura 4.7.1-2 – Class Diagram de Incidents

CD-03. Maintenance

MaintenanceVisit es el agregado principal. La reprogramación es un estado (RESCHEDULE_PROPOSED) más la fecha y el motivo propuestos. La visita pasa a COMPLETED solo si tiene una Intervention con el estado final del equipo. La empresa registra en la intervención el costo preventivo. Si el estado final es OPERATIONAL, se crea una SavingsProjection con ese costo y una copia del costo de referencia vigente.

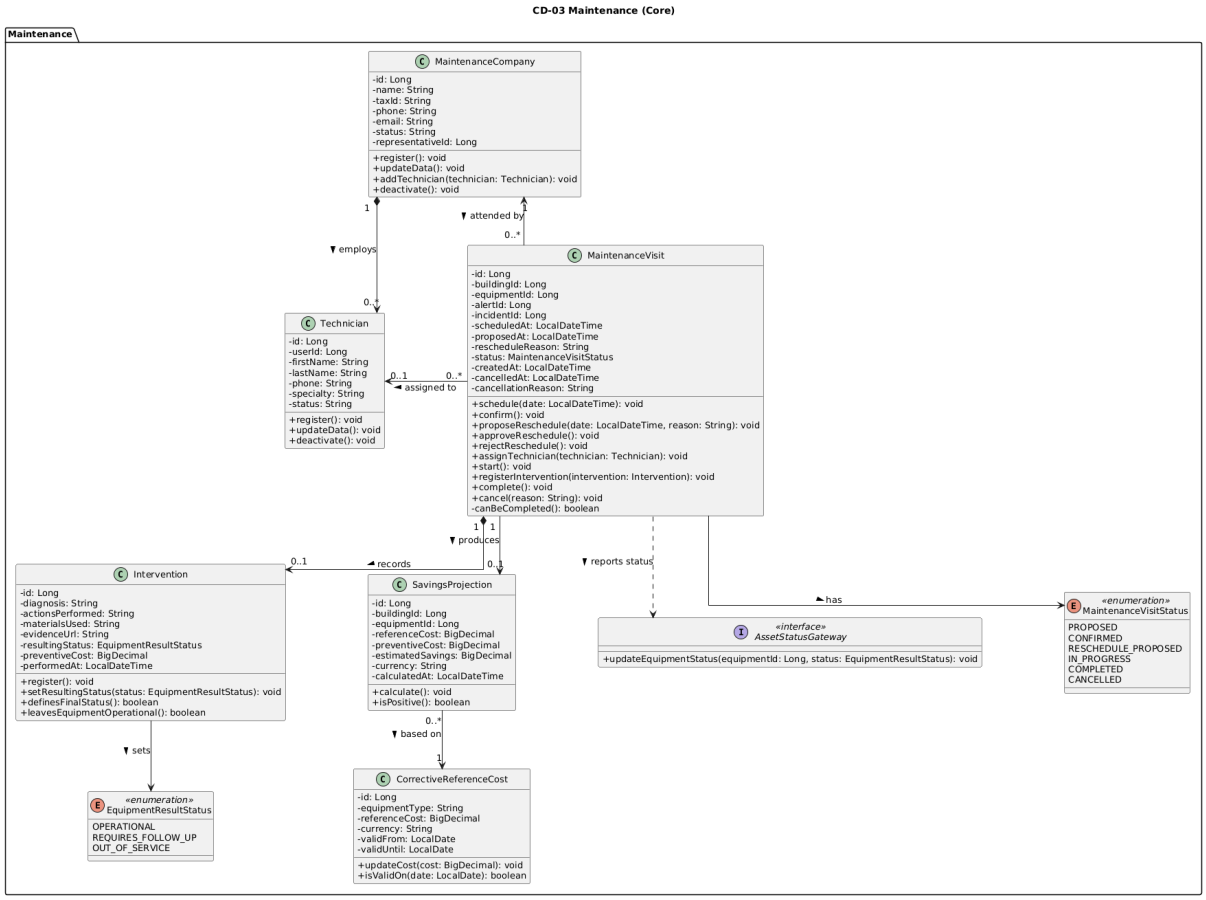


Figura 45: Figura 4.7.1-3 – Class Diagram de Maintenance

CD-04. Notifications

Cada Notification tiene un destinatario y un canal, IN_APP o EMAIL. NotificationDispatcher revisa la NotificationPreference del usuario antes de entregar el aviso al NotificationSender del canal.

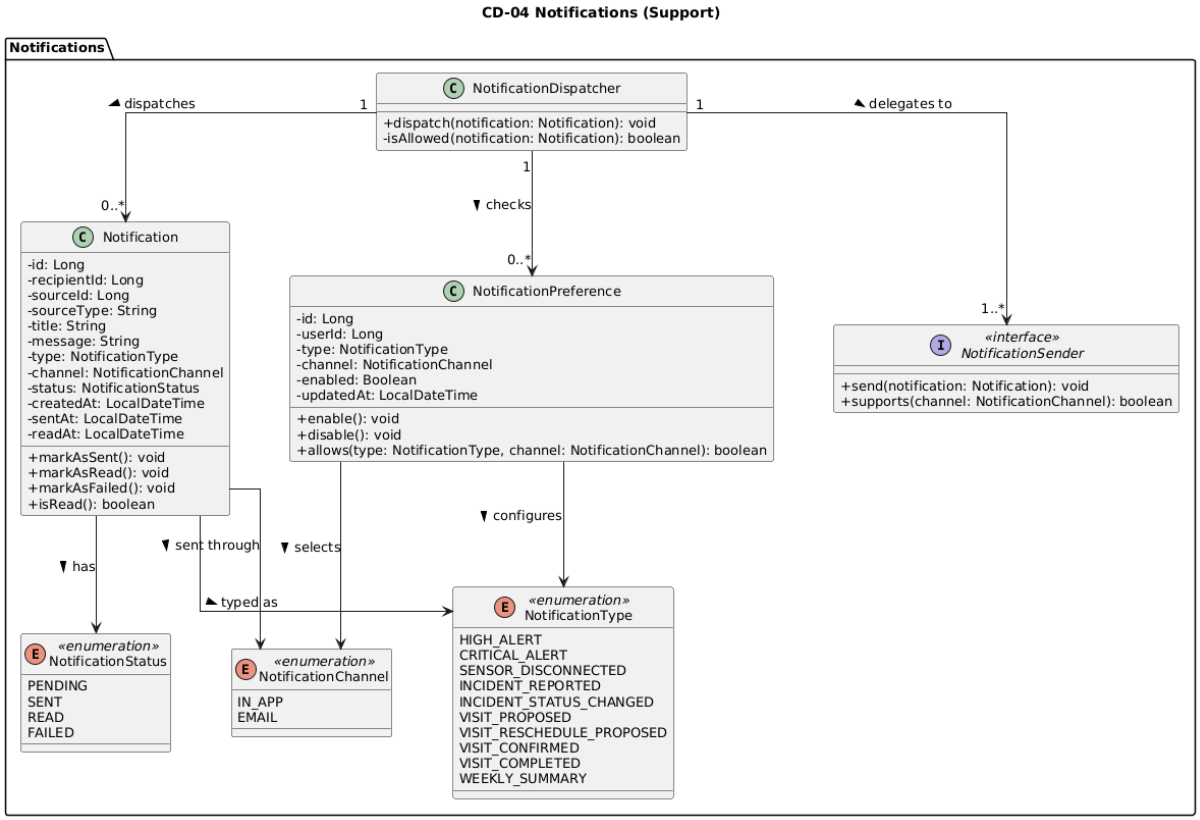


Figura 46: Figura 4.7.1-4 – Class Diagram de Notifications

CD-05. IAM

User guarda el rol: administrador, residente, representante de empresa de mantenimiento o técnico. La contraseña se guarda como hash. PasswordResetToken sirve para recuperar la cuenta y se usa una sola vez. El inicio de sesión devuelve un token JWT; no hay tabla de sesiones.

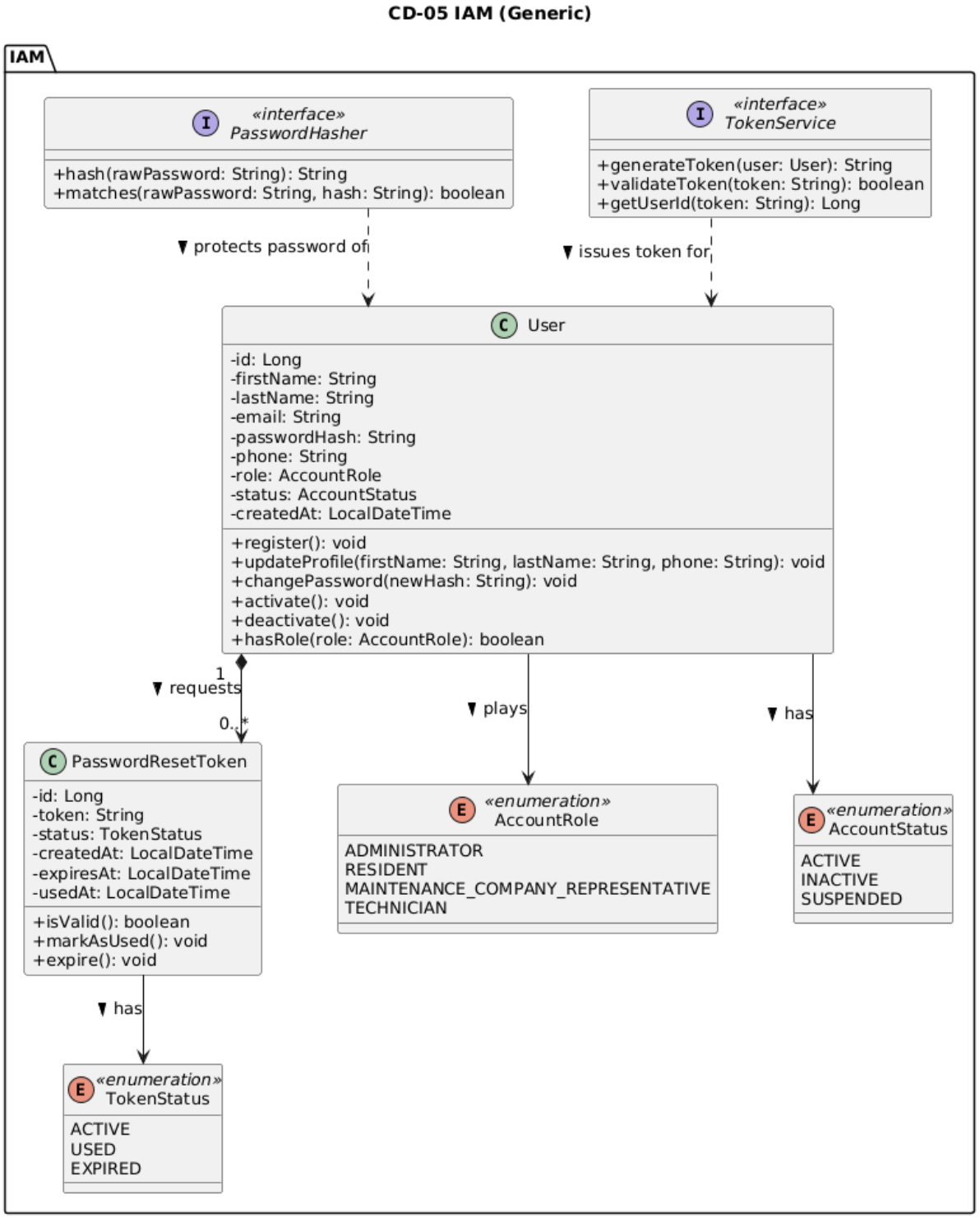


Figura 47: Figura 4.7.1-5 – Class Diagram de IAM

CD-06. Web Application: Shared

`shared` tiene las clases base que usan todos los contextos del frontend. `BaseApiEndpoint` hace las operaciones CRUD con `HttpClient` y usa un `BaseAssembler` para convertir recursos JSON en entidades. `BaseApi` agrupa los endpoints de un contexto. `BaseForm` centraliza los mensajes de validación de los formularios.

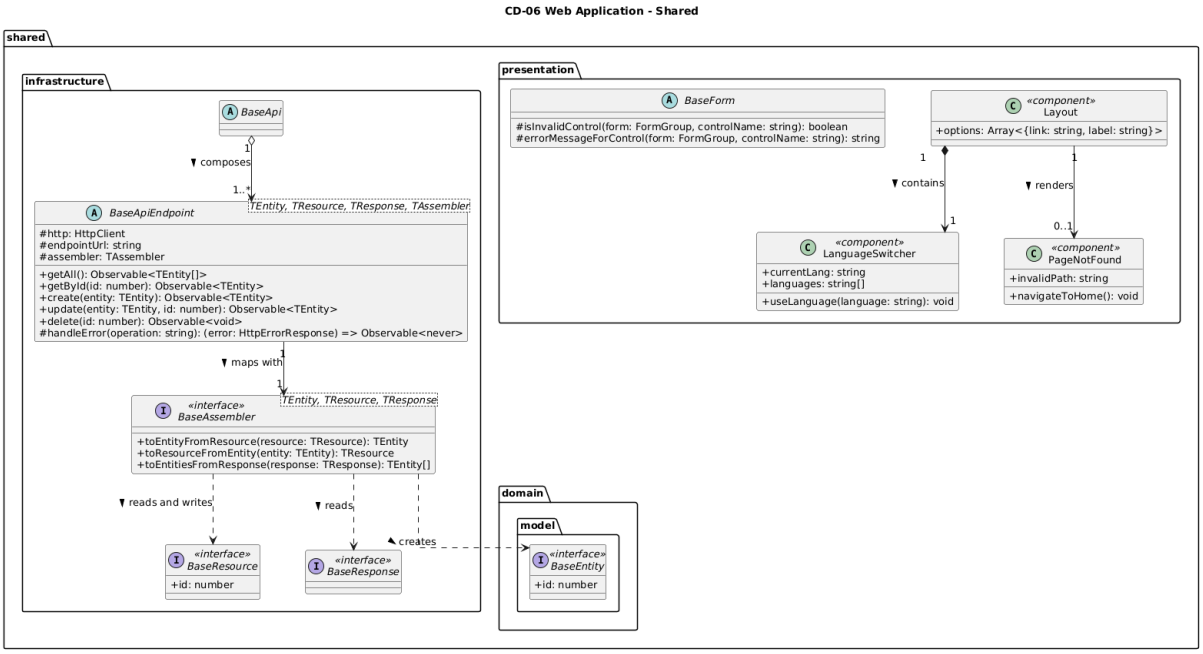


Figura 48: Figura 4.7.1-6 – Class Diagram de Web Application: Shared

CD-07. Web Application: Asset Monitoring (domain y application)

Las entidades usan campos privados # con get y set públicos, y guardan las reglas simples. Por ejemplo, `CriticalEquipment.decommission()` cambia el estado a DECOMMISSIONED sin borrar el equipo, y `Sensor.assignTo()` lanza un error si el sensor ya tiene equipo. `AssetMonitoringStore` es el único store del contexto: guarda el estado en signals y expone `activeAlerts`, ordenadas de mayor a menor severidad.

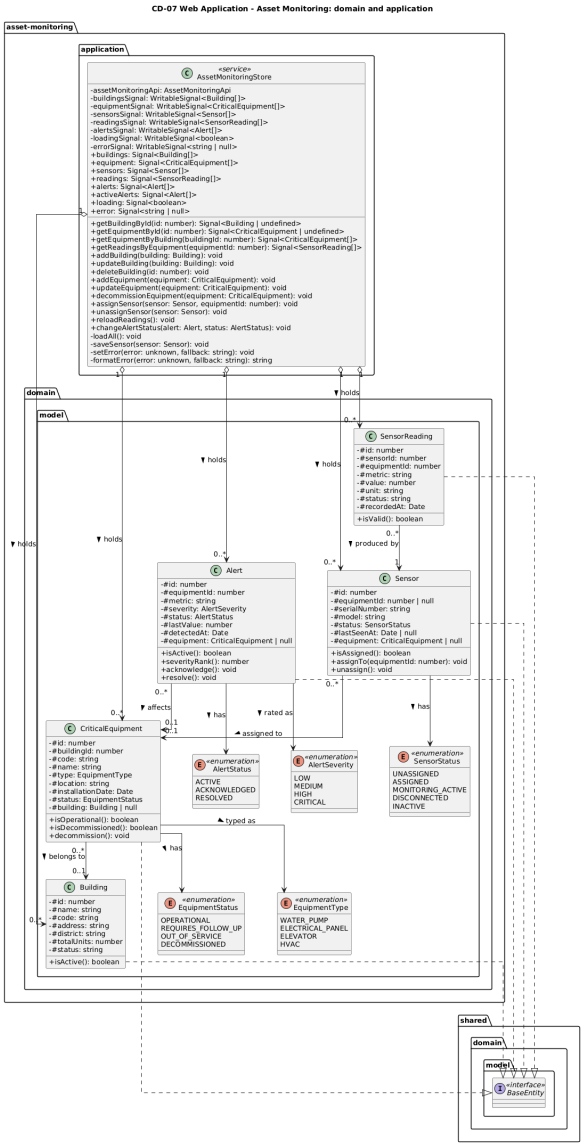


Figura 49: Figura 4.7.1-7 – Class Diagram de Web Application: Asset Monitoring, domain y application

CD-08. Web Application: Asset Monitoring (infrastructure y presentation)

`AssetMonitoringApi` agrupa un endpoint por recurso: edificios, equipos, sensores, lecturas y alertas. Cada endpoint usa su assembler para pasar de JSON a entidad. Las vistas solo leen el store y llaman a sus métodos; los formularios heredan de `BaseForm`.

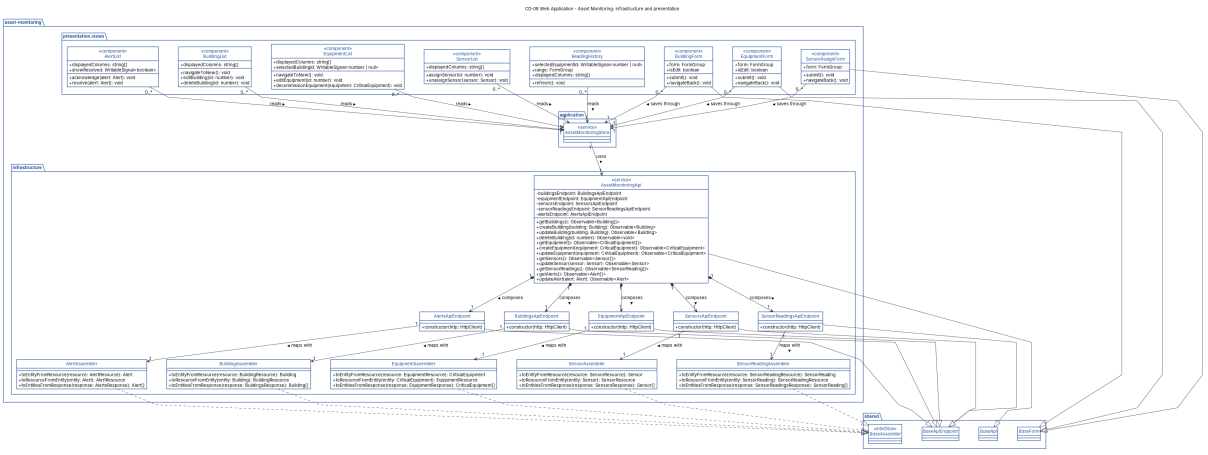


Figura 50: Figura 4.7.1-8 – Class Diagram de Web Application: Asset Monitoring, infrastructure y presentation

CD-09. Web Application: Incidents

Incidents tiene sus cuatro capas y su propio store, `IncidentsStore`. `Incident.canBeRated()` solo es verdadero si el incidente está RESOLVED y aún no tiene calificación. El formulario de reporte lee los edificios de `AssetMonitoringStore`; es la única lectura entre contextos del frontend. Mientras no hay IAM, el residente es un usuario de prueba fijo.

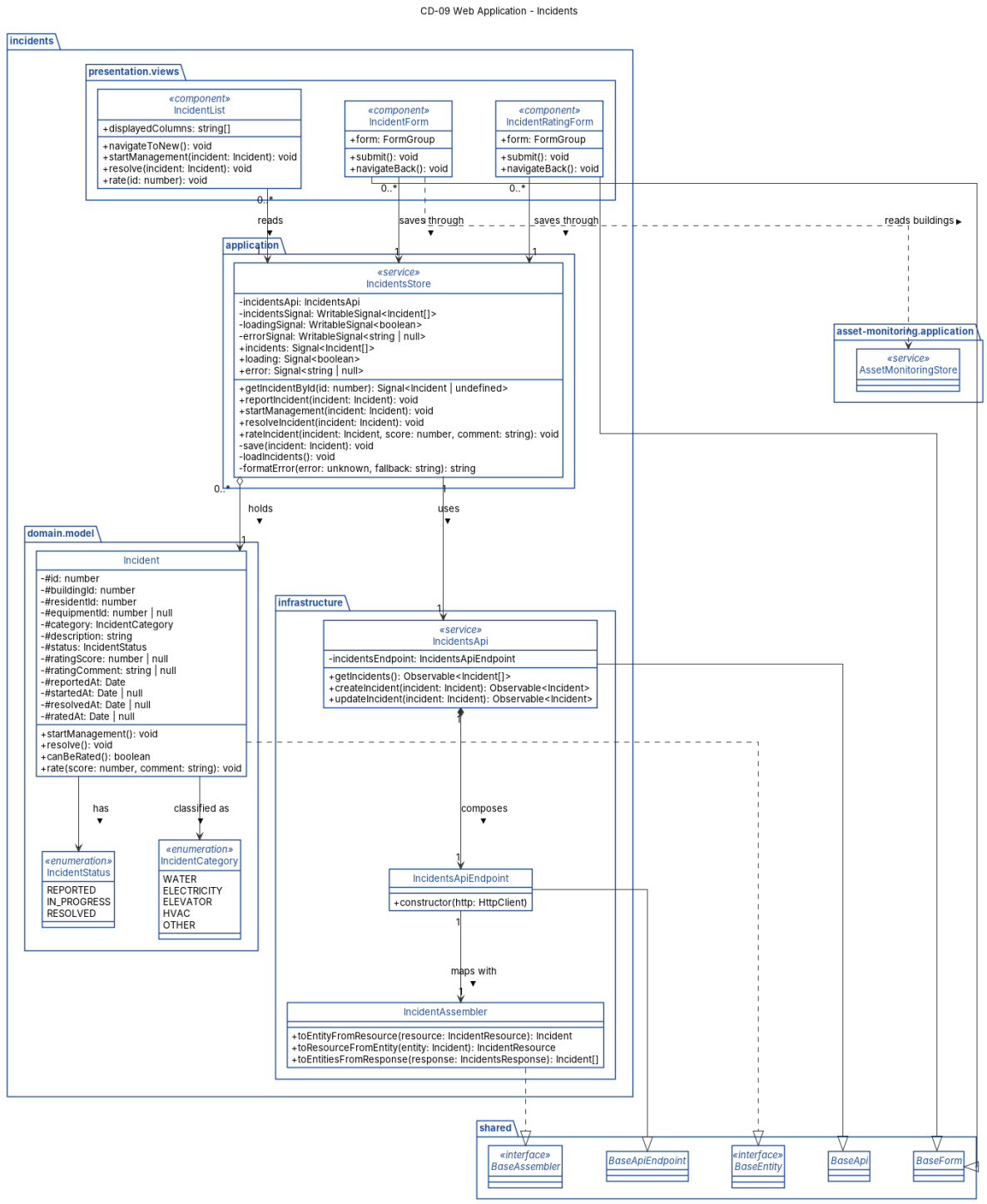


Figura 51: Figura 4.7.1-9 – Class Diagram de Web Application: Incidents

4.8. Database Design

PostgreSQL guarda los cinco contextos en una sola base de datos. Cada tabla lleva el prefijo de su contexto: `asset_`, `incident_`, `maintenance_`, `notification_` e `iam_`. Las foreign keys solo unen tablas del mismo contexto; una referencia a otro contexto es una columna de identificador sin FK. Los enums se guardan como VARCHAR.

4.8.1. Database Diagrams

4.8.1.1. Asset Monitoring

`asset_sensors.equipment_id` admite NULL, porque un sensor puede estar sin asignar. Una sola columna de equipo hace que el sensor pertenezca a un equipo a la vez.

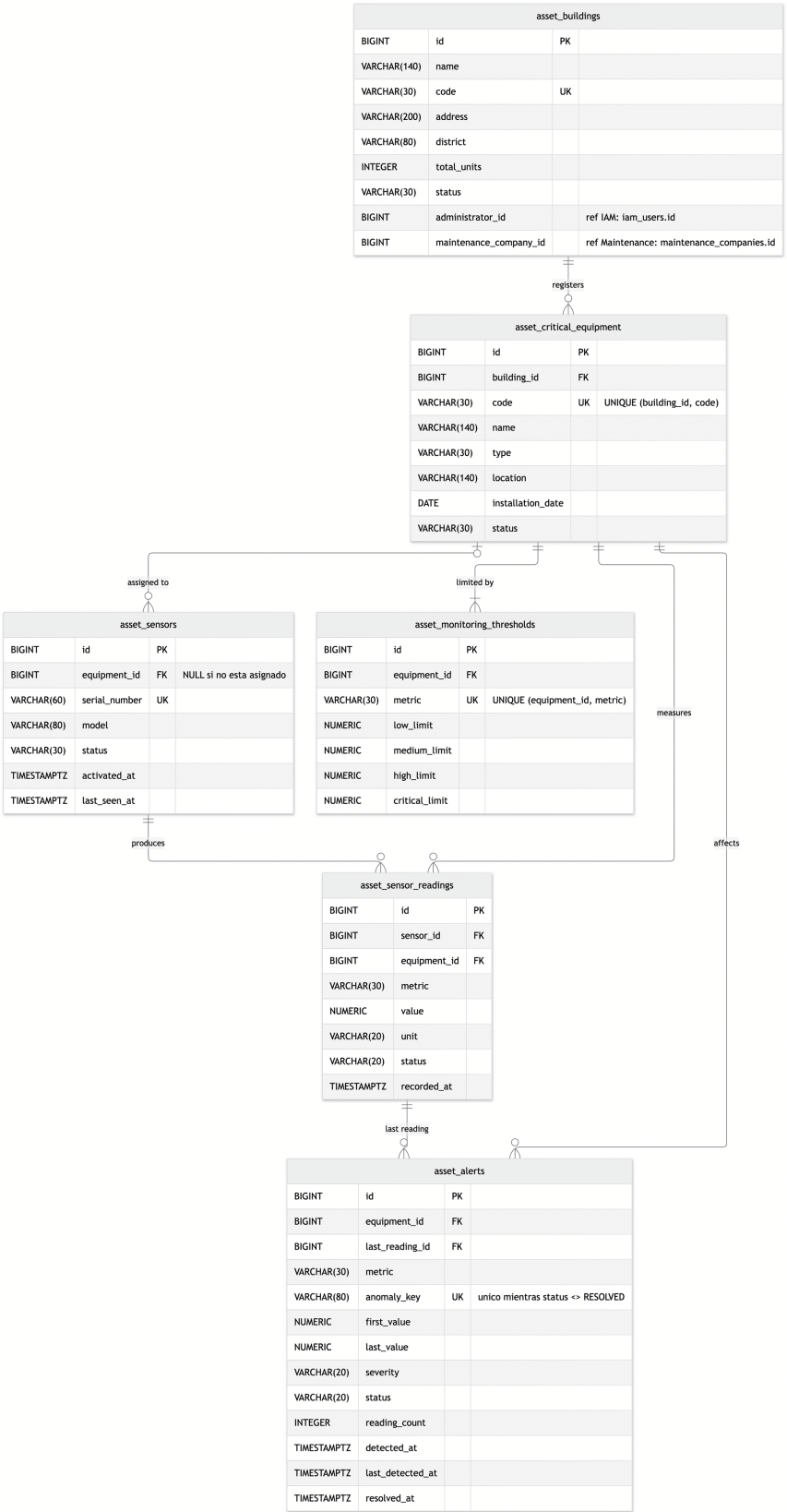


Figura 52: Figura 4.8.1-1 – Database Diagram de Asset Monitoring

Tabla	Constraint	Motivo
asset_critical_equipment	UNIQUE (building_id, code)	Código único por edificio
asset_monitoring_thresholds	UNIQUE (equipment_id, metric)	Un umbral por métrica
asset_alerts	UNIQUE (anomaly_key) WHERE status <> ‘RESOLVED’	Sin alertas activas duplicadas

4.8.1.2. Incidents

`incident_evidence.public_id` es único porque identifica el archivo en el sistema de almacenamiento de imágenes. El límite de tres evidencias se valida en el agregado Incident. La calificación es una columna del incidente con CHECK de 1 a 5.

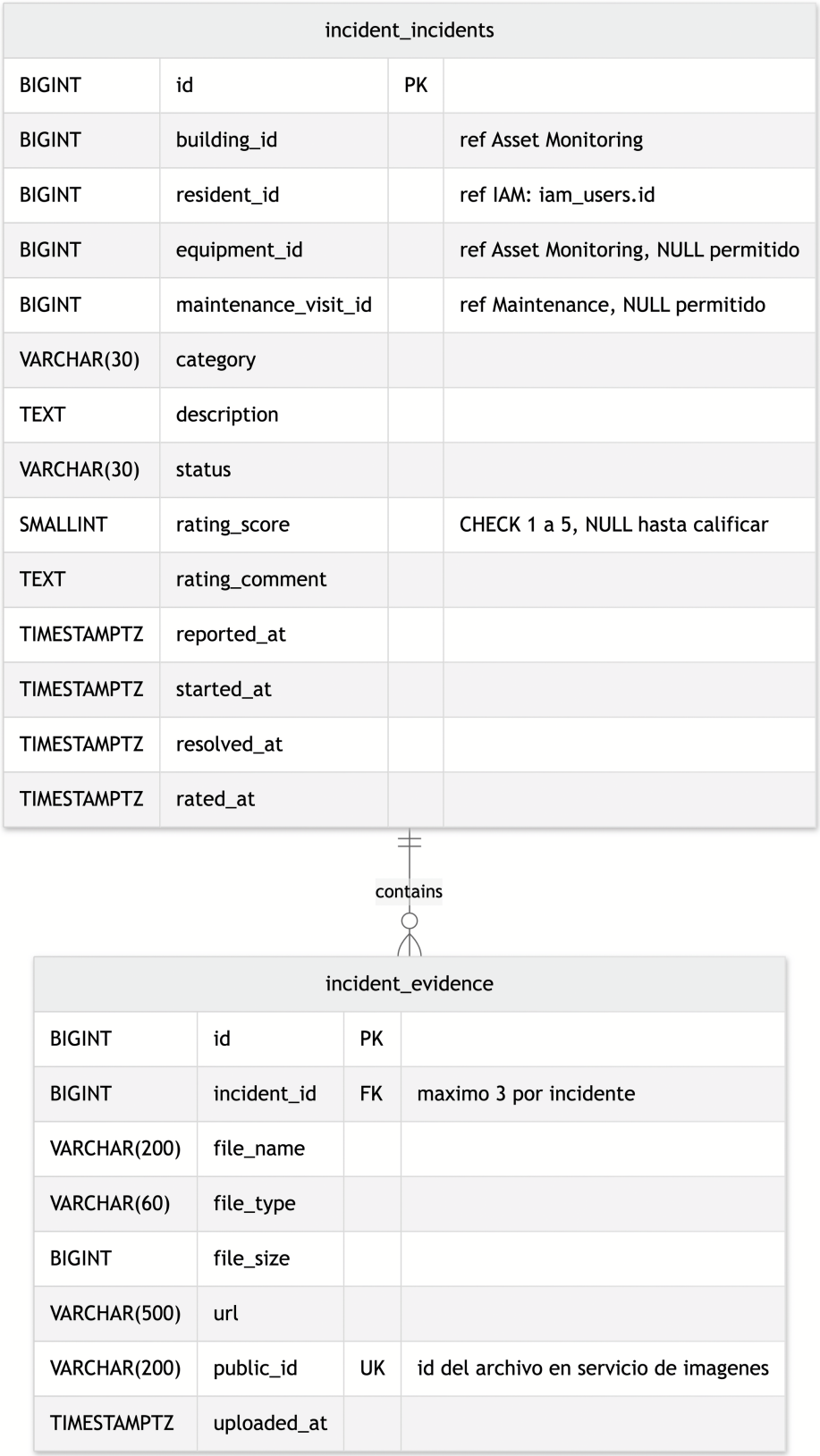


Figura 53: Figura 4.8.1-2 – Database Diagram de Incidents

4.8.1.3. Maintenance

`visit_id` es UNIQUE en intervenciones y en proyecciones de ahorro: cada visita tiene como máximo una de cada una. `maintenance_savings_projections.reference_cost` copia el valor usado, así el historial no cambia si se actualiza el catálogo.

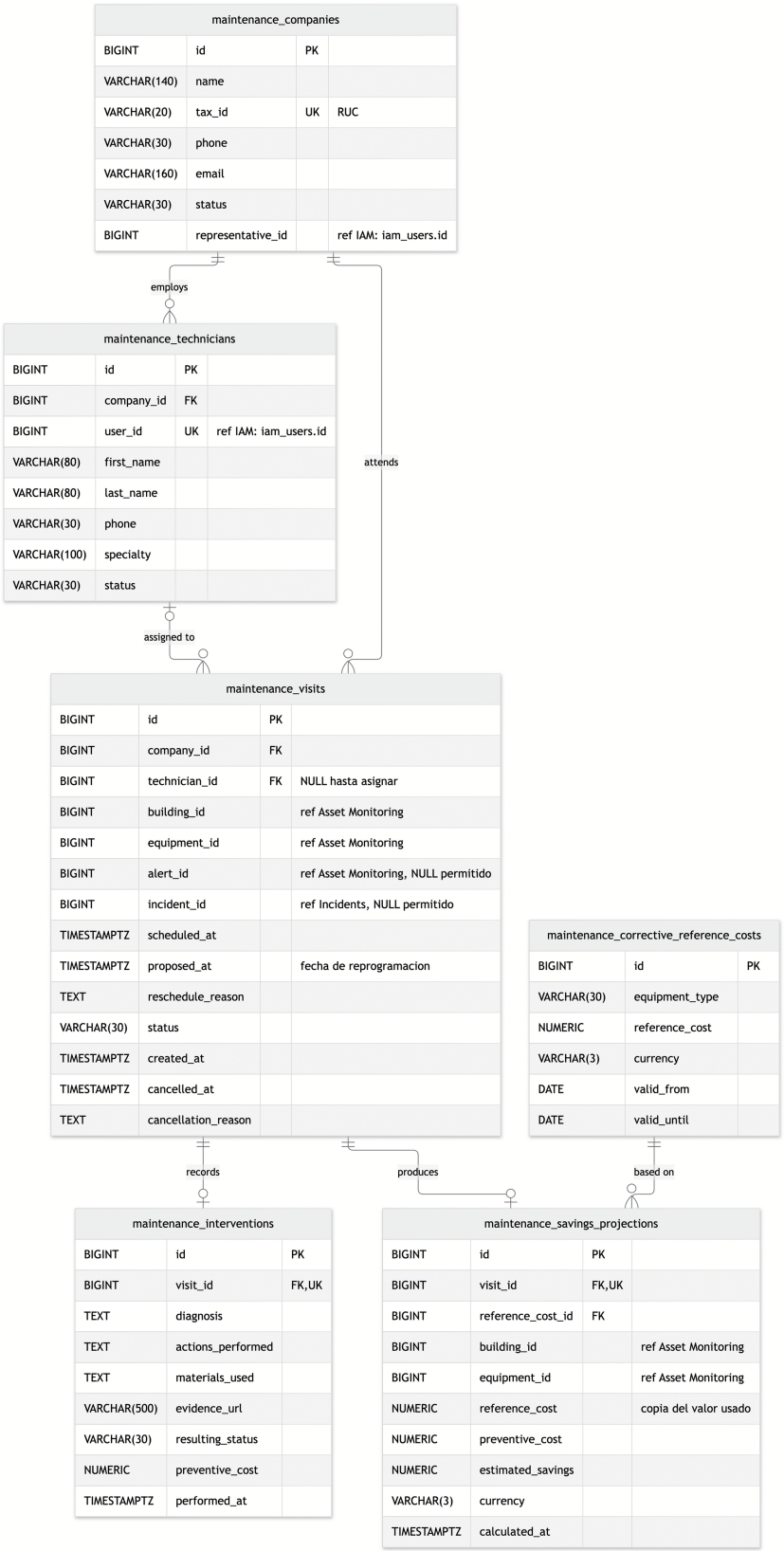


Figura 54: Figura 4.8.1-3 – Database Diagram de Maintenance

4.8.1.4. Notifications

Cada fila de `notification_notifications` es un aviso para un destinatario y un canal. `notification_preferences` tiene UNIQUE (user_id, type, channel).

notification_notifications			
BIGINT	id	PK	
BIGINT	recipient_id		ref IAM: iam_users.id
BIGINT	source_id		id de la alerta, incidente o visita
VARCHAR(30)	source_type		
VARCHAR(140)	title		
TEXT	message		
VARCHAR(40)	type		
VARCHAR(20)	channel		IN_APP o EMAIL
VARCHAR(20)	status		
TIMESTAMPTZ	created_at		
TIMESTAMPTZ	sent_at		
TIMESTAMPTZ	read_at		

notification_preferences			
BIGINT	id	PK	
BIGINT	user_id	UK	ref IAM; UNIQUE (user_id, type, channel)
VARCHAR(40)	type		
VARCHAR(20)	channel		
BOOLEAN	enabled		
TIMESTAMPTZ	updated_at		

Figura 55: Figura 4.8.1-4 – Database Diagram de Notifications

4.8.1.5. IAM

`iam_users.email` y `iam_password_reset_tokens.token` son UNIQUE. Así se evitan cuentas duplicadas y tokens reutilizados.

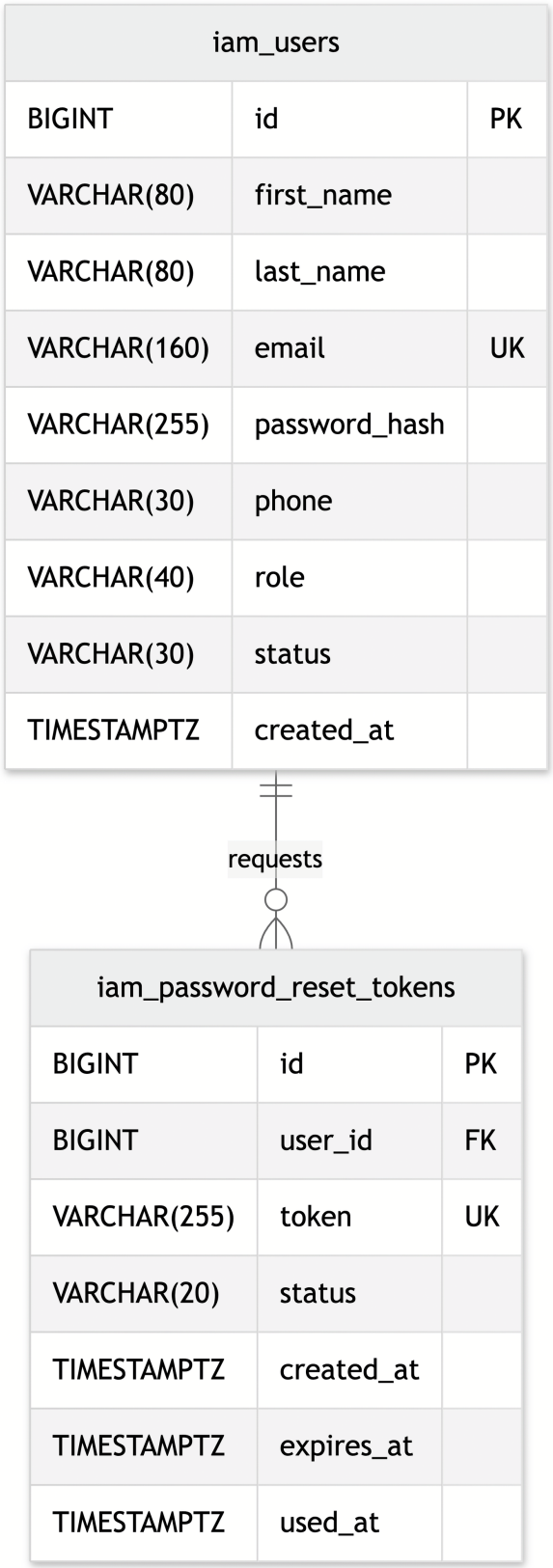


Figura 56: Figura 4.8.1-5 – Database Diagram de IAM

Capítulo V: Product Implementation, Validation & Deployment

5.1. Software Configuration Management

En esta sección definimos cómo trabajamos el código y los documentos durante todo el proyecto: qué herramientas usamos, cómo manejamos las ramas, qué convenciones de código seguimos y cómo desplegamos cada producto.

5.1.1. Software Development Environment Configuration

Estas son las herramientas que usa el equipo, agrupadas por actividad.

Actividad	Producto	Propósito de uso	Ruta de referencia
Project Management	Trello	Product Backlog y Sprint Backlog en tableros Kanban.	https://trello.com
Requirements Management	UXPressia	User Personas, User Journey Maps y Empathy Maps.	https://uxpressia.com
Product UX/UI Design	Figma	Wireframes, mock-ups y prototipos del Landing Page y la Web Application.	https://www.figma.com
Product UX/UI Design	FigJam	Wireflows, user flows y Big Picture Event Storming.	https://www.figma.com/figjam
Software Architecture Design	Miro	Design-Level Event Storming (sección 4.6.1).	https://miro.com
Software Architecture Design	Structurizr	Diagramas C4: Context, Container y Component.	https://structurizr.com

Actividad	Producto	Propósito de uso	Ruta de referencia
Software Architecture Design	PlantUML	Class diagrams del REST API y de la Web Application.	https://plantuml.com
Software Architecture Design	Mermaid	Database diagrams por bounded context.	https://mermaid.js.org
Software Development	Visual Studio Code	Desarrollo del Landing Page (HTML, CSS y JavaScript).	https://code.visualstudio.com
Software Development	WebStorm	Desarrollo de la Frontend Web Application (Angular y TypeScript).	https://www.jetbrains.com/webstorm
Software Development	Node.js 24 LTS y Angular CLI 22	Entorno de ejecución y comandos para crear, ejecutar y compilar la aplicación Angular.	https://angular.dev/tools/cli
Software Development	json-server 0.17.4	Fake API con los mismos endpoints que tendrá el RESTful API.	https://github.com/typicode/json-server
Software Deployment	GitHub Pages	Hosting del Landing Page.	https://pages.github.com
Software Deployment	Vercel	Hosting de la Frontend Web Application.	https://vercel.com
Software Deployment	Render	Hosting del fake API en el Sprint 2, y del RESTful API y PostgreSQL desde el Sprint 3.	https://render.com
Software Documentation	Markdown, Pandoc y Eisvogel	Redacción del informe en Markdown y exportación a PDF.	https://pandoc.org

Actividad	Producto	Propósito de uso	Ruta de referencia
Version Control	Git y GitHub	Control de versiones de los repositorios en la organización del equipo.	https://github.com/code-nova-1asi0729

5.1.2. Source Code Management

Usamos GitHub como plataforma de control de versiones. Todos los repositorios están en la organización pública code-nova-1asi0729:

Producto	Repositorio
Landing Page	https://github.com/code-nova-1asi0729/landing-page
Frontend Web Application	https://github.com/code-nova-1asi0729/Frontend
RESTful API	Se crea en el Sprint 3.
Informe del proyecto	https://github.com/code-nova-1asi0729/project-report

GitFlow. Cada repositorio sigue GitFlow:

- `main`: solo tiene versiones estables. Cada merge a `main` despliega el producto.
- `develop`: rama de integración. Aquí se juntan las features terminadas.
- `feature/<nombre>`: una rama por historia o tarea, creada desde `develop`. El nombre está en inglés y en kebab-case. Ejemplos: `feature/asset-monitoring-equipment`, `feature/asset-monitoring-alerts`, `feature/incidents`.
- `release/<versión>`: prepara una versión antes de pasarla a `main`. Ejemplo: `release/0.2.0`.
- `hotfix/<nombre>`: corrige un error urgente sobre `main`. Ejemplo: `hotfix/fix-equipment-date-offset`.

Ninguna rama entra a `develop` ni a `main` sin un Pull Request.

Conventional Commits. Los mensajes de commit siguen el formato `<tipo>(<scope>): <descripción>`, en inglés y en presente. El scope es el bounded context o la parte compartida: `asset-monitoring`, `incidents`, `maintenance`, `notifications`, `iam` o `shared`. En el informe el scope es el capítulo, por ejemplo `docs(chapter-5)`.

```
feat(asset-monitoring): add severity classification to alerts
fix(iam): correct token expiration validation
refactor(maintenance): extract scheduling logic into service
test(incidents): add unit tests for incident status transitions
docs(chapter-5): add sprint 2 backlog
style(shared): format layout styles
chore(deps): update Angular to v22
```

5.1.3. Source Code Style Guide & Conventions

Todo el código se escribe en inglés: clases, variables, métodos, componentes, archivos, tablas y endpoints. El dominio se nombra con los términos del Ubiquitous Language (sección 2.5).

Lenguaje o tecnología	Convención	Ejemplo en Vigilia
HTML y CSS	Google HTML/CSS Style Guide	Clases en kebab-case: `.alert-card`, `.severity-chip`
TypeScript	Angular Style Guide y Google TypeScript Style Guide	Archivos en kebab-case (`critical-equipment.entity.ts`). Clases en PascalCase y métodos en camelCase. Entidades con campos privados #. Un store por bounded context (`AssetMonitoringStore`).

Lenguaje o tecnología	Convención	Ejemplo en Vigilia
Angular	Estructura por bounded context	Una carpeta por contexto con `domain`, `application`, `infrastructure` y `presentation`. Los componentes con ruta van en `views` y se cargan con lazy loading.
i18n	Claves por pantalla en `public/i18n/en.json` y `es.json`	`equipment.title`, `alerts.resolve`. Ningún texto fijo en las vistas.
Java	Google Java Style Guide	Clases en PascalCase (`MaintenanceVisit`) y métodos en camelCase (`completeVisit()`).
Spring Boot	Paquetes por bounded context	`com.codenova.vigilia.assetmonitor` `.incidents`, `.maintenance`, `.notifications`, `.iam`
REST	Sustantivos en plural y base `/api/v1`	`/api/v1/equipment`, `/api/v1/sensor-readings`. Las referencias usan el recurso en singular más `Id`: `buildingId`.
SQL	Tablas en snake_case y en plural, con prefijo del contexto	`asset_critical_equipment`, `incident_incidents`
Gherkin	Gherkin Reference	Escenarios en español con Dado, Cuando y Entonces, en tercera persona y sin detalles de interfaz (sección 3.1).

5.1.4. Software Deployment Configuration

Cada producto se despliega desde la rama `main` de su repositorio.

- **Landing Page.** Sitio estático (HTML, CSS y JavaScript) publicado en GitHub Pages. GitHub publica de nuevo el sitio cada vez que hay un cambio en `main`.
- **Frontend Web Application.** Vercel está conectado al repositorio `Frontend`. Con cada merge a `main`, Vercel ejecuta `ng build` y publica el contenido de `dist`. El archivo `vercel.json` redirige todas las rutas a `index.html`, porque Angular es una single page application. La URL base del API está en dos archivos:
  - `src/environments/environment.development.ts`: json-server en `http://localhost:30` para desarrollo.
  - `src/environments/environment.ts`: json-server desplegado en Render (`https://vigilia-fake-api.onrender.com/api/v1`), para producción.
- **Fake API.** Web Service de Render con la carpeta `server` como raíz y el comando `npm start`. Expone los datos de `db.json` con la regla `/api/v1/*` de `routes.json`. Lo usamos mientras el RESTful API no está listo.
- **RESTful API (desde el Sprint 3).** Spring Boot empaquetado como imagen Docker y desplegado en Render. La conexión a la base de datos y el secreto del JWT van en variables de entorno.
- **Base de datos (desde el Sprint 3).** Una sola instancia de PostgreSQL en Render. Las tablas de cada bounded context llevan su prefijo (sección 4.8).

Cuando el RESTful API esté desplegado, solo cambiaremos la URL de `environment.ts`. Las vistas y los stores no cambian.

5.2. Landing Page, Services & Applications Implementation

En esta sección mostramos cómo avanzó la implementación del Landing Page, la Frontend Web Application y los Web Services en cada sprint. Cada sprint parte del Product Backlog de la sección 3.3 e incluye su planificación, las tareas, las evidencias de desarrollo y despliegue, y cómo trabajó el equipo.

5.2.1. Sprint 1

El Sprint 1 tuvo un objetivo concreto: publicar la primera versión del Landing Page para la entrega AV1. En el planning también incluimos el registro y el inicio de sesión, pero esa parte no se llegó a implementar. En el Sprint 2 explicamos por qué la movimos al final del backlog (sección 5.2.2.1).

5.2.1.1. Sprint Planning 1

La reunión de planning se hizo de forma virtual. En ella definimos el goal, revisamos las historias del landing page y repartimos las tareas.

Sprint #	Sprint 1
Sprint Planning Background	
Date	2026-09-02
Time	12:00 PM
Location	Reunión virtual en Google Meet
Prepared By	Valladolid Jiménez, Arturo Fernando / Romero Veliz, Matthias Alonso
Attendees (to planning meeting)	Valladolid Jiménez, Arturo Fernando / Diaz Vargas, Fernanda Ysabella / Romero Veliz, Matthias Alonso / Salazar Miranda, Mateo Paolo
Sprint 0 Review Summary	No aplica. Es el primer sprint del proyecto.
Sprint 0 Retrospective Summary	No aplica. Es el primer sprint del proyecto.
Sprint Goal & User Stories	
Sprint 1 Goal	Nuestro foco es dar a los visitantes y futuros administradores una primera impresión confiable de Vigilia y un punto de entrada a la plataforma. Creemos que esto genera confianza en los administradores que buscan una forma de prevenir fallas en su edificio. Lo confirmaremos cuando un visitante pueda ver el Landing Page desde cualquier dispositivo y solicitar una demo sin ayuda del equipo.
Sprint 1 Velocity	18 story points
Sum of Story Points	18 (US48: 2, US49: 2, US50: 3, US01: 3, US02: 3, US07: 2, US08: 3)

5.2.1.2. Aspect Leaders and Collaborators

En el Sprint 1 el trabajo de implementación fue el Landing Page. Lo dividimos en cuatro aspectos, uno por grupo de secciones del sitio. Cada aspecto tuvo un líder y los demás colaboraron revisando y ajustando.

Team Member (Last Name, First Name)	GitHub Username	Hero y Navbar	Nosotros y Equipo	Soluciones y Footer	Idioma y Animaciones
Valladolid Jiménez, Arturo Fernando	artuvall	C	C	C	L
Diaz Vargas, Fernanda Ysabella	Fern9901	C	C	L	C
Romero Veliz, Matthias Alonso	AlonsoVelizUpc	L	C	C	C
Salazar Miranda, Mateo Paolo	mateossm	C	L	C	C

5.2.1.3. Sprint Backlog 1

El Sprint 1 incluyó las historias del Landing Page y las primeras historias de registro y de edificios. La tabla muestra el estado de cada tarea al cierre del sprint. Las tareas del RESTful API y del login quedaron en To-do y pasaron al backlog, porque en el Sprint 2 cambiamos la prioridad hacia los bounded contexts core.

Sprint #	Sprint 1

User Story	**Work-Item / Task**

Sprint #	Sprint 1						
Story Id	Story Title	Task Id	Task Title	Task Description	Estimation (Hours)	Assigned To	Status
US48	Visualización de la landing page informativa	T01	Maquetar la sección hero y la propuesta de valor (HTML y CSS)	Maquetar el hero, la barra de navegación y la propuesta de valor según el mock-up.	6	Romero Veliz, Matthias Alonso	Done
US48	Visualización de la landing page informativa	T02	Implementar las secciones “Cómo funciona” y “Nosotros”	Agregar las secciones de pasos y del equipo.	5	Salazar Miranda, Mateo Paolo	Done
US49	Solicitud de demo desde la landing page	T03	Formulario de contacto con validación de correo	Crear el formulario de solicitud de demo y validar que el correo tenga un formato válido.	4	Romero Veliz, Matthias Alonso	Done

Sprint #	Sprint 1						
US50	Onboarding guiado para nuevo administrador	T04	Diseñar los pasos del tutorial guiado (wireframe)	Definir en Figma los pasos del recorrido guiado para un administrador nuevo.	3	Salazar Miranda, Mateo Paolo	Done
US01	Registro de administrador con validación de edificio	T05	Endpoint POST de registro de administrador (Spring Boot)	Crear el endpoint que registra al administrador y su edificio.	6	Romero Veliz, Matthias Alonso	To-do
US01	Registro de administrador con validación de edificio	T06	Formulario de registro en Angular con validaciones	Crear el formulario de registro con sus validaciones.	5	Diaz Vargas, Fernanda Ysabella	To-do
US02	Inicio de sesión por rol	T07	Endpoint POST de inicio de sesión con JWT	Crear el endpoint que valida credenciales y devuelve un token JWT.	6	Romero Veliz, Matthias Alonso	To-do

Sprint #	Sprint 1						
US02	Inicio de sesión por rol	T08	Pantalla de inicio de sesión y guard de rutas	Crear la vista de inicio de sesión y proteger las rutas con un guard.	5	Diaz Vargas, Fernanda Ysabella	To-do
US07	Registro de un edificio y sus datos generales	T09	Endpoint POST `/buildings`	Crear el endpoint que registra un edificio.	4	Romero Veliz, Matthias Alonso	To-do
US08	Registro de equipos críticos del edificio	T10	Endpoint POST `/equipment`	Crear el endpoint que registra un equipo crítico.	5	Romero Veliz, Matthias Alonso	To-do
—	Configuración transversal	T11	Agregar el selector de idioma y las animaciones del sitio	Agregar el cambio de idioma y las animaciones de entrada de las secciones.	3	Valladolid Jiménez, Arturo Fernando	Done

Sprint #	Sprint 1						
—	Configuración transversal	T12	Configurar y publicar el Landing Page en GitHub Pages	Activar GitHub Pages sobre la rama `main` y revisar el sitio publicado.	3	Valladolid Jiménez, Arturo Fernando	Done

5.2.1.4. Development Evidence for Sprint Review

En el Sprint 1 implementamos el Landing Page en el repositorio `landing-page`. Trabajamos directo sobre `main` y varias subidas se hicieron desde la web de GitHub. Eso dejó mensajes de commit poco claros. Lo corregimos desde el Sprint 2 con GitFlow y Conventional Commits (sección 5.1.2).

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
code-nova-1asi0729/landing-page	main	519b96d	feat: initial release of landing page website	Primera versión del sitio: estructura HTML, estilos y script de navegación.	17/09/2026
code-nova-1asi0729/landing-page	main	69bcdeb	Add files via upload	Agrega los extras del landing (`landing-extras.js`) y un primer prototipo de vistas de la aplicación.	18/09/2026

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
code-nova-1asi0729/landing-page	main	54dc142	Add files via upload	Vuelve a subir el sitio completo (HTML, CSS y JS) alineado al mock-up.	18/09/2026
code-nova-1asi0729/landing-page	main	2daa7ce	Add files via upload	Reorganiza el contenido de `index.html`.	18/09/2026
code-nova-1asi0729/landing-page	main	46d3c61	feat: add new team member to landing page	Tarjetas del equipo en la sección “Nosotros”.	18/09/2026
code-nova-1asi0729/landing-page	main	4532dcc	Add files via upload	Agrega imágenes y favicon, y ajusta el script de navegación.	18/09/2026
code-nova-1asi0729/landing-page	main	1f21b16	Add files via upload	Ajustes finales de estilos y contenido. Es la versión publicada al cierre del sprint.	18/09/2026

5.2.1.5. Execution Evidence for Sprint Review

Al cierre del sprint el Landing Page muestra la propuesta de valor, las funciones principales, los planes, el equipo y un formulario de contacto. Las capturas son del sitio publicado.

Figura 57: Figura 5.2.1-1 – Landing Page: hero y propuesta de valor

Figura 58: Figura 5.2.1-2 – Landing Page: funciones principales

Figura 59: Figura 5.2.1-3 – Landing Page: planes

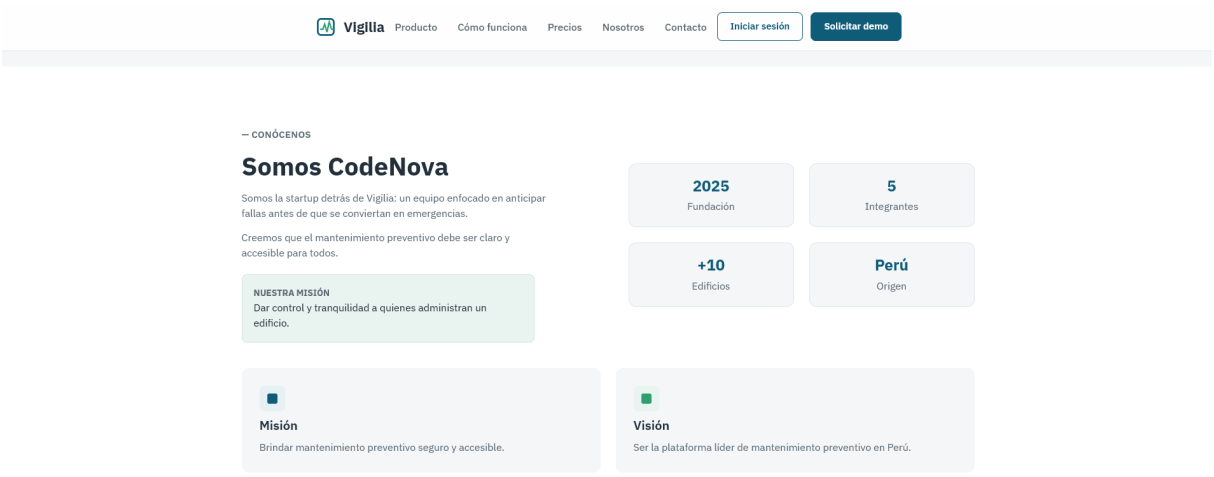


Figura 60: Figura 5.2.1-4 – Landing Page: nosotros

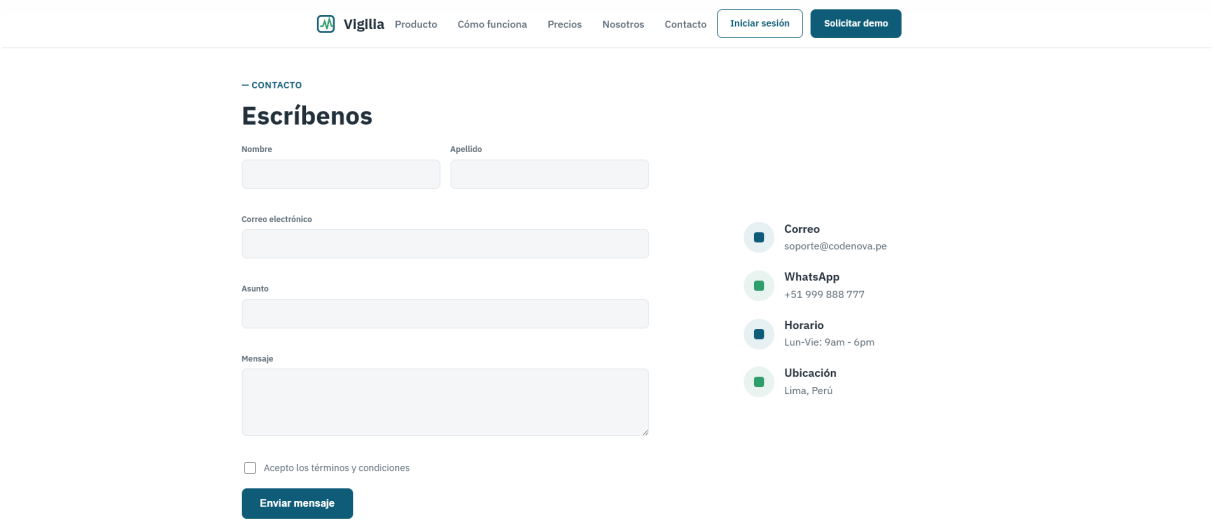


Figura 61: Figura 5.2.1-5 – Landing Page: formulario de contacto

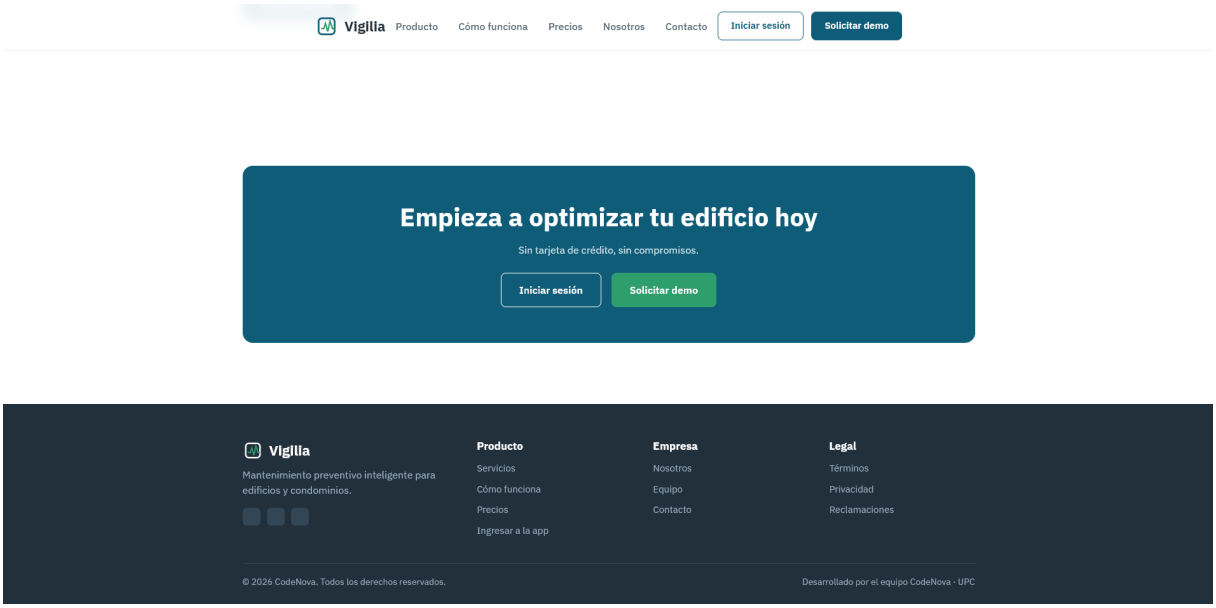


Figura 62: Figura 5.2.1-6 – Landing Page: llamado a la acción y footer

5.2.1.6. Services Documentation Evidence for Sprint Review

En el Sprint 1 no implementamos Web Services, así que no hay endpoints que documentar. El Landing Page es un sitio estático y no consume ningún API. La documentación de endpoints empieza en el Sprint 2 con el fake API (sección 5.2.2.6).

5.2.1.7. Software Deployment Evidence for Sprint Review

Publicamos el Landing Page en GitHub Pages. Los pasos fueron:

1. En el repositorio `landing-page`, entrar a *Settings* > *Pages*.
2. En *Build and deployment*, elegir *Deploy from a branch*, la rama `main` y la carpeta raíz.
3. Guardar y esperar a que GitHub termine el despliegue.
4. Abrir la URL publicada y revisar cada sección.

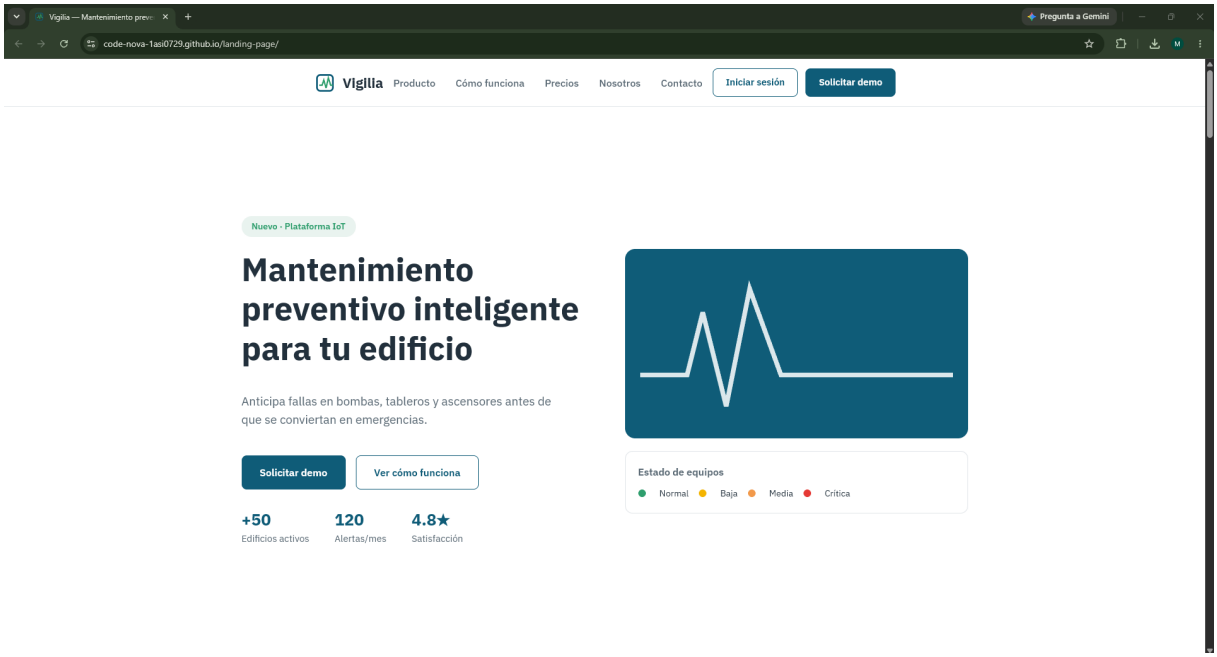


Figura 63: Figura 5.2.1-7 – Configuración de GitHub Pages del Landing Page

URL del sitio: https://code-nova-1asi0729.github.io/landing-page/

5.2.1.8. Team Collaboration Insights during Sprint

Cada integrante lideró un aspecto del Landing Page y revisó el trabajo de los demás. Diseñamos primero en Figma y luego pasamos a HTML y CSS. Al inicio costó que lo implementado se viera igual que el mock-up. Lo resolvimos revisando juntos cada sección antes de subirla.

Lo que no salió bien fue el manejo del repositorio. Trabajamos sobre `main`, sin ramas `feature`, y varias subidas se hicieron arrastrando archivos en la web de GitHub. Por eso hay commits como “Add files via upload” o “Delete app directory”. En el Sprint 2 cambiamos a GitFlow, con Pull Requests y Conventional Commits.

5.2.2. Sprint 2

En el Sprint 2 construimos la primera versión de la Frontend Web Application y una nueva versión del Landing Page. La aplicación cubre los dos bounded contexts core que el administrador y el residente usan todos los días: Asset Monitoring (edificios, equipos, sensores, lecturas y alertas) e Incidents. Como el RESTful API se construye en el Sprint 3, la aplicación consume un fake API con los mismos endpoints.

5.2.2.1. Sprint Planning 2

Antes de planificar revisamos qué salió del Sprint 1 y la retroalimentación del AV1. La decisión más importante fue cambiar el orden del backlog. En el Sprint 1 habíamos planificado el registro y el inicio de sesión, pero IAM es un bounded context genérico: no es lo que hace valiosa a Vigilia. Por eso lo movimos al final del Product Backlog (sección 3.3) y en este sprint empezamos por lo que el administrador necesita ver cada día: sus edificios, sus equipos y las alertas.

Sprint #	Sprint 2
Sprint Planning Background	
Date	2026-10-02
Time	07:30 PM
Location	Reunión virtual en Google Meet
Prepared By	Valladolid Jiménez, Arturo Fernando
Attendees (to planning meeting)	Valladolid Jiménez, Arturo Fernando / Diaz Vargas, Fernanda Ysabella / Romero Veliz, Matthias Alonso / Salazar Miranda, Mateo Paolo

Sprint #	Sprint 2
Sprint 1 Review Summary	Publicamos la primera versión del Landing Page en GitHub Pages, con la propuesta de valor, las funciones, los planes, el equipo y el formulario de demo. Las historias del Landing Page (US48 y US49) quedaron terminadas. Las tareas del RESTful API y del login no se empezaron. En la revisión del AV1 recibimos tres observaciones: priorizar los bounded contexts core antes que IAM, corregir el Container Diagram para separar el contenido estático de la aplicación que corre en el navegador, y llevar el informe a un repositorio con GitFlow.
Sprint 1 Retrospective Summary	Lo que salió bien: cada integrante lideró una parte del Landing Page y lo entregamos a tiempo. Lo que debemos mejorar: trabajamos el Landing Page directo en `main` y con subidas desde la web de GitHub, el informe estaba en un documento compartido sin historial claro y planificamos más historias de las que podíamos terminar. Acordamos usar GitFlow con Pull Requests y Conventional Commits en todos los repositorios, pasar el informe a Markdown y tomar menos historias, pero terminarlas.

Sprint Goal & User Stories

Sprint #	Sprint 2
Sprint 2 Goal	Nuestro foco es que el administrador vea el estado de sus equipos y las alertas activas de su edificio, revise el historial de lecturas de un equipo y atienda los incidentes que reportan los residentes. Creemos que esto le permite decidir qué equipo atender primero y responder a los residentes sin depender de llamadas o de grupos de WhatsApp. Lo confirmaremos cuando el administrador identifique la alerta más severa en menos de 30 segundos desde que abre la aplicación, y cuando un residente reporte un incidente y lo vea en estado “Reportado” en menos de 2 minutos.
Sprint 2 Velocity	30 story points
Sum of Story Points	29 (US07: 2, US08: 3, US09: 3, US10: 3, US11: 2, US14: 3, US19: 3, US20: 2, US23: 3, US25: 3, US27: 2)

No incluimos la recepción de lecturas (US12), la generación automática de alertas (US17 y US18) ni la detección de sensores desconectados (US13). Son reglas que corren en el RESTful API. En este sprint las lecturas las envía el simulador de sensores al fake API, y las alertas de prueba ya vienen en los datos iniciales.

5.2.2.2. Aspect Leaders and Collaborators

Dividimos el sprint en siete aspectos. Cinco son partes de la Web Application y siguen los bounded contexts. Los otros dos son la nueva versión del Landing Page y el despliegue. Cada líder implementó su aspecto en su propia rama `feature` y los colaboradores revisaron el Pull Request antes del merge.

Team Member (Last Name, First Name)	GitHub Username	Base y Edificios	Equipos Críticos	Alertas	Sensores y Lecturas	Incidentes	Landing Page v2	Despliegue
Valladolid Jiménez, Arturo Fernando	artuvall	L	C	L	C	C	C	L
Diaz Vargas, Fernanda Ysabella	Fern9901	C	C	C	L	C	C	C
Romero Veliz, Matthias Alonso	AlonsoVelizUpc	C	L	C	C	C	C	C
Salazar Miranda, Mateo Paolo	mateossm	C	C	C	C	L	L	C

5.2.2.3. Sprint Backlog 2

El objetivo del sprint es que el administrador vea el estado de sus equipos y alertas, y que el residente reporte incidentes. Partimos cada historia en tareas pequeñas, siguiendo las capas de cada bounded context: domain, infrastructure, application y presentation. Las tareas sin historia son de configuración del proyecto, del Landing Page o del despliegue.

URL del tablero:

Sprint #	Sprint 2						
User Story		**Work-Item / Task**					
Story Id	Story Title	Task Id	Task Title	Task Description	Estimation (Hours)	Assigned To	Status
—	Configuración transversal	T01	Crear el proyecto Angular	Crear el proyecto con Angular 22, Angular Material y ngx-translate (en y es). Agregar el layout, el selector de idioma, Home y la vista 404.	4	Valladolid Jiménez, Arturo Fernando	Done

Sprint #	Sprint 2					
—	Configuración transversal	T02	Crear las clases base	Implementar `BaseEntity`, `BaseAssembler`, `BaseApiEndpoint`, `BaseApi` y `BaseForm` en `shared`.	3	Valladolid Jiménez, Arturo Fernando	Done
—	Configuración transversal	T03	Crear el fake API y el simulador	Crear `db.json` y `routes.json` para json-server con la base `/api/v1`, y el simulador que envía lecturas cada 10 segundos.	3	Valladolid Jiménez, Arturo Fernando	Done

Sprint #	Sprint 2						
US07	Registro de un edificio y sus datos generales	T04	Modelo e infraestructura de edificios	Crear la entidad `Building`, su assembler, su endpoint y `AssetMonitoringStore`.	3	Valladolid Jiménez, Arturo Fernando	Done
US07	Registro de un edificio y sus datos generales	T05	Vistas de edificios	Crear la lista con tabla, orden y paginación, y el formulario para registrar y editar edificios.	4	Valladolid Jiménez, Arturo Fernando	Done
US08	Registro de equipos críticos del edificio	T06	Modelo de equipos críticos	Crear `CriticalEquipment` y los enums `EquipmentType` y `EquipmentStatus`.	2	Romero Veliz, Matthias Alonso	Done

Sprint #	Sprint 2						
US08	Registro de equipos críticos del edificio	T07	Infraestructura y store de equipos	Crear el assembler y el endpoint de equipos, y agregar sus métodos a `AssetMonitoringApi` y `AssetMonitoringStore`.	4	Romero Veliz, Matthias Alonso	Done
US08	Registro de equipos críticos del edificio	T08	Formulario de equipos	Crear el formulario con edificio, tipo, ubicación, fecha de instalación (datepicker) y estado.	4	Romero Veliz, Matthias Alonso	Done

Sprint #	Sprint 2						
US11	Consulta del listado de equipos por edificio	T09	Lista de equipos	Crear la tabla de equipos con filtro por edificio.	3	Romero Veliz, Matthias Alonso	Done
US09	Edición o baja de un equipo registrado	T10	Editar y dar de baja	Reusar el formulario para editar y agregar `decommission()`, que cambia el estado a DECOMMISSIONED sin borrar el equipo.	2	Romero Veliz, Matthias Alonso	Done
US19	Visualización de alertas activas en el dashboard	T11	Modelo e infraestructura de alertas	Crear `Alert`, `AlertSeverity`, `AlertStatus`, su assembler y su endpoint.	3	Valladolid Jiménez, Arturo Fernando	Done

Sprint #	Sprint 2					
US19	VisualizaciónT12 de alertas activas en el dashboard	Alertas activas ordenadas	Agregar al store el computed `activeAlerts`, ordenado de mayor a menor severidad.	2	Valladolid Jiménez, Arturo Fernando	Done
US19	VisualizaciónT13 de alertas activas en el dashboard	Vista de alertas	Crear tarjetas con el conteo por severidad y una tabla con un color por nivel. Enlazar Home con esta vista.	4	Valladolid Jiménez, Arturo Fernando	Done

Sprint #	Sprint 2						
US20	Cambio de estado de una alerta	T14	Cambiar estado de alertas	Agregar los botones “En gestión” y “Resolver”, y el método `changeAlertStatus` del store.	2	Valladolid Jiménez, Arturo Fernando	Done
US10	Asociación de sensores a un equipo específico	T15	Modelo e infraestructura de sensores	Crear `Sensor` con `assignTo()` y `unassign()`, su assembler y su endpoint. Un sensor solo puede tener un equipo activo.	3	Diaz Vargas, Fernanda Ysabella	Done

Sprint #	Sprint 2						
US10	Asociación de sensores a un equipo específico	T16	Vistas de sensores	Crear la lista de sensores y el formulario para asignar un sensor a un equipo.	4	Diaz Vargas, Fernanda Ysabella	Done
US14	Consulta del historial de lecturas de un equipo	T17	Infraestructura de lecturas	Crear `SensorReading`, su assembler y su endpoint.	2	Diaz Vargas, Fernanda Ysabella	Done
US14	Consulta del historial de lecturas de un equipo	T18	Vista de historial de lecturas	Crear la vista con selector de equipo y rango de fechas. Mostrar “No hay lecturas registradas en este periodo” si no hay datos.	4	Diaz Vargas, Fernanda Ysabella	Done

Sprint #	Sprint 2						
US23	Reporte de incidente por parte del residente	T19	Modelo de incidentes	Crear el bounded context Incidents con `Incident`, `IncidentStatus` e `IncidentCategory`.	2	Salazar Miranda, Mateo Paolo	Done
US23	Reporte de incidente por parte del residente	T20	Infraestructura y store de incidentes	Crear `IncidentsApi`, su endpoint, su assembler e `IncidentsStore`.	3	Salazar Miranda, Mateo Paolo	Done
US23	Reporte de incidente por parte del residente	T21	Formulario de reporte	Crear el formulario con edificio, equipo opcional, categoría y descripción.	3	Salazar Miranda, Mateo Paolo	Done

Sprint #	Sprint 2						
US25	Seguimiento del estado de un incidente reportado	T22	Lista de incidentes	Crear la tabla de incidentes con los botones “Iniciar gestión” y “Resolver” según el estado.	3	Salazar Miranda, Mateo Paolo	Done
US27	Calificación del incidente resuelto	T23	Calificación del incidente	Crear el formulario de calificación de 1 a 5. Solo se califica una vez y solo si el incidente está resuelto.	3	Salazar Miranda, Mateo Paolo	Done
—	Landing Page	T24	Nueva versión del Landing Page	Actualizar el contenido y hacer que los botones de llamado a la acción lleven a la Web Application.	4	Salazar Miranda, Mateo Paolo	Done

Sprint #	Sprint 2						
—	Despliegue	T25	Desplegar el fake API	Crear el Web Service en Render con la carpeta `server`.	2	Valladolid Jiménez, Arturo Fernando	Done
—	Despliegue	T26	Desplegar la Web Application	Conectar el repositorio a Vercel y configurar `environment.ts` con la URL del fake API.	2	Valladolid Jiménez, Arturo Fernando	Done

5.2.2.4. Development Evidence for Sprint Review

En este sprint creamos el repositorio `Frontend` y trabajamos con GitFlow. La base del proyecto se hizo en `develop` y salió a `main` como versión 0.1.0. Desde ahí, cada aspecto tuvo su rama `feature` y entró a `develop` con un Pull Request. La tabla muestra los commits de implementación de cada repositorio.

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
code-nova-1asi0729/Frontend	develop	ecef535	chore: initialize Angular 22 project	—	07/10/2026

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
code-nova-1asi0729/Frontend	develop	79cab77	build(deps): add Angular Material	—	07/10/2026
code-nova-1asi0729/Frontend	develop	2149a70	build(deps): add ngx-translate and json-server	—	07/10/2026
code-nova-1asi0729/Frontend	develop	794ff3e	feat(shared): add base entity, assembler and api endpoint classes	—	07/10/2026
code-nova-1asi0729/Frontend	develop	70c9d07	feat(shared): add layout, language switcher, home and page not found views	—	07/10/2026
code-nova-1asi0729/Frontend	develop	268cf6c	feat(server): add fake API seed data and sensor simulator	—	07/10/2026
code-nova-1asi0729/Frontend	develop	6380b49	feat(asset-monitoring): add buildings list and form	—	07/10/2026

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
code-nova-1asi0729/Frontend	develop	0870228	build: add Vercel configuration	—	07/10/2026
code-nova-1asi0729/Frontend	develop	bfa9982	docs: add README, frontend class diagrams and sprint 2 tasks	—	07/10/2026
code-nova-1asi0729/Frontend	main	3359c43	Merge pull request #1 from code-nova-1asi0729/develop	Release 0.1.0: project base and buildings	07/10/2026
code-nova-1asi0729/Frontend	feature/asset-monitoring-equipment	d801f2f	add critical equipment domain, infrastructure and store	—	08/10/2026
code-nova-1asi0729/Frontend	feature/asset-monitoring-equipment	64f5787	feat(asset-monitoring): add equipment list and form views	—	08/10/2026
code-nova-1asi0729/Frontend	develop	e300cee	Merge pull request #3 from code-nova-1asi0729/feature/asset-monitoring-equipment	feat(asset-monitoring): add equipment list and form views	08/10/2026

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
code-nova-1asi0729/Frontend	feature/asset-monitoring-alerts	19b0fef	feat(asset-monitoring): add alert domain, infrastructure and store	—	08/10/2026
code-nova-1asi0729/Frontend	feature/asset-monitoring-alerts	6bddb8c	feat(asset-monitoring): add active alerts dashboard view	—	08/10/2026
code-nova-1asi0729/Frontend	feature/asset-monitoring-alerts	124ce46	feat(shared): link home button to the alerts dashboard	—	08/10/2026
code-nova-1asi0729/Frontend	feature/asset-monitoring-alerts	c60a37f	style(asset-monitoring): let the alerts table scroll on small screens	—	08/10/2026
code-nova-1asi0729/Frontend	develop	cf8356d	Merge pull request #4 from code-nova-1asi0729/feature/asset-monitoring-alerts	—	08/10/2026

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
code-nova-1asi0729/Frontend	feature/asset-monitoring-sensors	ff2b051	feat(asset-monitoring): add sensor and reading domain, infrastructure and store	—	08/10/2026
code-nova-1asi0729/Frontend	feature/asset-monitoring-sensors	ddd413e	feat(asset-monitoring): add sensor list and assign form views	—	08/10/2026
code-nova-1asi0729/Frontend	feature/asset-monitoring-sensors	652179a	feat(asset-monitoring): add reading history view	—	08/10/2026
code-nova-1asi0729/Frontend	feature/asset-monitoring-sensors	d76ede2	fix(server): keep the sensor simulator running while json-server reloads	—	08/10/2026
code-nova-1asi0729/Frontend	feature/asset-monitoring-sensors	cbf6f19	style(asset-monitoring): keep sensor serial numbers on one line	—	08/10/2026

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
code-nova-1asi0729/Frontend	develop	e949b10	Merge pull request #5 from code-nova-1asi0729/feature/asset-monitoring-sensors	—	08/10/2026
code-nova-1asi0729/Frontend	feature/incidents	4d96e2f	feat(incidents): add incident domain and infrastructure	—	08/10/2026
code-nova-1asi0729/Frontend	feature/incidents	4d927ec	feat(incidents): add incidents store	—	08/10/2026
code-nova-1asi0729/Frontend	feature/incidents	d9a4cb5	feat(incidents): add incident list, report and rating views	—	08/10/2026
code-nova-1asi0729/Frontend	develop	c0a76cc	Merge pull request #6 from code-nova-1asi0729/feature/incidents	—	08/10/2026
code-nova-1asi0729/Frontend	main	e675ddb	Merge pull request #7 from code-nova-1asi0729/develop	release: sprint 2 frontend	08/10/2026
code-nova-1asi0729/landing-page	main	c05402a	Add files via upload	Actualiza el contenido del Landing Page.	01/10/2026

Repository	Branch	Commit Id	Commit Message	Commit Message Body	Committed on (Date)
code-nova-1asi0729/landing-page	develop	12efd2f	feat(landing): link the landing page to the web app and remove request a demo	—	08/10/2026
code-nova-1asi0729/landing-page	main	73ca6fe	Merge pull request #1 from code-nova-1asi0729/develop	—	08/10/2026

5.2.2.5. Execution Evidence for Sprint Review

Al cierre del sprint la Web Application tiene siete opciones en el menú: Home, Buildings, Equipment, Alerts, Sensors, Readings e Incidents. Todas las vistas están en inglés por defecto y en español con el selector de idioma. Las capturas usan los datos de prueba del fake API.

En Home el administrador entra directo a sus alertas.

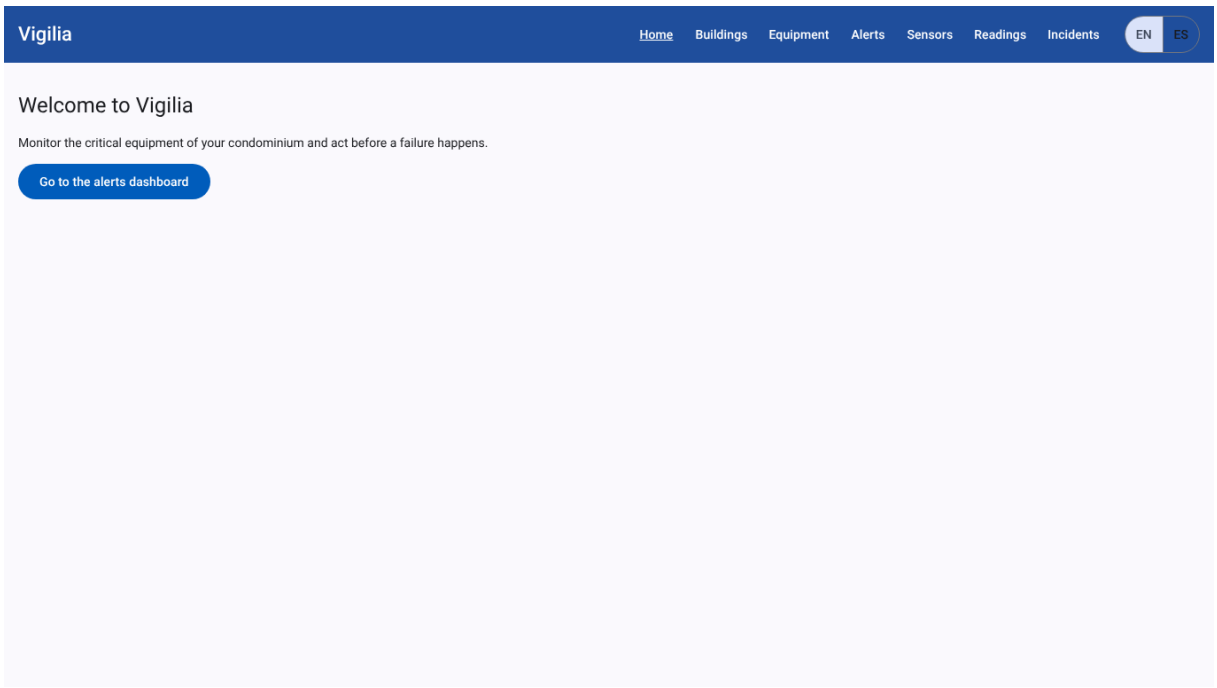


Figura 64: Figura 5.2.2-1 – Home de la Web Application

En Buildings el administrador lista, registra y edita sus edificios (US07).

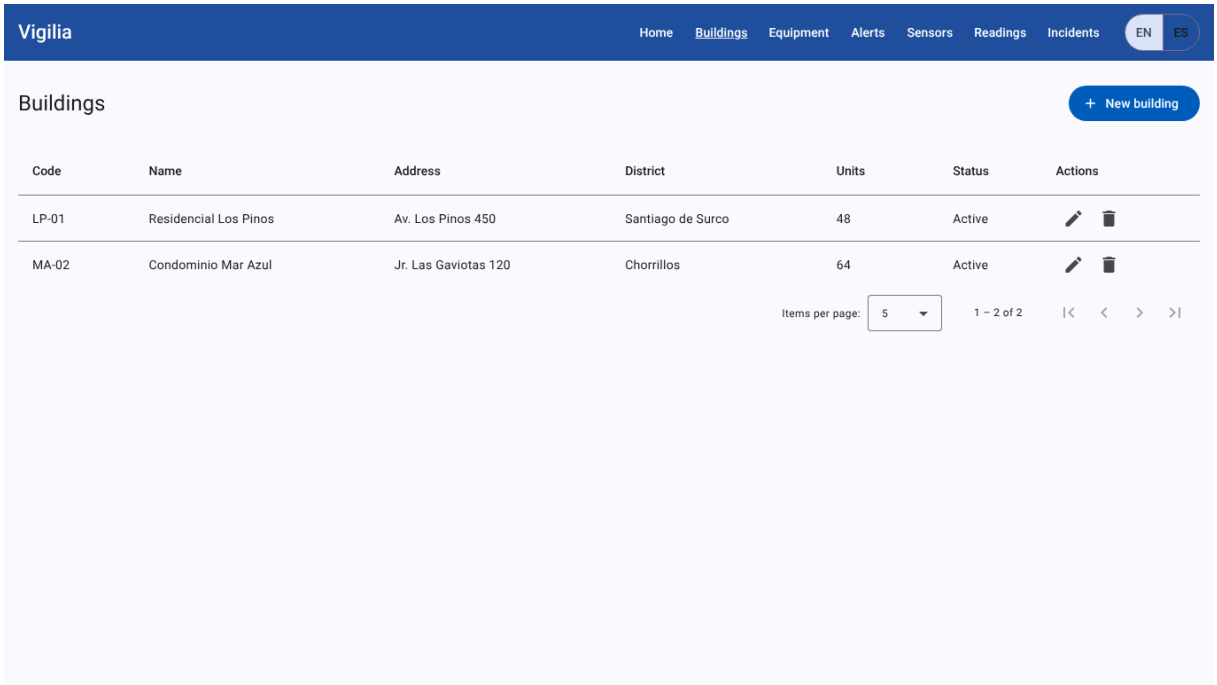


Figura 65: Figura 5.2.2-2 – Lista de edificios

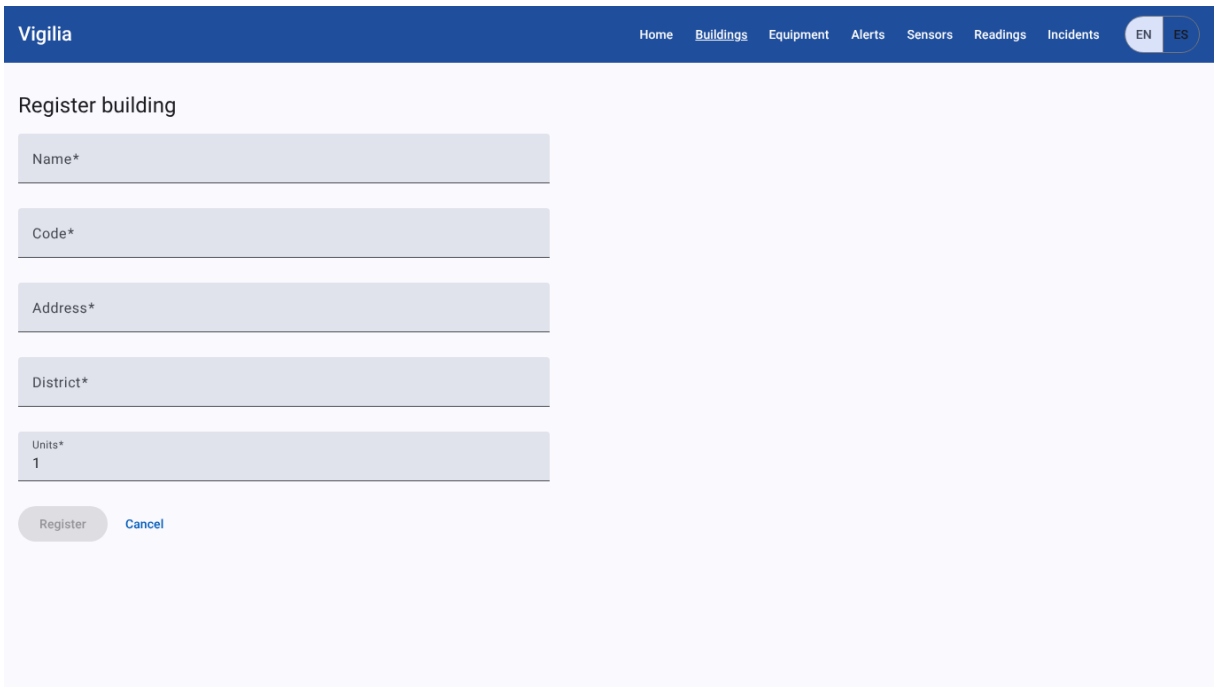


Figura 66: Figura 5.2.2-3 – Formulario de nuevo edificio

En Equipment ve los equipos críticos y los filtra por edificio (US11). Desde la misma tabla edita un equipo o lo da de baja (US08 y US09). El estado se muestra con un color: verde si está operativo y ámbar si requiere seguimiento.

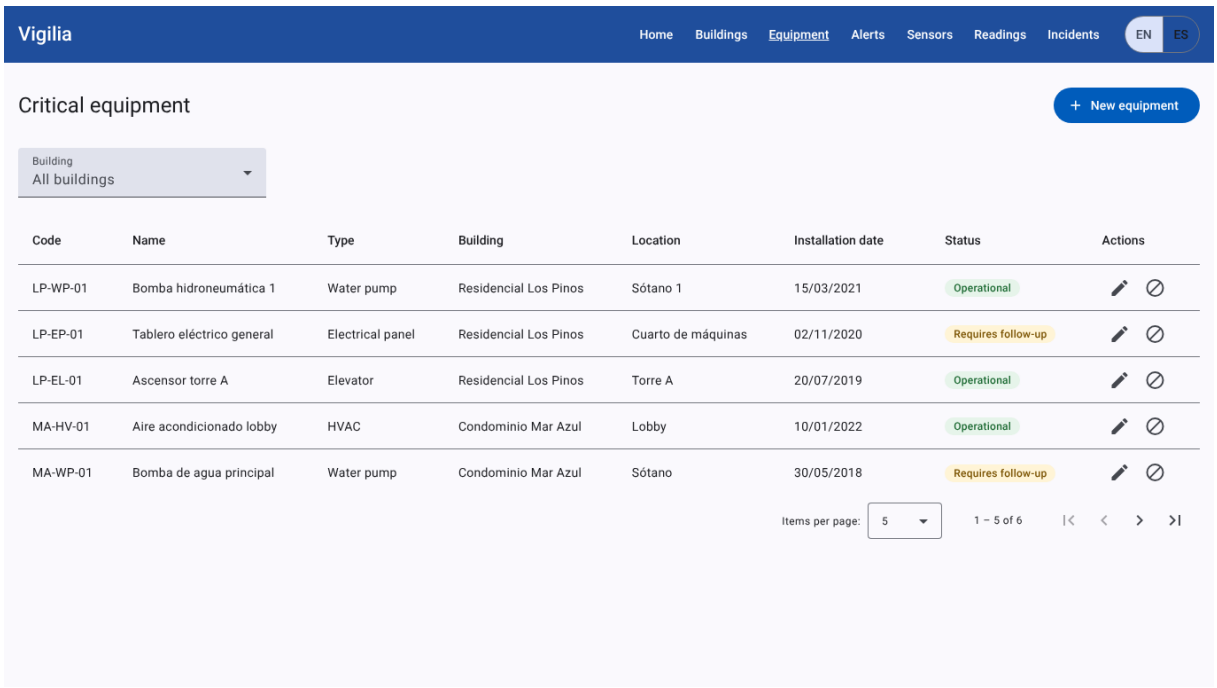


Figura 67: Figura 5.2.2-4 – Lista de equipos críticos

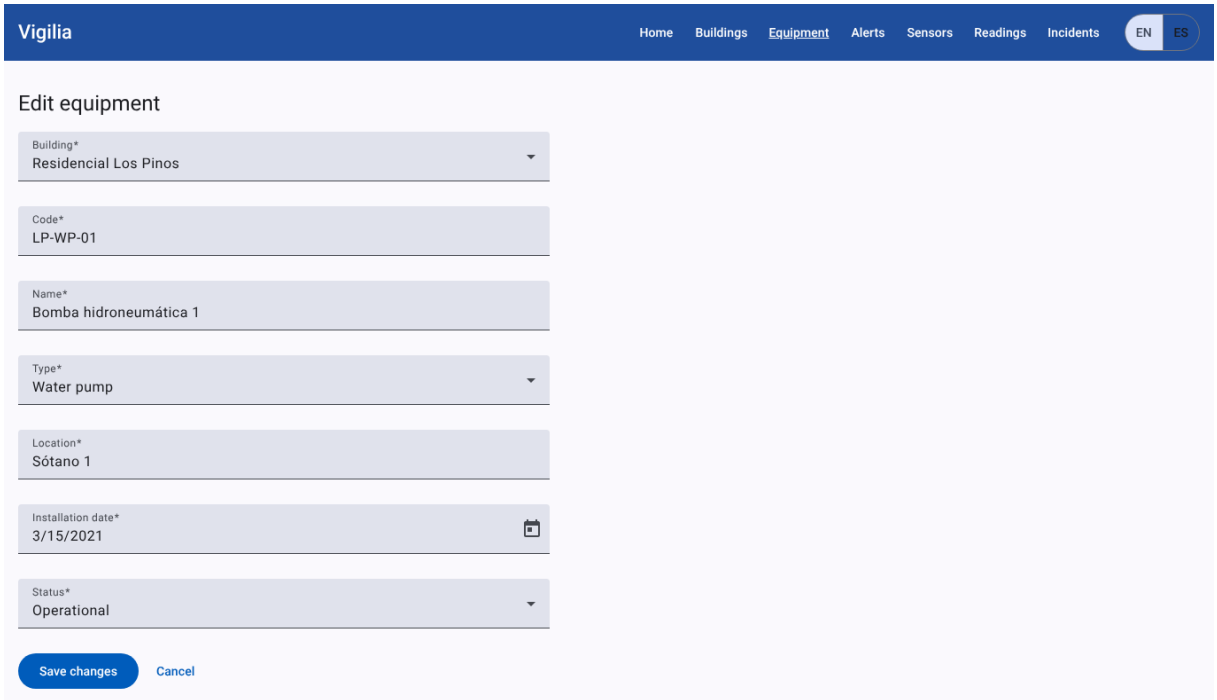


Figura 68: Figura 5.2.2-5 – Formulario de edición de un equipo

La misma vista con el idioma en español:

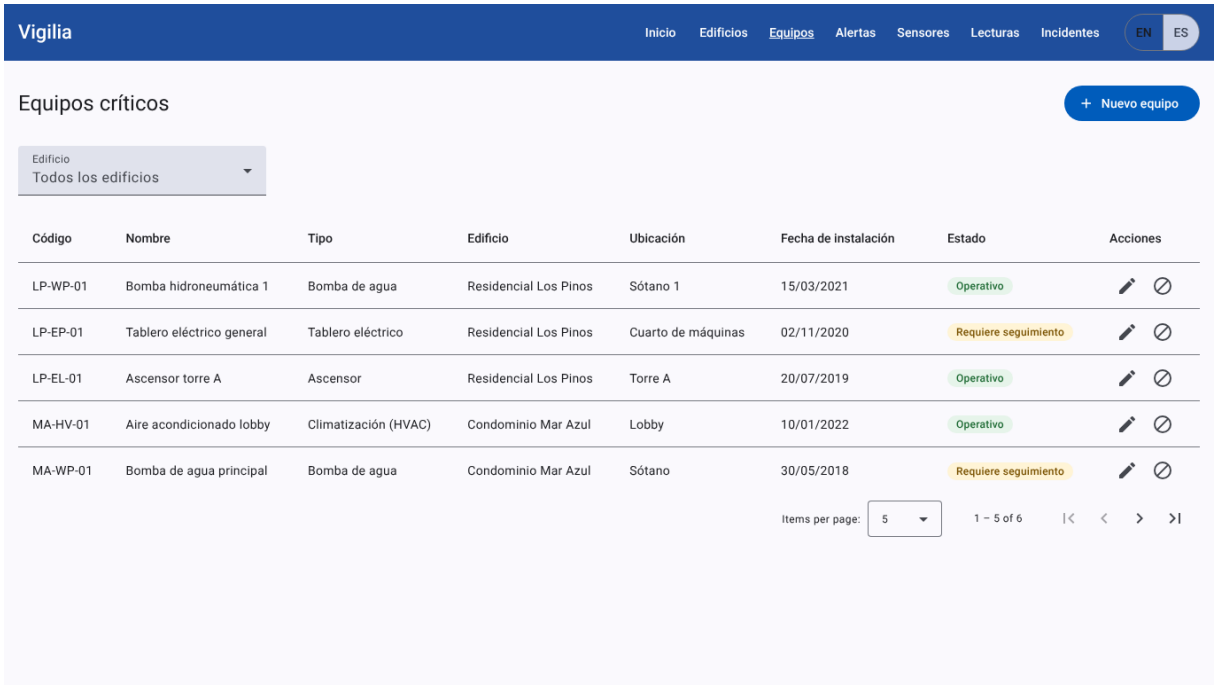


Figura 69: Figura 5.2.2-6 – Lista de equipos críticos en español

En Alerts el administrador ve cuántas alertas activas hay por severidad y una tabla ordenada de la más grave a la menos grave (US19). Desde la tabla marca una alerta como “In progress” (en gestión) o la resuelve (US20). Un interruptor muestra también las alertas resueltas.

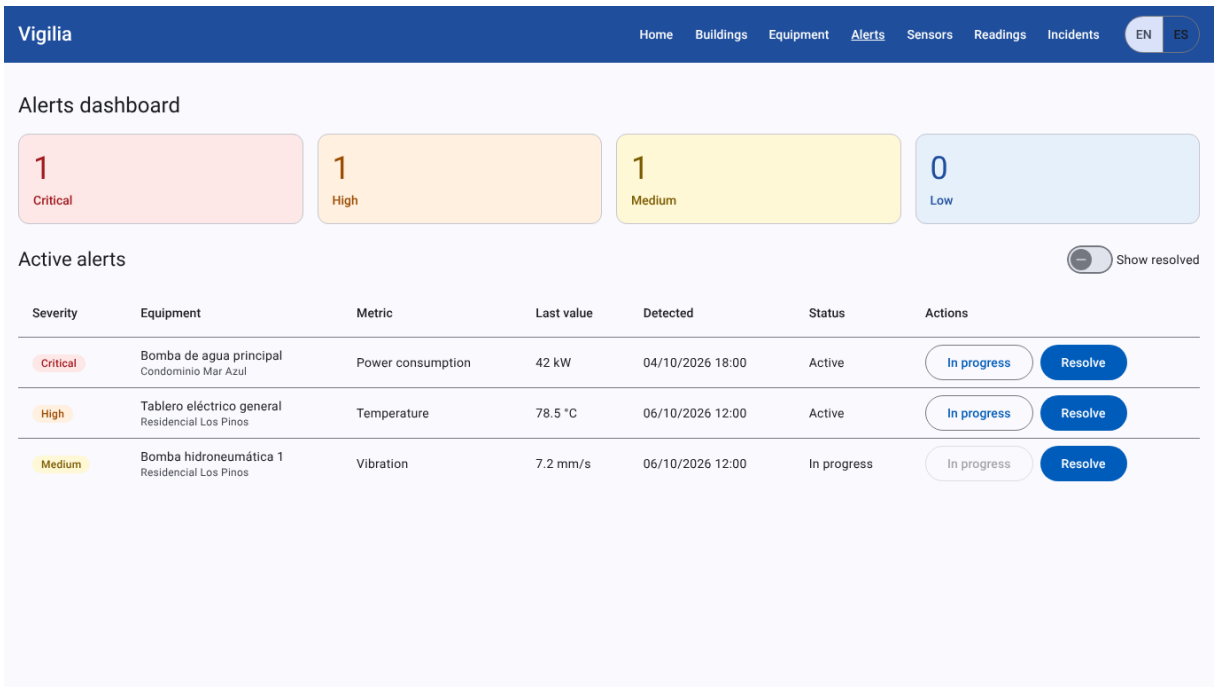


Figura 70: Figura 5.2.2-7 – Panel de alertas activas

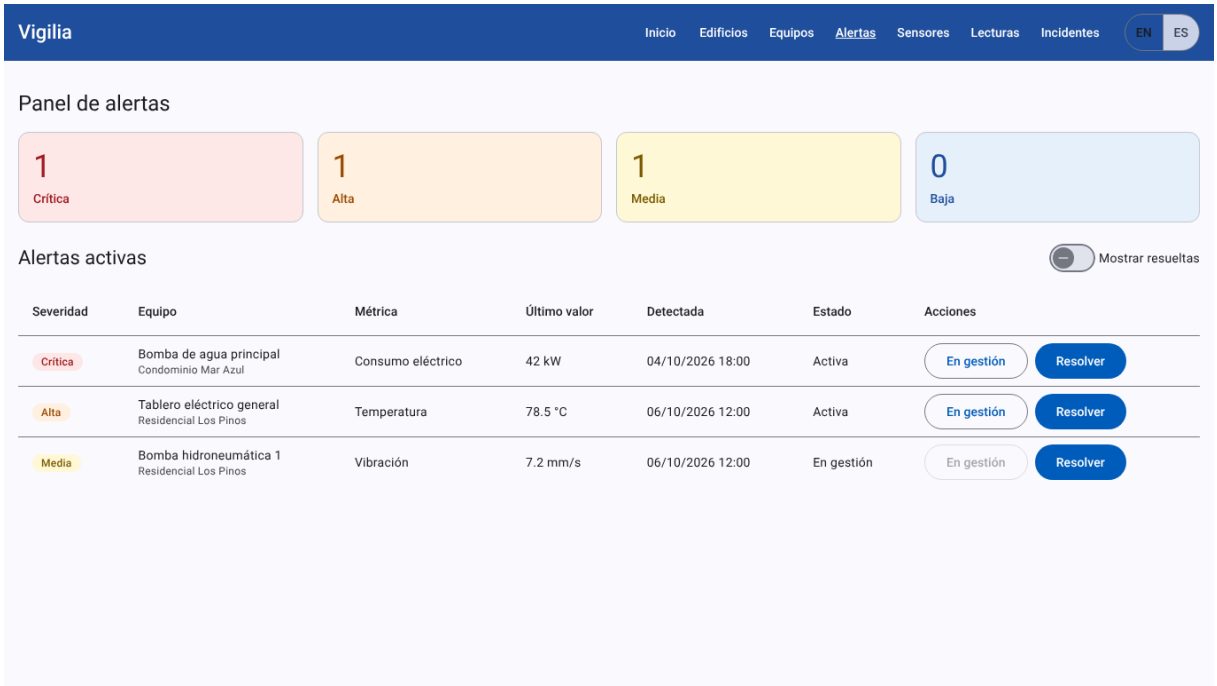


Figura 71: Figura 5.2.2-8 – Panel de alertas en español

En Sensors el administrador asigna un sensor libre a un equipo (US10). Un sensor que ya tiene equipo no se puede asignar a otro.

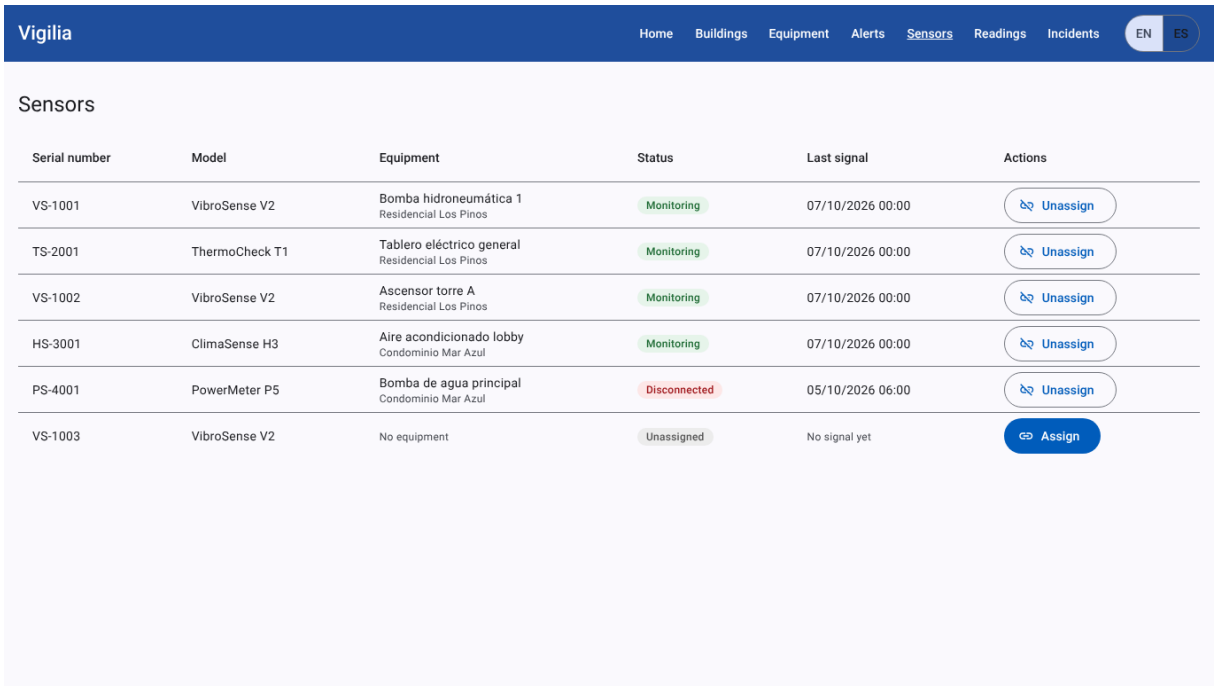


Figura 72: Figura 5.2.2-9 – Lista de sensores

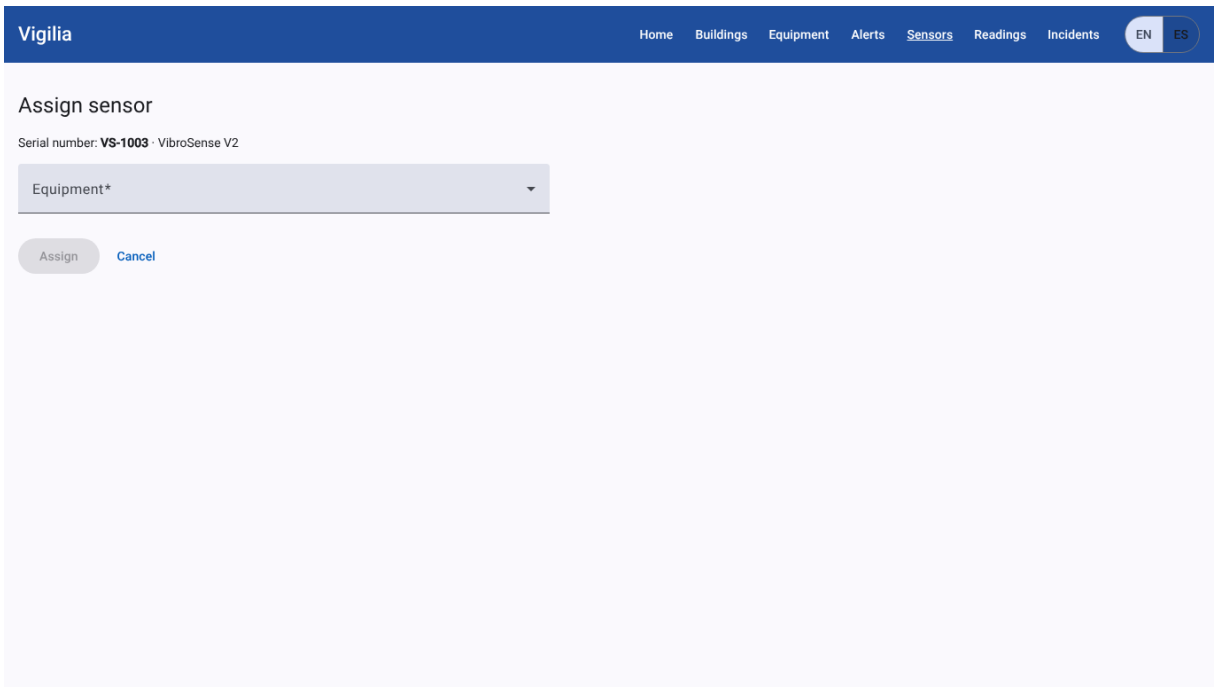


Figura 73: Figura 5.2.2-10 – Asignación de un sensor a un equipo

En Readings elige un equipo y un rango de fechas, y ve sus lecturas de la más reciente a la más antigua (US14). Con el simulador encendido, el botón “Actualizar” trae las lecturas nuevas.

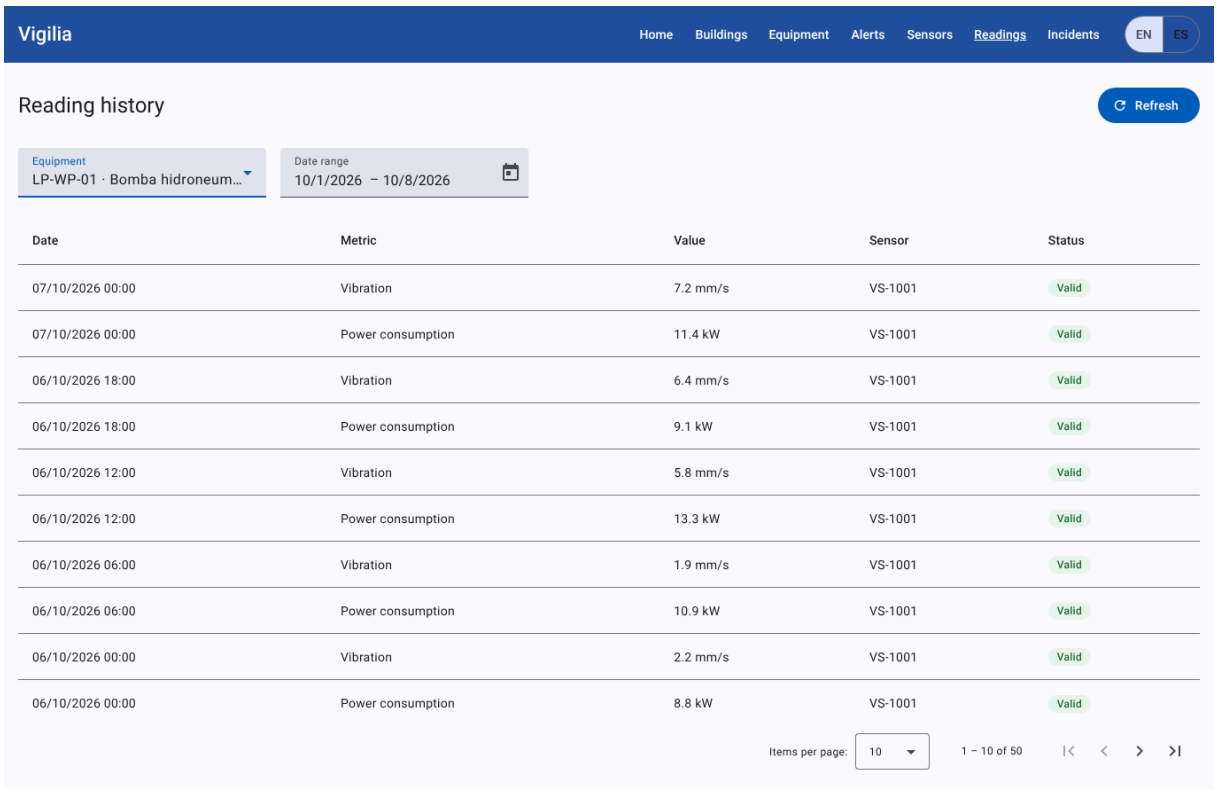


Figura 74: Figura 5.2.2-11 – Historial de lecturas de un equipo

En Incidents el residente reporta un incidente (US23). La administración cambia su estado de “Received” a “In progress” y luego a “Resolved” (US25). Mientras no hay IAM, el formulario usa un residente de prueba fijo. Cuando está resuelto, el residente lo califica de 1 a 5 (US27).

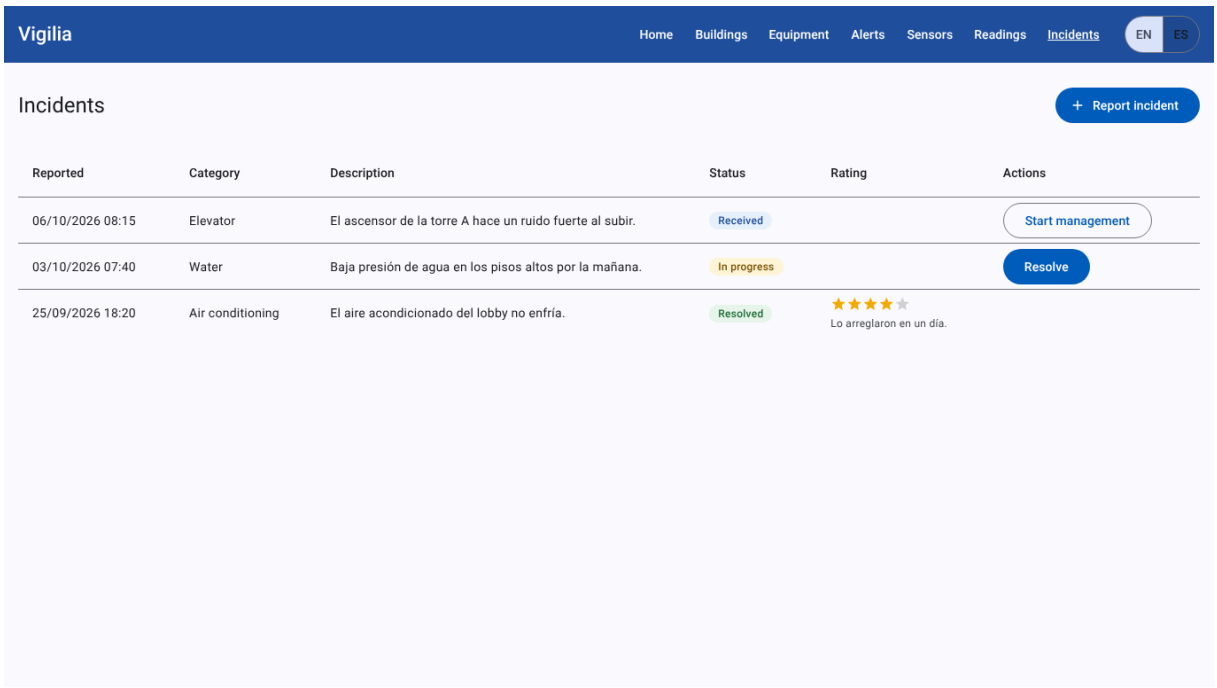


Figura 75: Figura 5.2.2-12 – Lista de incidentes

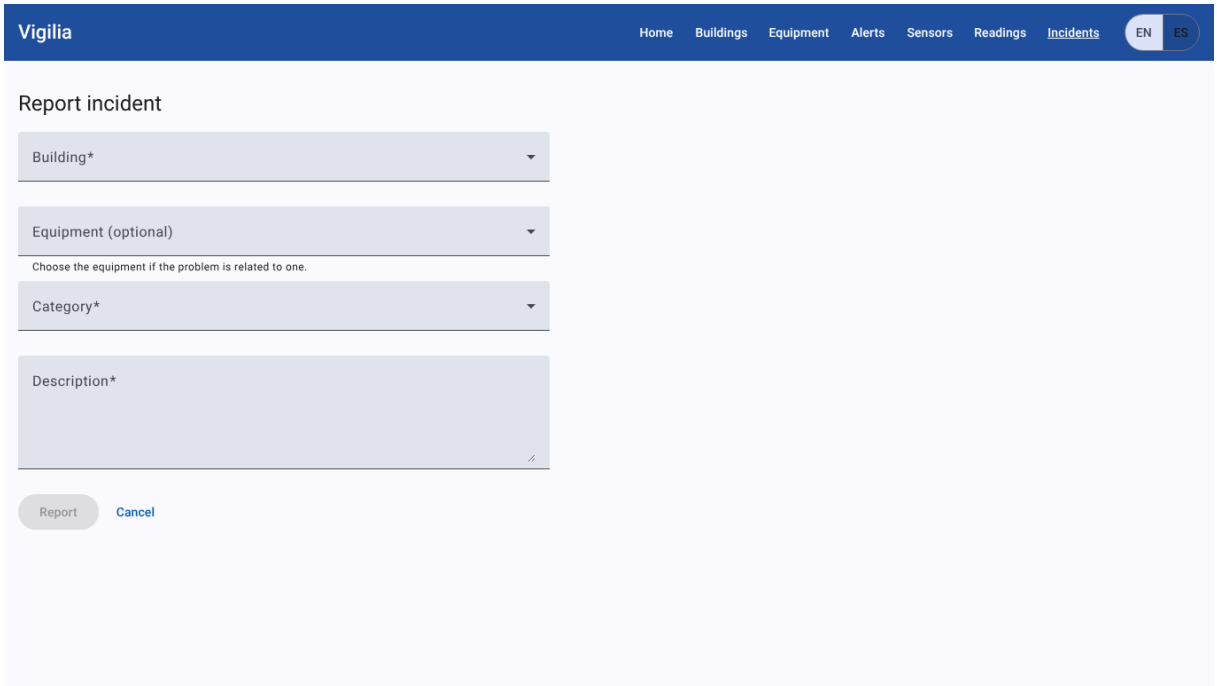


Figura 76: Figura 5.2.2-13 – Formulario de reporte de incidente

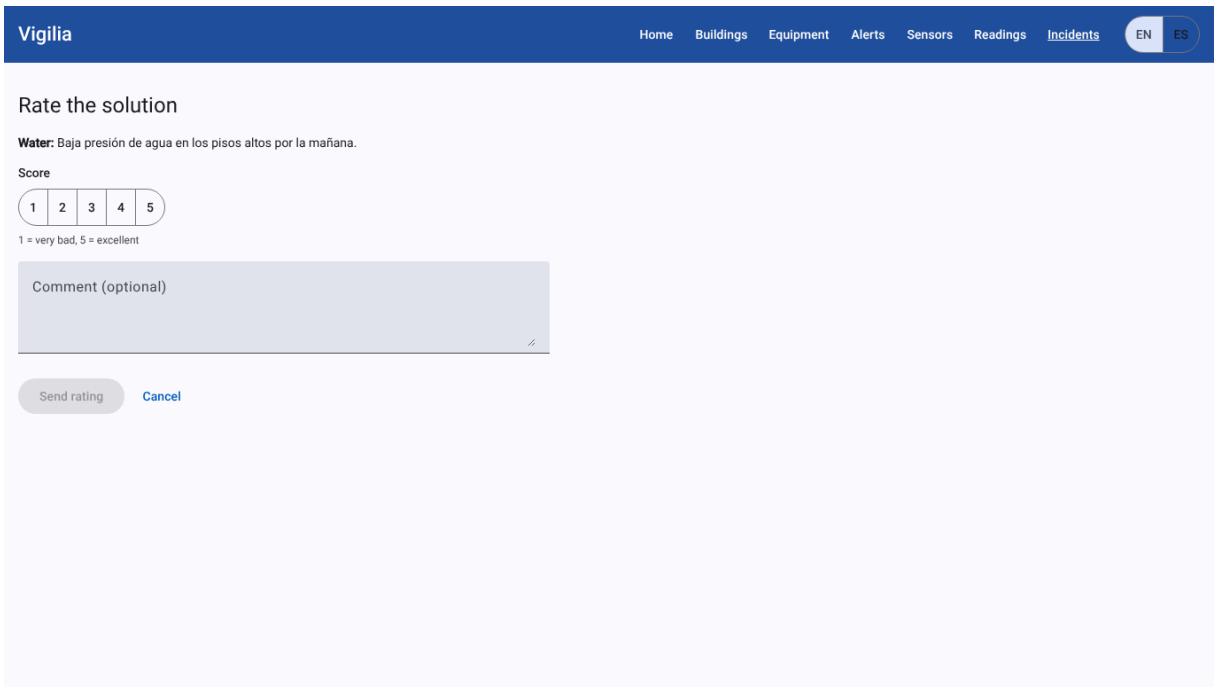


Figura 77: Figura 5.2.2-14 – Calificación de un incidente resuelto

Si el usuario escribe una ruta que no existe, la aplicación muestra la vista 404 con un enlace a Home.

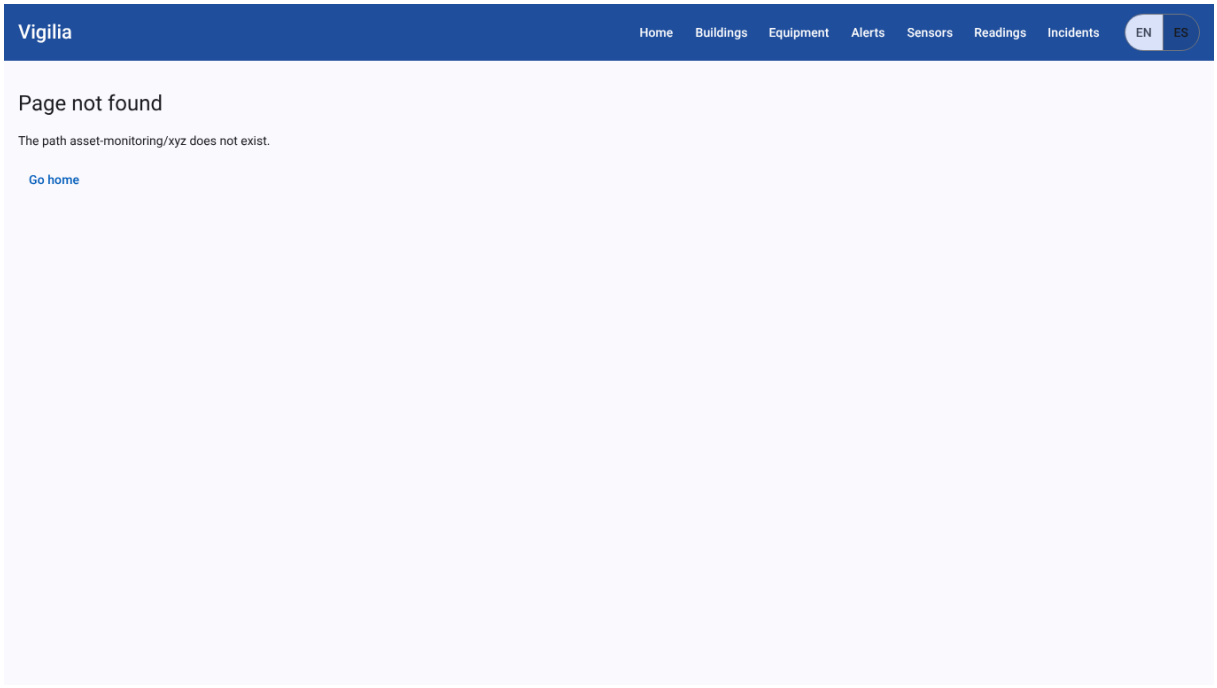


Figura 78: Figura 5.2.2-15 – Vista de página no encontrada

Video de navegación del Sprint 2:

5.2.2.6. Services Documentation Evidence for Sprint Review

En este sprint todavía no hay RESTful API, así que no hay documentación OpenAPI. Lo que sí documentamos son los endpoints del fake API que consume la Web Application. Son los mismos recursos, verbos y nombres de campos que tendrá el RESTful API en el Sprint 3 (sección 4.6.4). Así el cambio al API real solo afecta la URL base.

El fake API usa json-server 0.17.4. La regla de `routes.json` (`"/api/v1/*": "/$1"`) expone todos los recursos bajo `/api/v1`. Está desplegado en https://vigilia-fake-api.onrender.com/api/v1 y en local corre en http://localhost:3000/api/v1. El código está en la carpeta `server` del repositorio Frontend (commit `268cf6c`).

Endpoint	Acción	Verbo HTTP	Sintaxis de llamada	Parámetros	Ejemplo de response
Buildings	Listar edificios	GET	`/api/v1/buildings`	—	`[{"id":1,"name":"Res` Los Pinos`","code":"LP-01","district":"Sant` de Sur-co`","totalUnits":48,"`
Buildings	Registrar un edificio	POST	`/api/v1/buildings`	Body: `name`, `code`, `address`, `district`, `totalUnits`, `status`	El edificio creado con su `id`. Código 201.
Buildings	Editar un edificio	PUT	`/api/v1/buildings/{id}`	`id` en la ruta. Body: el edificio completo	El edificio actualizado. Código 200.
Buildings	Eliminar un edificio	DELETE	`/api/v1/buildings/{id}`	`id` en la ruta	`{}`. Código 200.

Endpoint	Acción	Verbo HTTP	Sintaxis de llamada	Parámetros	Ejemplo de response
Equipment	Listar equipos	GET	`/api/v1/equipment`	Opcional: `buildingId`	`[{"id":1,"buildingId"` `WP-01","type":"WATER_PUM` `03-15","status":"OPERATI`
Equipment	Registrar un equipo	POST	`/api/v1/equipment`	Body: `buildingId`, `code`, `name`, `type`, `location`, `installationDate` (`yyyy-MM-dd`), `status`	El equipo creado con su `id`. Código 201.
Equipment	Editar o dar de baja un equipo	PUT	`/api/v1/equipment/{id}`	`id` en la ruta. Para dar de baja se envía `status: DECOMMISSIONED`	El equipo actualizado. Código 200.
Sensors	Listar sensores	GET	`/api/v1/sensors`	Opcional: `status`, `equipmentId`	`[{"id":1,"equipmentId` `1001","status":"MONIT` `10-07T00:00:00Z"}]`
Sensors	Asignar o desasignar un sensor	PUT	`/api/v1/sensors/{id}`	`id` en la ruta. Body con `equipmentId` (o `null`) y `status`	El sensor actualizado. Código 200.

Endpoint	Acción	Verbo HTTP	Sintaxis de llamada	Parámetros	Ejemplo de response
Sensor Readings	Listar lecturas	GET	`/api/v1/sensor-readings`	Opcional: `equipmentId`, `_sort`, `_order`, `_limit`	`[{"id":1,"sensorId":1 09-30T06:00:00Z"}]`
Sensor Readings	Registrar una lectura (simulador)	POST	`/api/v1/sensor-readings`	Body: `sensorId`, `equipmentId`, `metric`, `value`, `unit`, `status`, `recordedAt`	La lectura creada con su `id`. Código 201.
Alerts	Listar alertas	GET	`/api/v1/alerts`	Opcional: `status`, `equipmentId`	`[{"id":1,"equipmentId`
Alerts	Cambiar el estado de una alerta	PUT	`/api/v1/alerts/{id}/status`	`id` en la ruta. Body con `status`: ACKNOWLEDGED o RESOLVED	La alerta actualizada. Código 200.
Incidents	Listar incidentes	GET	`/api/v1/incidents`	Opcional: `buildingId`, `residentId`, `status`	`[{"id":1,"buildingId"`

Endpoint	Acción	Verbo HTTP	Sintaxis de llamada	Parámetros	Ejemplo de response
Incidents	Reportar un incidente	POST	`/api/v1/incidents`	Body con `buildingId`, `residentId`, `equipmentId` (opcional), `category`, `description`	El incidente creado con estado REPORTED. Código 201.
Incidents	Cambiar estado o calificar	PUT	`/api/v1/incidents/{id}`	`id` en la ruta. Body con `status`, fechas del cambio y, al calificar, `ratingScore` y `ratingComment`	El incidente actualizado. Código 200.

Estas capturas muestran respuestas del fake API con los datos de prueba.

`GET /api/v1/buildings` devuelve los dos edificios de prueba:

Pretty-print ☐


```
[
  {
    "id": 1,
    "name": "Residencial Los Pinos",
    "code": "LP-01",
    "address": "Av. Los Pinos 450",
    "district": "Santiago de Surco",
    "totalUnits": 48,
    "status": "ACTIVE"
  },
  {
    "id": 2,
    "name": "Condominio Mar Azul",
    "code": "MA-02",
    "address": "Jr. Las Gaviotas 120",
    "district": "Chorrillos",
    "totalUnits": 64,
    "status": "ACTIVE"
  }
]
```


Figura 79: Figura 5.2.2-16 – Respuesta de GET /api/v1/buildings

`GET /api/v1/equipment?buildingId=1` devuelve solo los equipos del edificio 1. Es el filtro que usa la lista de equipos:

Pretty-print ☐


```
[
  {
    "id": 1,
    "buildingId": 1,
    "code": "LP-WP-01",
    "name": "Bomba hidroneumática 1",
    "type": "WATER_PUMP",
    "location": "Sótano 1",
    "installationDate": "2021-03-15",
    "status": "OPERATIONAL"
  },
  {
    "id": 2,
    "buildingId": 1,
    "code": "LP-EP-01",
    "name": "Tablero eléctrico general",
    "type": "ELECTRICAL_PANEL",
    "location": "Cuarto de máquinas",
    "installationDate": "2020-11-02",
    "status": "REQUIRES_FOLLOW_UP"
  },
  {
    "id": 3,
    "buildingId": 1,
    "code": "LP-EL-01",
    "name": "Ascensor torre A",
    "type": "ELEVATOR",
    "location": "Torre A",
    "installationDate": "2019-07-20",
    "status": "OPERATIONAL"
  }
]
```


Figura 80: Figura 5.2.2-17 – Respuesta de GET /api/v1/equipment?buildingId=1

`GET /api/v1/alerts?status=ACTIVE` devuelve solo las alertas activas:

Pretty-print ☐


```
[
  {
    "id": 1,
    "equipmentId": 2,
    "metric": "TEMPERATURE",
    "severity": "HIGH",
    "status": "ACTIVE",
    "lastValue": 78.5,
    "detectedAt": "2026-10-06T12:00:00Z"
  },
  {
    "id": 3,
    "equipmentId": 5,
    "metric": "POWER_CONSUMPTION",
    "severity": "CRITICAL",
    "status": "ACTIVE",
    "lastValue": 42,
    "detectedAt": "2026-10-04T18:00:00Z"
  }
]
```


Figura 81: Figura 5.2.2-18 – Respuesta de GET /api/v1/alerts?status=ACTIVE

`GET /api/v1/sensor-readings?equipmentId=1&_sort=recordedAt&_order=desc&_limit=3` devuelve las tres lecturas más recientes de un equipo:

Pretty-print ☐


```
[
  {
    "id": 28,
    "sensorId": 1,
    "equipmentId": 1,
    "metric": "VIBRATION",
    "value": 7.2,
    "unit": "mm/s",
    "status": "VALID",
    "recordedAt": "2026-10-07T00:00:00Z"
  },
  {
    "id": 56,
    "sensorId": 1,
    "equipmentId": 1,
    "metric": "POWER_CONSUMPTION",
    "value": 11.4,
    "unit": "kW",
    "status": "VALID",
    "recordedAt": "2026-10-07T00:00:00Z"
  },
  {
    "id": 27,
    "sensorId": 1,
    "equipmentId": 1,
    "metric": "VIBRATION",
    "value": 6.4,
    "unit": "mm/s",
    "status": "VALID",
    "recordedAt": "2026-10-06T18:00:00Z"
  }
]
```


Figura 82: Figura 5.2.2-19 – Respuesta de GET /api/v1/sensor-readings

5.2.2.7. Software Deployment Evidence for Sprint Review

En este sprint desplegamos tres cosas: el fake API, la Web Application y la nueva versión del Landing Page.

Fake API en Render.

1. Crear una cuenta en Render con GitHub.
2. Crear un *Web Service* conectado al repositorio `Frontend`.
3. Configurar `server` como *Root Directory*, `npm install` como *Build Command* y `npm start` como *Start Command*.
4. Elegir el plan gratuito y desplegar.
5. Probar `https://vigilia-fake-api.onrender.com/api/v1/buildings` en el navegador.

El plan gratuito apaga el servicio cuando no recibe tráfico. La primera solicitud después de un rato tarda unos segundos en responder.

Web Application en Vercel.

1. Crear una cuenta en Vercel con GitHub.
2. Importar el repositorio `Frontend`. Vercel reconoce el proyecto Angular y usa `ng build`.
3. Elegir `main` como rama de producción.
4. Agregar `vercel.json`, que redirige todas las rutas a `index.html`. Sin este archivo, recargar una ruta como `/asset-monitoring/equipment` da error 404.
5. Hacer el merge de `develop` a `main`. Vercel compila y publica la aplicación.

URL de la Web Application: https://vigilia-frontend.vercel.app/

Landing Page en GitHub Pages. La nueva versión se publicó con la misma configuración del Sprint 1 (sección 5.2.1.7). Ahora los botones “Go to app” y “Get started” llevan a la Web Application. También quitamos el modal de “Request a demo”: quien quiera hablar con el equipo usa el formulario de contacto, que cubre US49.

URL del Landing Page: https://code-nova-1asi0729.github.io/landing-page/

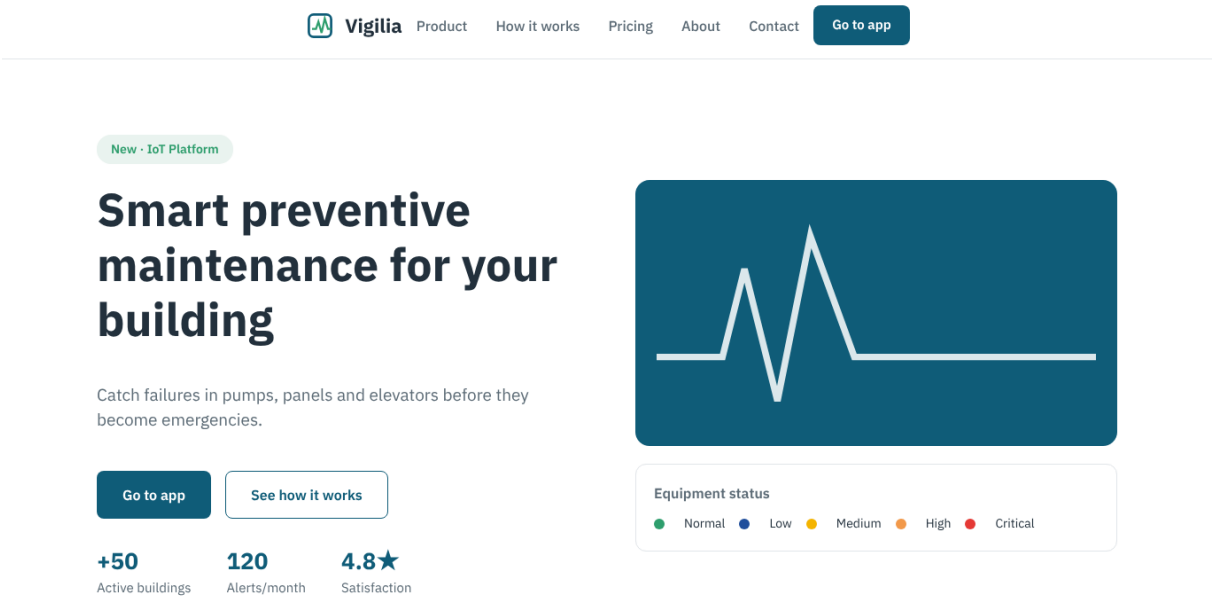


Figura 83: Figura 5.2.2-20 – Nueva versión del Landing Page con el botón “Go to app”

5.2.2.8. Team Collaboration Insights during Sprint

Este sprint fue el primero con GitFlow en todos los repositorios. Arturo creó la base del proyecto (clases base, layout, i18n, fake API y la vista de edificios) para que sirviera de ejemplo. También dejó en el repositorio una guía de tareas (`docs/sprint-2-tasks.md`) con los diagramas de clases de cada parte. Con esa base, cada integrante tomó un aspecto y lo trabajó en su propia rama: Matthias los equipos críticos, Arturo las alertas, Fernanda los sensores y el historial de lecturas, y Mateo los incidentes y el Landing Page.

La parte más difícil fue que varias tareas tocaban los mismos archivos: el store, el API y las rutas de Asset Monitoring, el menú y los archivos de traducción. Para evitar conflictos acordamos un orden de merge. Primero entró el modelo de equipos, porque las alertas y los sensores lo usan. Antes de abrir cada Pull Request, cada uno actualizaba su rama con `develop` y revisaba que `npm run build` terminara sin errores.

6. Conclusiones

- La mayoría de edificios de Lima hace mantenimiento reactivo: se llama al técnico cuando la bomba ya se quemó o el ascensor ya se detuvo. Las entrevistas lo confirmaron en los tres segmentos. Nadie tiene datos del estado de los equipos antes de la falla.
- El problema que planteamos en el Lean UX Process se sostiene. Las administradoras entrevistadas no tienen cómo mostrar a la junta lo que se ahorró por prevenir, y las empresas de mantenimiento priorizan por orden de llegada y no por gravedad.
- Los residentes no quieren datos técnicos. Les basta un canal claro para reportar y ver el estado de su reporte. Por eso el reporte de incidentes es simple y muestra solo tres estados.
- Ordenar el backlog por valor de negocio nos ayudó a enfocarnos. En el Sprint 2 construimos primero lo que el administrador usa cada día (edificios, equipos, sensores, lecturas y alertas) y el reporte de incidentes del residente. IAM quedó para el final, porque no diferencia a Vigilia.
- Usar los mismos cinco bounded contexts en el diseño, en el frontend y en el backend hizo que el informe y el código hablen el mismo idioma. Cada carpeta del frontend corresponde a un contexto del capítulo IV.
- Trabajar con un fake API con los mismos endpoints que tendrá el RESTful API nos permitió terminar la Web Application sin esperar al backend. En el Sprint 3 solo cambia la URL base.
- Pasar el informe a Markdown y usar GitFlow en todos los repositorios ordenó el trabajo del equipo. Ahora cada cambio tiene un autor, una rama y un Pull Request.

7. Recomendaciones

- Hacer las entrevistas de validación con administradores, residentes y empresas de mantenimiento usando la Web Application desplegada, para medir si cumplen sus tareas sin ayuda.
- Muchos cuartos de bombas y tableros están en sótanos con mala señal. Los sensores deberían guardar las lecturas y enviarlas cuando recuperen la conexión, para no perder datos.
- Construir el RESTful API empezando por Asset Monitoring e Incidents, para reemplazar el fake API sin cambiar las vistas.
- Mostrar mensajes simples en las alertas y reportes. Por ejemplo, “La bomba de agua tiene vibraciones fuera de lo normal; se recomienda una revisión preventiva” en lugar de lecturas técnicas.
- Antes de sumar más edificios, hacer pruebas de seguridad, rendimiento y usabilidad.

8. Referencias bibliográficas

CAPECO. 2025. *Informe Económico de la Construcción (Edición 86, febrero 2025)*. Cámara Peruana de la Construcción. https://iec.capeco.org/descargas/IEC86_0225.pdf.

Decreto Legislativo N.° 1568. 2023. *Decreto Legislativo del Régimen de la Propiedad Horizontal*. Diario Oficial El Peruano. https://busquedas.elperuano.pe/dispositivo/NL/2181939-6.

Instituto Nacional de Defensa Civil. 2025. *Compendio Estadístico - Gestión Reactiva 2025 (periodo 2003-2024)*. https://www.gob.pe/institucion/indeci/informes-publicaciones/7480334-compendio-estadistico-indeci-2025.

Ministerio de Economía y Finanzas. 2022. *Herramienta para la estimación de los costos de mantenimiento de proyectos de inversión*. https://www.mef.gob.pe/contenidos/inv_publica/docs/capacitaciones/Capacitacion_2022_08_19_1.pdf.

Ministerio de Energía y Minas. 2022. *Guía de Orientación del Uso Eficiente de la Energía - Sector Residencial*. https://eficienciaenergetica.minem.gob.pe/Content/fileman/Uploads/Documents/Gu%C3%ADas%202022/GUIA%20SECTOR%20RESIDENCIAL_VF3.pdf.

Samiria Soluciones. 2025. «Mantenimiento de bombas en edificios y condominios en Lima: frecuencia recomendada». https://samiriasoluciones.com/mantenimiento-bombas-edificios-condominios-frecuencia-lima/.

Anexos

Anexo A. Videos de Exposiciones

Video de Exposición – AV1

- **Nombre del archivo:** upc-pre-202620-1asi0729-16692-CodeNova-expo-av1.mp4
- **URL del video:** https://upcedupe-my.sharepoint.com/:v:/g/personal/u202420147_upc_edu_pe/IQC_FY9hlQVz
  b-BfCo?nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJPbmVEcml2ZUZvckJ1c2luZXNzIiwicmVmZX
- **Duración:** 07:26

Video de Exposición – TB1

- **Nombre del archivo:** upc-pre-202620-1asi0729-16692-CodeNova-expo-tb1.mp4
- **URL del video:** https://upcedupe-my.sharepoint.com/:v:/g/personal/u202420147_upc_edu_pe/IQC_pPkHHyr
  6ATmuJOPVnGILVa0wgTR8PVg?e=L89BeE&nav=eyJyZWZlcnJhbEluZm8iOnsicmVmZXJyYWxBcHAiOiJTdHJlYW
- **Duración:** 06:29